# Logica Matematica

APPUNTI DEL CORSO DI LOGICA MATEMATICA
TENUTO DAL PROF. MARCELLO MAMINO

Diego Monaco
d.monaco2@studenti.unipi.it

Daria Pasqualetti
inserisci.mail@esempio.it

Università di Pisa

Anno Accademico 2024-25

# Indice


**1 Introduzione** **5**

**I Semantica** **9**

**2 Formule, Strutture e Teorie** **10**
2.1 Formule . . . 10
2.2 Cosa significano le formule . . . 15
2.3 Sostituzioni (di $L$-termini al posto di variabili) . . . 21
2.4 Teorie . . . 26
2.5 Esempi di teorie . . . 27
2.6 PA e Q di Robinson . . . 30

**3 Eliminazione dei quantificatori, forme normali ed ultrafiltri** **33**
3.1 Eliminazione dei quantificatori . . . 33
3.2 CNF e DNF . . . 35
3.3 Ultraprodotti . . . 38
3.4 Teorema di Ax-Grothendieck . . . 42
3.5 Compattezza semantica e teorema di Łoś . . . 44
3.6 Applicazioni del teorema di compattezza . . . 47
3.7 Teoremi di Löwenheim-Skolem . . . 60
3.8 Categoricità e completezza . . . 64

**II Sintassi** **67**

**4 Deduzione naturale, teoremi di correttezza e completezza** **68**
4.1 Sistema di deduzione naturale (ND) . . . 68
4.2 Sistema ridotto ($\mathsf{ND}_{\to,\bot,\exists,=}$) . . . 69
4.3 Equivalenza tra $\vdash$ e $\models$ . . . 71
4.4 Teorema di correttezza . . . 74
4.5 Teorema di completezza . . . 77

**III Calcolabilità** **84**

**5 Teoria della calcolabilità** **85**
5.1 Funzioni primitivo ricorsive . . . 85
5.2 Codifica delle liste . . . 90
5.3 Funzioni ricorsive . . . 91
5.4 Macchine di Turing . . . 93
5.5 Funzioni Turing-computabili . . . 94
5.6 Teoremi di punto fisso e $s$-$m$-$n$ . . . 107

**6 Gerarchie aritmetiche e decidibilità** **112**
6.1 Insiemi decidibili, semidecidibili e ricorsivamente enumerabili . . . 112
6.2 Insiemi aritmetici . . . 116
6.3 Teorema di Rosser . . . 124

**7 Teoremi di incompletezza** **131**
7.1 Numerazione di Gödel . . . . . 131
7.2 Predicato di dimostrazione . . . . . 134
7.3 Primo teorema di incompletezza . . . . . 136
7.4 Secondo teorema di incompletezza . . . . . 139

## Premessa

Queste dispense sono la quasi esatta trascrizione in LaTeX delle dispense del corso di Logica Matematica [**?**], tenuto dal prof. Marcello Mamino nell'anno accademico 2023-24 presso l'Università di Pisa.

## Ringraziamenti

Antonio De Lucreziis.

Quest'opera è stata rilasciata con la licenza Creative Commons Attribuzione – Non commerciale 4.0 Internazionale. Per leggere una copia della licenza visita il sito web https://creativecommons.org/licenses/by-nc/4.0/deed.it.

# Parte I

# Semantica

# Parte II

# Sintassi

# Parte III
# Calcolabilità

# §5 Teoria della calcolabilità

## §5.1 Funzioni primitivo ricorsive

Vorremmo definire la classe delle funzioni implementabili in un calcolatore, tale classe è stata definita in vari modi, ad inizio '900 da Church, Turing, Post e altri, e si è poi visto che le varie definizioni date erano equivalenti, seppur magari non fosse quella l'intenzione originaria di chi le aveva date. Le principali definizioni che vediamo e che sono state date, sono:

1. tramite gli operatori di chiusura,
2. tramite un modello di calcolo (macchine di Turing),
3. tramite formule aritmetiche.

**Definizione 5.1** (Funzioni primitive ricorsive - PRF)**.** Definiamo la classe delle **funzioni primitive ricorsive** a partire da alcune funzioni di base e chiudendo la classe sotto alcune operazioni di chiusura. Le funzioni di base vanno da $\mathbb{N}^k \to \mathbb{N}$ per qualche $k \in \mathbb{N}$ e sono:

- $0_k : \mathbb{N}^k \to \mathbb{N}$, per ogni $k \in \mathbb{N}$, tale che $0_k(x_1, \dots, x_k) = 0_k(\mathbf{x}) = 0$,
- $s : \mathbb{N} \to \mathbb{N}$, la funzione successore, $s(x) = x + 1$,
- $\pi_{k,i} : \mathbb{N}^k \to \mathbb{N}$, per ogni $k \in \mathbb{N}$ e $1 \leq i \leq k$, tale che $\pi_{k,i}(x_1, \dots, x_k) = x_i$.

Chiudiamo queste funzioni di base sotto le seguenti operazioni, siano $f : \mathbb{N}^a \to \mathbb{N}$ e $g_1, \dots, g_a : \mathbb{N}^b \to \mathbb{N}$ primitive ricorsive, allora:

$$\mathbb{N}^b \to \mathbb{N} : (x_1, \dots, x_b) \mapsto f(g_1(x_1, \dots, x_b), \dots, g_a(x_1, \dots, x_b))$$

è una funzione primitiva ricorsiva, cioè chiudiamo sotto composizione; siano $h : \mathbb{N}^k \to \mathbb{N}$ e $g : \mathbb{N}^{k+2} \to \mathbb{N}$ primitive ricorsive, allora:

$$f : \mathbb{N}^{k+1} \to \mathbb{N} : \begin{cases} f(0, y_1, \dots, y_k) = h(y_1, \dots, y_k) \\ f(x+1, y_1, \dots, y_k) = g(x, f(x, y_1, \dots, y_k), y_1, \dots, y_k) \end{cases}$$

è primitiva ricorsiva, cioè chiudiamo sotto ricorsione.

**Osservazione 5.2** (Permutazione di primitive ricorsive) — Sia $f(x_1, \dots, x_k)$ primitiva ricorsiva, e sia $\sigma \in \mathrm{S}_k$ una permutazione, allora:

$$g(x_1, \dots, x_k) := f(x_{\sigma(1)}, \dots, x_{\sigma(k)})$$

è primitiva ricorsiva, infatti $g$ si ottiene da $f$ componendo con le proiezioni relative alla permutazione $\sigma$:

$$g(x_1, \dots, x_k) = f(\pi_{k,\sigma(1)}(x_1, \dots, x_k), \dots, \pi_{k,\sigma(k)}(x_1, \dots, x_k))$$

Anzi non è necessario che $\sigma$ sia una permutazione, basta che sia una qualsiasi funzione da $\{1, \dots, k\}$ a $\{1, \dots, k\}$.

**Notazione 5.3** (PR) — D'ora in avanti PR sarà l'abbreviazione di "primitivo ricorsivo/a".

Vediamo alcuni esempi di funzioni primitive ricorsive.

**Esempio 5.4** (Funzioni costanti)
Qualsiasi funzione costante $c_k : \mathbb{N}^k \to \mathbb{N}$ è primitiva ricorsiva, infatti $c_k$ si ottiene da $0_k$ componendo con la funzione successore $s$ un numero finito di volte:

$$c_k(x_1, \dots, x_k) = s^k \circ 0_k(x_1, \dots, x_k)$$

**Esempio 5.5** (Funzione somma)
La funzione somma $f(x, y) = x + y$ è primitiva ricorsiva, infatti è definita per ricorsione sulla $x$ a partire da $h(y) := \pi_{1,1}(y)$ e $g(x, z, y) := s \circ \pi_{3,2}(x, z, y)$:

$$f(x, y) := \begin{cases} f(0, y) = h(y) = \pi_{1,1}(y) = y \\ f(x + 1, y) = g(x, f(x, y), y) = s \circ \pi_{3,2}(x, f(x, y), y) = s(f(x, y)) \end{cases}$$

**Esempio 5.6** (Funzione prodotto)
La funzione prodotto $f(x, y) = x \cdot y$ è primitiva ricorsiva, infatti è definita per ricorsione sulla $x$ a partire da $h(y) := 0_1(y)$ e $g(x, y, z) := \pi_{3,2}(x, y, z) + \pi_{3,3}(x, y, z)$ (che è PR perché è composizione di funzioni PR):

$$f(x, y) := \begin{cases} f(0, y) = h(y) = 0_1(y) = 0 \\ f(x + 1, y) = g(x, f(x, y), y) = f(x, y) + y \end{cases}$$

**Esempio 5.7** (Funzione potenza)
La funzione potenza $f(x, y) = x^y$ è primitiva ricorsiva, infatti posso definire $f(x, y) := F(y, x)$, dove $F$ è definita per ricorsione usando $h(x) := s \circ 0_1(x) = c_1$ e $g(a, b, c) = \pi_{3,2}(a, b, c) \cdot \pi_{3,3}(a, b, c)$:

$$F(y, x) := \begin{cases} F(0, x) = h(x) = c_1(x) = 1 \\ F(y + 1, x) = g(y, F(y, x), x) = F(y, x) \cdot x \end{cases}$$

**Esempio 5.8** (Funzione predecessore)
Definiamo la funzione **predecessore** `pred` come:

$$\texttt{pred}(x) := \begin{cases} 0 & \text{se } x = 0 \\ x - 1 & \text{se } x > 0 \end{cases}$$

`pred` è primitiva ricorsiva, ed è definita per ricorsione da $h := 0_0$ e $g(x, z) := \pi_{2,1}(x, z)$:

$$\texttt{pred}(x) := \begin{cases} \texttt{pred}(0) = h(0) = 0_0(0) = 0 \\ \texttt{pred}(x + 1) = g(x, \texttt{pred}(x)) = \pi_{2,1}(x, \texttt{pred}(x)) = x \end{cases}$$

**Esempio 5.9** (Sottrazione troncata)
Definiamo la funzione **sottrazione troncata** $\dot{-}$ come:

$$x \dot{-} y := \begin{cases} 0 & \text{se } x \leq y \\ x - y & \text{se } x > y \end{cases}$$

$\dot{-}$ è primitiva ricorsiva, infatti è definita per ricorsione sulla $y$ a partire da $h(x) := \pi_{1,1}(x)$ e $g(a,b,c) := \texttt{pred} \circ \pi_{3,2}(a,b,c)$:

$$x \dot{-} y := \begin{cases} x \dot{-} 0 = h(x) = \pi_{1,1}(x) = x \\ x \dot{-} (y+1) = g(y, x \dot{-} y, x) = \texttt{pred} \circ \pi_{3,2}(y, x \dot{-} y, x) = \texttt{pred}(x \dot{-} y) \end{cases}$$

Notare come stia facendo ricorsione sull'altra variabile, e non su $x$, ma, come già visto, a meno di permutare, posso fare ricorsione su qualsiasi variabile, quindi è ininfluente.

**Definizione 5.10** (Predicato)**.** $P \subseteq \mathbb{N}^k$ è un **predicato PR** se esiste una funzione $f : \mathbb{N}^k \to \mathbb{N}$ PR tale che:

$$f(\mathbf{x}) = \begin{cases} 0 & \text{se } \mathbf{x} \in P \\ 1 & \text{se } \mathbf{x} \notin P \end{cases}$$

**Notazione 5.11** (Notazione per i predicati) — Eventualmente si confonderanno le notazioni $\mathbf{x} \in P$ e $P(\mathbf{x})$, intendendo con $P(\mathbf{x})$ la formula $f(\mathbf{x}) = 0$.

**Esempio 5.12** (Il predicato `zerop`)
Un esempio di predicato è $\texttt{zerop} := \{0\}$, infatti è definito dalla funzione PR:

$$f(x) := 1 \dot{-} (1 \dot{-} x)$$

infatti, se $\mathbb{N} \ni x > 1$ allora $1 \dot{-} x = 0$ e $1 \dot{-} 0 = 1$, per cui $x \notin \texttt{zerop}$; se $x = 1$ vale ancora $f(1) = 1$, mentre se $x = 0$ si ha $1 \dot{-} 0 = 1$ e $1 \dot{-} 1 = 0$, quindi $x \in \texttt{zerop}$.

**Osservazione 5.13** (Algebra di boole dei predicati PR) — I predicati PR sottoinsiemi dello stesso $\mathbb{N}^k$ formano un'algebra di boole.

*Dimostrazione.* Dati $P, Q \subseteq \mathbb{N}^k$ predicati PR devo verificare che i seguenti sono predicati PR: $P^{\complement}$, $P \cup Q$, $\mathbb{N}^k$[41].

$\boxed{\mathbb{N}^k}$ È un predicato prendendo la funzione PR data da $0_k$.

$\boxed{P^{\complement}}$ Sia $f$ la funzione PR che definisce $P$, allora $P^{\complement}$ è definito dalla funzione PR $1 \dot{-} f$.

$\boxed{P \cup Q}$ Siano $f$ e $g$ le funzioni PR che definiscono rispettivamente $P$ e $Q$, allora $P \cup Q$ è definito dalla funzione PR $(1 \dot{-} ((1 \dot{-} f) + (1 \dot{-} g)))$, oppure da $f \cdot g$.

□

[41] In questo modo ottengo che gli insiemi formano un'algebra sulle parti in senso classico, che corrisponde ad una $\mathbb{F}_2$-algebra sulle parti, detta anche algebra booleana.

**Esempio 5.14** (Insiemi di confronto)

Gli insiemi $\{x \leq y\}$, $\{x < y\}$, $\{x = y\}$, $\{x \neq y\}$[a] sono predicati PR. Per $\{x = y\}$ è sufficiente usare:

$$f(x, y) := 1 \dot{-} (1 \dot{-} (y \dot{-} x + x \dot{-} y))$$

Per $\{x \leq y\}$ si usa:

$$g(x, y) := 1 \dot{-} (1 \dot{-} (x \dot{-} y))$$

e $g(x, y) = 0$ se e solo se $x \leq y$. Mettendo assieme queste due cose si ottiene

$$\{x < y\} = \{x \leq y\} \cap \{x \neq y\}.$$

Simmetricamente per $\geq$ e $>$.

[a] Nel senso di $\{(x, y) \in \mathbb{N}^2 | \ldots\}$.

**Osservazione 5.15** (Definizione per casi) — Se $P \subseteq \mathbb{N}^k$ è un predicato PR, e $f, g : \mathbb{N}^k \to \mathbb{N}$ sono funzioni PR, allora la funzione definita per casi da:

$$h(\mathbf{x}) := \begin{cases} f(\mathbf{x}) & \text{se } \mathbf{x} \in P \\ g(\mathbf{x}) & \text{se } \mathbf{x} \notin P \end{cases}$$

è una funzione PR, infatti posso scrivere $h$ come: $h(\mathbf{x}) = f(\mathbf{x}) \cdot (1 \dot{-} P(\mathbf{x})) + g(\mathbf{x}) \cdot P(\mathbf{x})$, che è una composizione di funzioni PR, e quindi è PR.
Notare che possiamo anche usare più predicati PR per definizioni con più casi, usando sempre lo schema sopra ed annidando le definizioni.

**Esempio 5.16** (Funzione modulo)

La funzione $x \bmod y = z \overset{\text{def}}{\iff} z < y \wedge y \mid x - z$, cioè il rappresentante di $x$ modulo $y$, è una funzione PR poiché è definita per ricorsione e per casi:

$$x \bmod y := \begin{cases} 0 \bmod y = 0 \\ (x+1) \bmod y := \begin{cases} 0 & \text{se } x \bmod y + 1 = y \\ (x \bmod y) + 1 & \text{se } x \bmod y + 1 \neq y \end{cases} \end{cases}$$

cioè, visto che $x = qy + x \mod y$, allora $x + 1 = qy + (x \mod y + 1)$, per cui se $x \mod y + 1 = y$ allora $x + 1 = (q + 1)y + 0$, altrimenti $x + 1 = qy + (x \mod y + 1)$. Osservare inoltre che nella definizione per casi abbiamo usato il predicato PR $P(a, b) := \{a + 1 = b\}$, e la $h$ per la ricorsione:

$$h(a, b, c) := \begin{cases} 0 & \text{se } a + 1 = b \\ a + 1 & \text{se } a + 1 \neq b \end{cases}$$

che è una funzione PR per definizione per casi.

**Osservazione 5.17** (Somma dei valori di una funzione PR sotto un certo punto) — Sia $f : \mathbb{N}^{k+1} \to \mathbb{N}$ una funzione PR, allora la funzione $(y_1, \dots, y_k, x) \mapsto \sum_{i<x} f(y_1, \dots, y_k, i)$ è PR, infatti è definita per ricorsione a partire da $h(y_1, \dots, y_k) := 0$ e $g(a, b, c, d_1, \dots, d_k) := b + f(d_1, \dots, d_k, a)$:

$$\sum_{i<x} f(y_1, \dots, y_k, i) := \begin{cases} \sum_{i<0} f(y_1, \dots, y_k, i) & = h(y_1, \dots, y_k) = 0 \\ \sum_{i<x+1} f(y_1, \dots, y_k, i) & = \sum_{i<x} f(y_1, \dots, y_k, i) \\ & +f(y_1, \dots, y_k, x) \end{cases}$$

**Esempio 5.18** (Funzione che conta gli elementi del predicato fino ad $x$)
Sia $P \subseteq \mathbb{N}^{k+1}$ un predicato PR, allora la funzione:

$$g : (y_1, \dots, y_k, x) \mapsto \#\{(y_1, \dots, y_k, i) \in P : i < x\}$$

è una funzione PR, infatti:

$$g(y_1, \dots, y_k, x) = \sum_{i<x} (1 \dot{-} P(y_1, \dots, y_k, i))$$

**Esempio 5.19** (Appartenenza ad un altro predicato sotto un certo punto)
Definiamo il predicato $Q \subseteq \mathbb{N}^{k+1}$ come:

$$(y_1, \dots, y_k, x) \in Q \iff \exists i < x \; (y_1, \dots, y_k, i) \in P$$

con $P \subseteq \mathbb{N}^{k+1}$ predicato PR. Tale sottoinsieme è un predicato PR in quanto è possibile definirlo come:

$$Q(y_1, \dots, y_k, x) = \{g(y_1, \dots, y_k, x) > 0\}$$

dove $g(y_1, \dots, y_k, x) = \#\{(y_1, \dots, y_k, i) \in P : i < x\}$ è la funzione PR (serve affinché l'insieme di confronto sia un predicato PR) che conta gli elementi del predicato $P$ sotto $x$.

**Esempio 5.20** (Funzione quoziente intero)
Definiamo $x \,\texttt{div}\, y := \left\lfloor \frac{x}{y} \right\rfloor$, con la convenzione $x \,\texttt{div}\, 0 := 0$. La funzione $x \,\texttt{div}\, y$ è PR, ad esempio per ricorsione su $x$ (con parametro $y$) e per casi:

$$\begin{cases} 0 \,\texttt{div}\, y = 0 \\ (x+1) \,\texttt{div}\, y := \begin{cases} 0 & \text{se } y = 0 \\ (x \,\texttt{div}\, y) + 1 & \text{se } y \neq 0 \text{ e } x \bmod y + 1 = y \\ x \,\texttt{div}\, y & \text{se } y \neq 0 \text{ e } x \bmod y + 1 \neq y \end{cases} \end{cases}$$

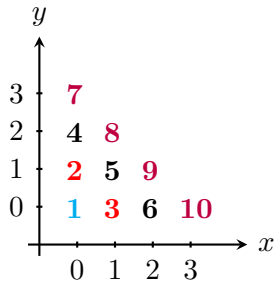
il predicato e la funzione usati per la ricorsione sono quasi gli stessi di quelli usati per definire il modulo, anche in questo caso vale che se $x = (x \,\texttt{div}\, y)y + r$, allora $x + 1 = (x \,\texttt{div}\, y)y + (r + 1)$, per cui se $r + 1 = y$ allora $x + 1 = (x \,\texttt{div}\, y + 1)y + 0$, altrimenti $x + 1 = (x \,\texttt{div}\, y)y + (r + 1)$.

## §5.2 Codifica delle liste

**Definizione 5.21** (Pairing function)**.** Definiamo la funzione **pairing function** (o funzione di accoppiamento):

$$\texttt{cons} : \mathbb{N}^2 \to \mathbb{N} : (x, y) \mapsto \frac{(x+y)(x+y+1)}{2} + x + 1$$

Tale funzione è biunivoca tra le coppie $(x, y) \in \mathbb{N}^2$ e i numeri naturali maggiori di 1 (voglio avere un null point, e quindi non voglio che $(0, 0)$ venga codificato da 0). La funzione di pairing ha il seguente grafico.




**Definizione 5.22** (Funzioni inverse di `cons`)**.** Definiamo le funzioni $\texttt{car}, \texttt{cdr} : \mathbb{N} \to \mathbb{N}$ come le funzioni inverse di `cons`, ovvero:

$$\texttt{car}(\texttt{cons}(x, y)) := x$$
$$\texttt{cdr}(\texttt{cons}(x, y)) := y$$

che sono funzioni PR. Un modo per verificare che sono PR è osservare che possono essere scritte come:

$$\texttt{car}(z) = \sum_{i<z} \sum_{j<z} i \cdot (1 \dot{-} \{z = \texttt{cons}(i, j)\})$$
$$\texttt{cdr}(z) = \sum_{i<z} \sum_{j<z} j \cdot (1 \dot{-} \{z = \texttt{cons}(i, j)\})$$

cioè itero la somma finché non becco la coppia $(i, j)$ tale che $z = \texttt{cons}(i, j)$, allora i due predicati danno 0 e quindi viene $1 \cdot i$ o $1 \cdot j$.

**Esercizio 5.23** (Funzione di Ackermann)**.** Dimostrare che la funzione di Ackermann non è una funzione primitiva ricorsiva:[42]

$$A(m, n) := \begin{cases} A(0, n) = n + 1 \\ A(m + 1, 0) = A(m, 1) \\ A(m + 1, n + 1) = A(m, A(m + 1, n)) \end{cases}$$

[42] **Hint**: Un qualunque programma costituito solo da cicli `for` annidati un numero finito di volte, e senza ricorsione, non può catturare l'intera funzione.

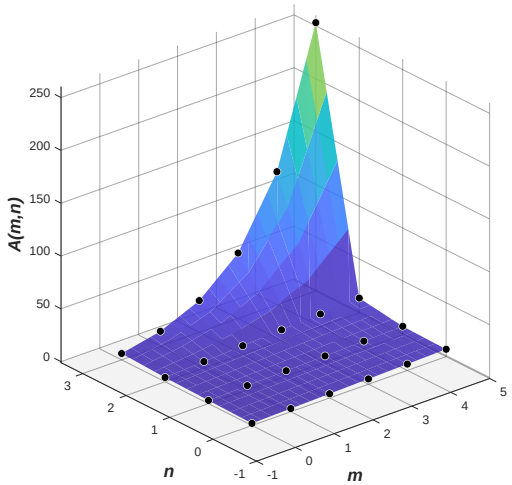



Figura 1: Plot della funzione di Ackermann

**Osservazione 5.24** (Le sezioni $n \mapsto A(m,n)$ sono PR) — Per ogni $m \in \mathbb{N}$, la funzione $A_m : \mathbb{N} \to \mathbb{N}$ definita da $A_m(n) := A(m,n)$ è PR. Si dimostra per induzione su $m$: $A_0(n) = n+1$ è il successore, e se $A_m$ è PR allora anche $A_{m+1}$ è PR, perché è definita per ricorsione da:

$$A_{m+1}(0) = A_m(1), \qquad A_{m+1}(n+1) = A_m(A_{m+1}(n)).$$

L'idea che abbiamo voluto catturare, mediante le funzioni primitive ricorsive, è quella di una classe di funzioni che può essere implementata in un calcolatore solo per mezzo di cicli `for` - per cui sarà già a priori fissato il numero di loop, e che non usino né i costrutti della ricorsione (quindi una funzione non può chiamare se stessa) e né usi cicli potenzialmente illimitati, come i cicli `while`.

## §5.3 Funzioni ricorsive

Definiamo inizialmente della notazione per rappresentare funzioni che non convergono.

**Definizione 5.25** ($\mathbb{N}_\perp$)**.** Definiamo $\mathbb{N}_\perp := \mathbb{N} \cup \{\perp\}$.

**Definizione 5.26** (Funzioni ricorsive)**.** Definiamo le **funzioni ricorsive** come le funzioni $f : \mathbb{N}_\perp^k \to \mathbb{N}_\perp$ a partire da alcune funzioni di base e chiudendo la classe sotto alcune operazioni di chiusura. Le funzioni di base, fissato $k \in \mathbb{N}$, vanno da $\mathbb{N}_\perp^k \to \mathbb{N}_\perp$ e sono:

$$\pi_{k,i}(x_1, \dots, x_k) = \begin{cases} x_i & \text{se } x_1, \dots, x_k \in \mathbb{N} \\ \perp & \text{altrimenti} \end{cases}$$

$$s(x) = \begin{cases} x+1 & \text{se } x \in \mathbb{N} \\ \perp & \text{se } x = \perp \end{cases}$$

$$0_k(x_1, \dots, x_k) = \begin{cases} 0 & \text{se } x_1, \dots, x_k \in \mathbb{N} \\ \perp & \text{altrimenti} \end{cases}$$

Partendo da queste funzioni faccio la chiusura per le operazioni di:

- composizione;
- ricorsione primitiva, con lo stesso schema visto per le funzioni PR, ma con l'aggiunta del caso $f(\perp, \dots) := \perp$;

- **minimizzazione**, ovvero se $f : \mathbb{N}^{k+1} \to \mathbb{N}_\perp$ è ricorsiva, allora la funzione:

$$\mu_k f(x_1, \dots, x_k) = a \in \mathbb{N} \overset{\text{def}}{\iff} \\ f(x_1, \dots, x_k, a) = 0 \wedge \forall i < a \; f(x_1, \dots, x_k, i) \in \mathbb{N} \setminus \{0\}$$

  in particolare $f(x_1, \dots, x_k, i)$ non è $\perp$; se non esistesse un tale $a$ allora poniamo $\mu_k f(x_1, \dots, x_k) := \perp$. Cioè $\mu_k f$ produce il valore $a$ tale che $f(x_1, \dots, x_k, a) = 0$, e tale $a$ è il minimo possibile - l'idea che si vuole catturare è quella della convergenza di un algoritmo, che si ferma al passo $a$, quando $f$ produce 0, e se non si ferma allora $\mu_k f$ non converge e quindi la poniamo $\perp$.

**Definizione 5.27** (Funzioni ricorsive totali)**.** Diciamo che $f : \mathbb{N}_\perp^k \to \mathbb{N}_\perp$ funzione ricorsiva è una **totale** se $\forall (x_1, \dots, x_k) \in \mathbb{N}^k \; f(x_1, \dots, x_k) \in \mathbb{N}$, cioè $f$ non produce mai $\perp$.

**Osservazione 5.28** (Input $\perp$) — Se $f$ è ricorsiva ed uno degli input è $\perp$, allora $f$ produce $\perp$:

$$f(x_1, \dots, x_{i-1}, \perp, x_{i+1}, \dots, x_k) = \perp$$

per ogni $i \in \{1, \dots, k\}$.

*Dimostrazione.* Si procede per induzione sulla costruzione di $f$. I casi delle 3 funzioni di base sono banali per la definizione stessa.

- Per la composizione, se $f = h \circ (g_1, \dots, g_m)$, e uno degli input di $f$ è $\perp$, allora anche uno dei $g_i$ è $\perp$, per cui $h$ produce $\perp$ per ipotesi induttiva, e quindi $f$ è $\perp$.
- Per la ricorsione primitiva, se $f$ è definita per ricorsione da $h$ e $g$, e uno degli input di $f$ è $\perp$, allora anche uno degli input di $h$ o di $g$ è $\perp$, per cui $h$ o $g$ producono $\perp$ per ipotesi induttiva, e quindi anche $f$ dà $\perp$.
- Per la minimizzazione, se $f = \mu_k f'$, e uno degli input di $f$ è $\perp$, allora anche uno degli input di $f'$ è $\perp$, per cui $f'$ produce $\perp$ a prescindere dalla variabile per cui si minimizza, e non fornendo mai un valore in $\mathbb{N} \setminus \{0\}$, il minimo non esiste, e quindi anche $f$ è $\perp$ per definizione.

$\square$

**Fatto 5.29** (Inclusione tra le classi di funzioni ricorsive)**.** Valgono le seguenti inclusioni, tutte strette:

$$\{\text{funzioni PR}\} \subsetneq \{\text{funzioni ricorsive totali}\} \subsetneq \{\text{funzioni ricorsive}\}$$

*Dimostrazione.* I contenimenti sono tutti banali per le definizioni date, vediamo solo i controesempi per mostrare che sono stretti. Per il secondo contenimento si osserva che $\mu_0 s \equiv \perp$, perché l'equazione $s(x) = 0$ non ha soluzioni. Per il primo è sufficiente considerare la funzione di Ackermann. $\square$

## §5.4 Macchine di Turing

Vogliamo caratterizzare le funzioni ricorsive come esattamente quelle che sono implementabili da una Macchina di Turing.

**Definizione 5.30** (Macchina di Turing - TM)**.** Siano $\Sigma$ un insieme finito di **simboli**, $Q$ un insieme finito di **stati**, una **macchina di Turing** (**TM**) è il dato di $\Sigma$, $Q$, di una **funzione di transizione**:

$$\delta : Q \times \Sigma \to Q \times \Sigma \times \{\leftarrow, \downarrow, \rightarrow\}$$

dove l'ultimo insieme si rappresenta formalmente con $\{0, 1, 2\}$ (con la corrispondenza in questo esatto ordine), ma è più comodo rappresentarlo con le frecce. Una **configurazione** è il dato di $(s, p, n)$ dove $s \in Q$ è lo stato della TM, $p \in \mathbb{N}$ la posizione, e $n \in \Sigma^*$[43] la stringa (o meglio tutto il nastro) in cui ci troviamo, i.e. $n \in \Sigma^{\mathbb{N}}$ che è quasi sempre 0, i.e. $\exists N \in \mathbb{N}\ \forall i > N\ n(i) = 0$.
Dico che la macchina di Turing $M$ passa da una configurazione all'altra:

$$(s, p, n) \overset{M}{\to} (s', p', n')$$

se: $s' = \pi_{3,1} \circ \delta(s, n(p))$, $p' = (p + \pi_{3,3} \circ \delta(s, n(p))) \dot{-} 1$ cioè la posizione finale è quella iniziale più 1 se la freccia è $\rightarrow$, è quella iniziale se la freccia è $\downarrow$, e $p - 1$ se la freccia è $\leftarrow$ - dove la terza componente di $\delta$ è identificata col numero non con la freccia, e:

$$n'(i) = \begin{cases} n(i) & \text{se } i \neq p \\ \pi_{3,2} \circ \delta(s, n(p)) & \text{se } i = p \end{cases}$$

cioè scorrendo il nastro modifico solo la cella in cui mi trovo inizialmente, $n(p)$, lascio le altre uguali e mi sposto.

**Definizione 5.31** (Chiusura transitiva)**.** Dico che $c = (s, p, n) \overset{M^*}{\to} c' = (s', p', n')$ è la **chiusura transitiva** di $c \overset{M}{\to} c'$, se esiste una sequenza finita di configurazioni $c_0, \ldots, c_k$ tale che:

$$c_0 \overset{M}{\to} c_1 \overset{M}{\to} \ldots \overset{M}{\to} c_k$$

con $c_0 = c$ e $c_k = c'$.

**Esercizio 5.32** (TM che riconosce i palindromi)**.** Costruire una TM, ovvero la tabella di transizione, che riceva in input:

$$0110101010111 \ldots 10112000 \ldots$$

ed ha due stati terminali, 0 e $0'$, e termina in 0 se e solo se la stringa è palindroma. Operiamo nell'alfabeto $\Sigma = \{0, 1, 2\}$.

**Esercizio 5.33** (Difficile)**.** Dimostra che una TM come nell'esercizio precedente, termina, nel caso peggiore in $O(\text{lunghezza stringa}^2)$ mosse - con $O$ si intende $\leq kl^2 + o(l^2)$ per qualche $k > 0$.[44]

[43] Questa è la notazione di Kleene.

[44] **Hint**: L'algoritmo più sensato da implementare è: sostituire il primo simbolo con 2, poi spostarsi fino alla fine della stringa, sostituire l'ultimo simbolo con 2, e poi tornare indietro fino al primo simbolo, ripetere finché non si sostituisce tutta la stringa con dei 2, oppure finché non si trova un simbolo diverso da quello sostituito con 2, per cui se la stringa è lunga $l$, allora si fanno $l$ passi per sostituire ciascun simbolo, e quindi $l^2$ passi in totale. Bisogna dimostrare che non si può fare meglio di così,

## §5.5 Funzioni Turing-computabili

**Definizione 5.34** (Turing-computabilità)**.** Sia $f : \mathbb{N}^k \to \mathbb{N}_\perp$[45] una funzione ricorsiva **parziale** - nel senso di non necessariamente totale, quindi ricorsiva generica, diciamo che $f$ è **Turing-computabile** se esiste una TM $M$, con:

- $\Sigma \supseteq \{0, 1, \blacktriangleright\}$,
- $Q \supseteq \{0, 1\}$, con 0 **stop** e 1 **start**,
- $\delta(0, x) = (0, x, \downarrow)$, ovvero lo stato terminale rimane fermo, non cambia simbolo né si muove,

e tale per cui:

- se $f(x_1, \ldots, x_k) = y$, allora $\texttt{input}(x_1, \ldots, x_k) \overset{M^*}{\to} \texttt{output}(y)$, con $\texttt{input}(x_1, \ldots, x_k)$ e $\texttt{output}(y)$ due configurazioni,
- se $f(x_1, \ldots, x_k) = \perp$, allora $\texttt{input}(x_1, \ldots, x_k) \overset{M^*}{\not\to} (0, \ldots)$, ovvero non termina in uno stato 0, dove avverrebbe lo stop, i.e. se non c'è un output non termina mai.

Dove:

$$\texttt{input}(x_1, \ldots, x_k) := (1, 0, \blacktriangleright \underbrace{111111}_{x_1} 0 \underbrace{1111}_{x_2} 0 \ldots 0 \underbrace{111}_{x_k} 00 \ldots) \in Q \times \mathbb{N} \times \Sigma^*$$

dove il numero $x_i$ è rappresentato da una sequenza di $x_i$ simboli 1 (notazione unaria) - tanti quanti il valore di $x_i$, e tra un numero e l'altro c'è uno 0; analogamente:

$$\texttt{output}(y) := (0, 0, \blacktriangleright \underbrace{111 \ldots 1}_{y} 00 \ldots) \in Q \times \mathbb{N} \times \Sigma^*$$

dove $y$ è rappresentato da una sequenza di $y$ simboli 1, seguita da una sequenza infinita di 0.

**Teorema 5.35** (Ricorsiva $\Rightarrow$ Turing-computabile)
Se $f : \mathbb{N}^k \to \mathbb{N}_\perp$ è ricorsiva parziale, allora $f$ è Turing-computabile.

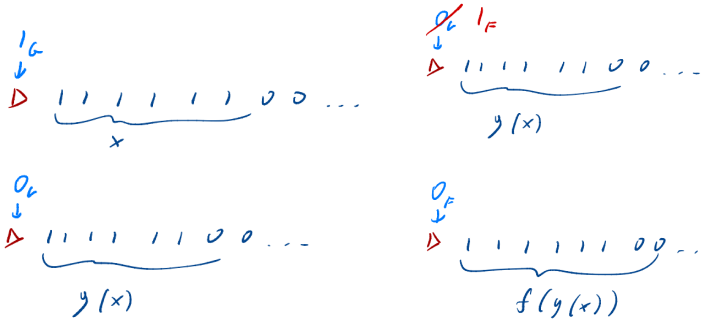
L'idea sarà far vedere che le funzioni di base sono Turing-computabili, e che le funzioni Turing computabili sono chiuse per operazioni di composizione, ricorsione primitiva e minimizzazione. Iniziamo con un caso particolare.

**Esempio 5.36** (Composizione di funzioni Turing computabili)
Siano $f, g : \mathbb{N} \to \mathbb{N}_\perp$ funzioni Turing computabili, verifichiamo che $h(x) := f(g(x))$ è Turing computabile.

l'idea è che se la stringa è palindroma, la taglio alla metà esatta e posso ricostruire la metà sinistra usando come informazione tutta la successione di stati che ho ottenuto attraversando il confine. La TM deve attraversare il confine almeno $l/2$ volte, quindi decido di aggiungere un blocco di 0 tra la stringa e la metà riflessa, in modo che la TM non possa usare come informazione il numero di passi fatti, e quindi deve attraversare il confine almeno $l/2$ volte, e ogni volta deve fare almeno $l/2$ passi, per cui ottengo $O(l^2)$.

[45] Non mettiamo $\perp$ perché, avendo visto che se uno degli input è $\perp$ allora anche l'output è $\perp$, non ci interessa studiare funzioni di questo tipo.

Ciò che vorrei è, detta $G$ la macchina associata a $g$ e $F$ quella associata a $f$, è far partire $G$ dalla sequenza iniziale $\blacktriangleright$, farla terminare sempre a $\blacktriangleright$, e poi da qui far partire $F$ e farla arrivare a $\blacktriangleright$ nello stato $0_F$, facendo queste cose avrei di fatto costruito una macchina $H$ che implementa $h$.




Posso costruire per bene una macchina $H$ che implementa le operazioni fatte a mano.

*Dimostrazione.* Supponiamo per comodità che $\Sigma_G = \Sigma_F = \{0, 1, \blacktriangleright\} =: \Sigma_H$, definiamo $Q_H := Q_G \sqcup Q_F$, dove poniamo $1_H := 1_G$ e $0_H := 0_F$ - rispettivamente stato iniziale della macchina $H$ e stato finale della macchina $H$. Per la funzione di transizione definiamo:

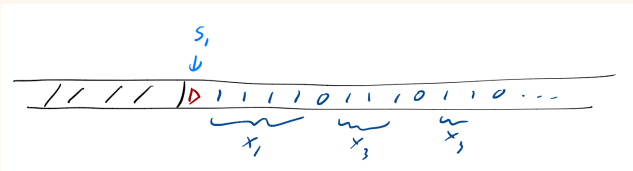
$$\delta_H : Q_H \times \Sigma_H \to Q_H \times \Sigma_H \times \{\leftarrow, \downarrow, \rightarrow\} : \delta_H(q, \sigma) := \begin{cases} \delta_G(q, \sigma) & \text{se } q \in Q_G \setminus \{0_G\} \\ \delta_F(1_F, \sigma) & \text{se } q = 0_G \\ \delta_F(q, \sigma) & \text{se } q \in Q_F \end{cases}$$

cioè se sono nello stato iniziale di $G$, mi comporto come se fossi in $F$ - questo corrisponderebbe al nostro intervento manuale, altrimenti mi comporto come la macchina dello stato a cui appartengo. $\square$

Ora si pone il problema di generalizzare questo argomento alla composizione arbitraria. Costruiamo delle macchine che calcolino $f$, ma con qualche vincolo in più rispetto a quelli richiesti dalla definizione di Turing-computabilità, questa scelta è sia comoda che possibile, per cui la adottiamo. Data $f$ vogliamo costruire la tabella delle funzioni di transizione della macchina che calcola $f$, lo facciamo a pezzi, cioè prendiamo dei sottoinsiemi degli stati e scriviamo dove vanno.

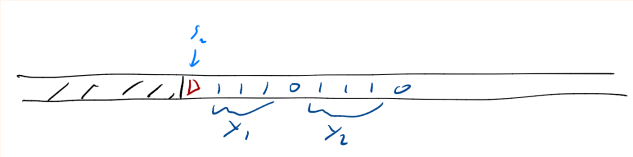
**Setting**: L'alfabeto per la costruzione di queste macchine sarà costituito solo dai tre simboli di base per la Turing-computabilità, ovvero $\Sigma = \{0, 1, \blacktriangleright\}$, per cui definiremo solo nuovi stati.

**Notazione 5.37** (Notazione per descrivere il comportamento di pezzi di TM) — Con la notazione: $s_1 \to \texttt{macchina} \to s_2$ intendiamo che, preso un sottoinsieme di stati, dove $s_1$ è lo stato iniziale, che prendiamo sopra a $\blacktriangleright$, seguito da una configurazione




o non ritorna mai nello stato $s_2$, o ritorna, nello stato $s_2$, sul $\blacktriangleright$, in una nuova

configurazione. Assumiamo inoltre che le macchine che stiamo costruendo non vadano mai a sinistra di ▶.




Quindi $s_1 \to \texttt{macchina} \to s_2 \overset{\text{def}}{\iff}$ la `macchina` viene posta nello stato $s_1$ sopra a ▶, con la configurazione - sempre scritta in notazione unaria con gli 0 - che sarà indicata tra parentesi, allora, senza mai toccare ciò che sta a sinistra del triangolino, porta la macchina nello stato $s_2$ sopra a ▶, con una nuova configurazione - nella solita notazione - che sarà indicata tra parentesi, oppure non termina mai:

$$s_1 \to \texttt{macchina} \to s_2(s_1 \blacktriangleright x_1x_2\dots x_k\dots) = s_2 \blacktriangleright y_1y_2\dots$$

Quindi con questa notazione possiamo facilmente descrivere il comportamento di pezzi di macchine, e poi comporli per ottenere macchine più complesse.

**Definizione 5.38** (Macchine elementari)**.** Definiamo le seguenti macchine elementari.

- $s_1 \to \texttt{push}_{n,i}s_2(s_1 \blacktriangleright x_1x_2\dots x_n) = s_2 \blacktriangleright x_1x_2\dots x_nx_i$, dove la nuova configurazione ha $n+1$ numeri, ha copiato l'$i$-esimo numero in fondo alla fila;
- $s_1 \to \texttt{rem}_{n,i}s_2(s_1 \blacktriangleright x_1x_2\dots x_n) = s_2 \blacktriangleright x_1x_2\dots x_{i-1}x_{i+1}\dots x_n$, dove la nuova configurazione ha $n-1$ numeri, ha rimosso l'$i$-esimo numero;
- $s_1 \to \texttt{exec}_{n,k}fs_2(s_1 \blacktriangleright x_1x_2\dots x_n) = s_2 \blacktriangleright x_1x_2\dots x_{n-k}f(x_{n-k+1},\dots,x_n)$, dove la nuova configurazione ha $n-k+1$ numeri, ha eseguito la funzione $f$ sugli ultimi $k$ numeri, e ha aggiunto il risultato in fondo alla fila;
- infine definiamo la macchina che controlla che l'$i$-esimo numero sia 0:

  $$s_1 \to \texttt{if}\, x_i = 0 \to s_2 \vee s_3(s_1 \blacktriangleright x_1x_2\dots x_n) = \begin{cases} s_2 \blacktriangleright x_1x_2\dots x_n & \text{se } x_i = 0 \\ s_3 \blacktriangleright x_1x_2\dots x_n & \text{se } x_i \neq 0 \end{cases}$$

  i simboli sono uguali, cambia solo lo stato in cui si ritrova la macchina su ▶, in base a se $x_i$ è 0 o no.

Adesso il piano è: assumiamo di avere queste macchine elementari, e dimostriamo che la classe delle funzioni Turing-computabili, rappresentabili mediante macchine di Turing che hanno come alfabeto $\Sigma = \{0, 1, \blacktriangleright\}$ e che non vanno mai a sinistra di ▶, è chiusa per composizione, ricorsione primitiva e minimizzazione. Dobbiamo anche definire macchine di Turing che implementino le funzioni: successore, sottrazione troncata per 1, costante 0 e proiezione. Infine vediamo come implementare le macchine elementari e con questo abbiamo dimostrato che ogni funzione ricorsiva è Turing-computabile.

*Dimostrazione.* Vediamo che la classe delle funzioni Turing-computabili è chiusa per composizione, ricorsione primitiva e minimizzazione. In tutti i casi faremo la seguente cosa: assumiamo di avere le macchine elementari che implementano le operazioni di `push`, `rem` e `exec`, e vediamo come usarle per costruire macchine che implementano le funzioni ottenute per composizione, ricorsione primitiva e minimizzazione. In particolare le nuove macchine saranno ottenute unendo gli stati - in maniera disgiunta per evitare problemi - e incollando le funzioni di transizione (come nell'esempio precedente), ci limiteremo

pertanto a descrivere soltanto come sono fatte queste ultime, componendo quelle delle macchine elementari, e non descrivendo per intero le macchine che implementano le funzioni.

composizione

Caso particolare: consideriamo prima il caso $f : \mathbb{N}^2 \to \mathbb{N}_\perp$ e $g_1, g_2 : \mathbb{N}^2 \to \mathbb{N}_\perp$, e poi generalizziamo. Vogliamo verificare che $f(g_1(x_1, x_2), g_2(x_1, x_2))$ sia Turing computabile.
Siano $F, G_1, G_2$ le macchine che implementano $f, g_1, g_2$, il nostro obiettivo è costruire una macchina che parta da $\blacktriangleright x_1x_2$ e arrivi a $\blacktriangleright f(g_1(x_1, x_2), g_2(x_1, x_2))$ - cioè a $\blacktriangleright y$ con $y = f(g_1(x_1, x_2), g_2(x_1, x_2))$, come da definizione.
In primis vogliamo copiare gli input, altrimenti nel calcolo usando $\texttt{exec}_{2,2}g_1$ scompaiono e non posso riutilizzarli, quindi operiamo due passaggi di `push` per copiare $x_1$ e $x_2$ in fondo alla lista:

$$s_1 \to \texttt{push}_{2,1} \to \texttt{push}_{3,2} \to s_2$$

per cui $s_2 \blacktriangleright x_1x_2x_1x_2$. Ora, dallo stato $s_2$ applico $\texttt{exec}_{4,2}g_1$:

$$s_2 \to \texttt{exec}_{4,2}g_1 \to s_3$$

con $s_3 \blacktriangleright x_1x_2g_1(x_1, x_2)$. Ora copio di nuovo $x_1$ e $x_2$ in fondo alla lista (per usare `exec` devo usare i numeri in fondo quindi ho necessità di copiarli) e calcolo $g_2$:

$$s_3 \to \texttt{push}_{3,1} \to \texttt{push}_{4,2} \to \texttt{exec}_{5,2}g_2 \to s_4$$

con $s_4 \blacktriangleright x_1x_2g_1(x_1, x_2)g_2(x_1, x_2)$. A questo punto posso cancellare i primi due e calcolare $f$:

$$s_4 \to \texttt{rem}_{4,1} \to \texttt{rem}_{3,1} \to \texttt{exec}_{2,2}f \to s_5$$

con $s_5 \blacktriangleright f(g_1(x_1, x_2), g_2(x_1, x_2))$, infine vado da $s_5$ a 0 per stoppare la macchina.
Caso generale: se $f : \mathbb{N}^k \to \mathbb{N}_\perp$ e $g_1, \dots, g_k : \mathbb{N}^h \to \mathbb{N}_\perp$, vogliamo costruire una macchina che parta da $\blacktriangleright x_1 \dots x_h$ e arrivi a $\blacktriangleright f(g_1(x_1, \dots, x_h), \dots, g_k(x_1, \dots, x_h))$.
Iniziamo copiando tutti gli input in fondo alla lista:

$$^{46}1 \to \texttt{push}_{h,1} \to \texttt{push}_{h+1,2} \to \dots \to \texttt{push}_{2h-1,h} \to 2$$

con $2 \blacktriangleright x_1 \dots x_hx_1 \dots x_h$. Ora usiamo $\texttt{exec}_{2h,h}g_1$ per calcolare $g_1$:

$$2 \to \texttt{exec}_{2h,h}g_1 \to 3$$

con $3 \blacktriangleright x_1 \dots x_hg_1(x_1, \dots, x_h)$. Ripetendo queste operazioni $k$ volte, e otteniamo lo stato $4 \blacktriangleright x_1 \dots x_hg_1(x_1, \dots, x_h) \dots g_k(x_1, \dots, x_h)$. A questo punto cancelliamo i primi $h$ numeri e calcoliamo $f$:

$$4 \to \texttt{rem}_{h+k,1} \to \texttt{rem}_{h+k-1,1} \to \dots \to \texttt{rem}_{k+1,1} \to \texttt{exec}_{k,k}f \to 5$$

con $5 \blacktriangleright f(g_1(x_1, \dots, x_h), \dots, g_k(x_1, \dots, x_h))$, infine vado da 5 a 0 per stoppare la macchina. Restano i casi in cui $g_i(x_1, \dots, x_h) = \perp$ per qualche $i$, oppure $f(g_1(x_1, \dots, x_h), \dots, g_k(x_1, \dots, x_h)) = \perp$, in ambo i casi, per ipotesi, le macchine che implementano $g_i$ o $f$ non terminano, e quindi anche $\texttt{exec}g_i$ o $\texttt{exec}f$ non termina, per cui la macchina che abbiamo costruito non termina, e quindi è coerente con $f(g_1(x_1, \dots, x_h), \dots, g_k(x_1, \dots, x_h)) = \perp$.

[46] Per comodità, d'ora in poi chiameremo gli stati solo con i numeri.

ricorsione primitiva

Siano $h : \mathbb{N}^k \to \mathbb{N}_\perp$ e $g : \mathbb{N}^{k+2} \to \mathbb{N}_\perp$, Turing computabili, e vogliamo implementare $f : \mathbb{N}^{k+1} \to \mathbb{N}_\perp$, definita via ricorsione primitiva a partire da $h$ e $g$:

$$f(x, y_1, \dots, y_k) := \begin{cases} f(0, y_1, \dots, y_k) := h(y_1, \dots, y_k) \\ f(x+1, y_1, \dots, y_k) := g(x, f(x, y_1, \dots, y_k), y_1, \dots, y_k) \end{cases}$$

Calcoliamo al contrario la ricorsione, e teniamoci un contatore in modo da sapere quando fermarci. Assumiamo, WLOG, di avere uno 0 alla fine della stringa di input, ovvero $1 \blacktriangleright xy_1 \dots y_k 0$, calcoliamo $f(0, y_1, \dots, y_k) = h(y_1, \dots, y_k)$:

$$1 \to \texttt{push}_{k+2,2} \to \texttt{push}_{k+3,3} \to \dots \to \texttt{push}_{2k+1,k+1} \to \texttt{exec}_{2k+2,k} h \to 2$$

con $2 \blacktriangleright xy_1 \dots y_k 0 h(y_1, \dots, y_k)$, ora controlliamo se siamo nel caso base o no:

$$2 \to \texttt{if}\, x = 0 \to \begin{cases} 3 & \text{se } x = 0 \\ 4 & \text{se } x \neq 0 \end{cases}$$

per cui $3 \blacktriangleright 0y_1 \dots y_k 0 h(y_1, \dots, y_k)$, e siamo arrivati in questo caso, è sufficiente applicare $k+2$ volte `rem` e poi andare a 0 per ottenere $0 \blacktriangleright f(0, y_1, \dots, y_k)$.
Dimostriamo preliminarmente, per induzione - il caso 0 è quello sopra, che, assumendo $x - t \neq 0$, che da $\blacktriangleright x - ty_1 \dots y_k t f(t, y_1, \dots, y_k)$, riusciamo a passare alla configurazione $\blacktriangleright x - (t+1) y_1 \dots y_k (t+1) f(t+1, y_1, \dots, y_k)$, in tal modo avremo dimostrato per induzione che se $x - t \neq 0$, allora possiamo passare alla configurazione con $t+1$; fatto questo si applica $\texttt{if}\, x - (t+1) = 0$ per verificare se siamo arrivati al caso base, ed in tal caso, a meno di cancellare i primi simboli abbiamo concluso, altrimenti si continua, in un altro stato, col passo ricorsivo. Osserviamo inoltre che, essendo $x \in \mathbb{N}$, si avrà necessariamente $x - t = 0$, per $t$ sufficientemente grande e questo ci permette di passare dallo stato 4 alla terminazione.
Resta da verificare, per induzione, che si può passare da $\blacktriangleright x - ty_1 \dots y_k t f(t, y_1, \dots, y_k)$ a $\blacktriangleright x - (t+1) y_1 \dots y_k (t+1) f(t+1, y_1, \dots, y_k)$: ricopiamo la stringa e cambiamola nel mentre.

$$4 \to \texttt{push}_{k+3,1} \to \texttt{exec}_{k+4,1} \dot{-} 1 \to 5$$

con $5 \blacktriangleright x - ty_1 \dots y_k t f(t, y_1, \dots, y_k) x - (t+1)$, copiamo ora gli input $y_1, \dots, y_k$:

$$5 \to \texttt{push}_{k+4,2} \to \texttt{push}_{k+5,3} \to \dots \to \texttt{push}_{2k+3,k+1} \to 6$$

con $6 \blacktriangleright x - ty_1 \dots y_k t f(t, y_1, \dots, y_k) x - (t+1) y_1 \dots y_k$, ora ricopiamo di nuovo tutta la stringa (includendo nel mezzo $f(t, y_1, \dots, y_k)$ e $t$ a cui applicare subito dopo $\texttt{exec} + 1$), cancelliamo quello che non serve e calcoliamo $\texttt{exec}g$. Cancellando con `rem` anche $f(t, y_1, \dots, y_k)$, otteniamo $\blacktriangleright x - (t+1) y_1 \dots y_k (t+1) f(t+1, y_1, \dots, y_k)$, e da qui si arriva allo stato terminale.
Rimane il caso in cui $h(y_1, \dots, y_k) = \perp$ o $g(x, f(x, y_1, \dots, y_k), y_1, \dots, y_k) = \perp$, in tal caso, per ipotesi, le macchine associate a $h$ e $g$ non terminano e di conseguenza non terminano $\texttt{exec}h$ o $\texttt{exec}g$, per cui non termina la macchina che implementa $f$, coerentemente col fatto che $f(x, y_1, \dots, y_k) = \perp$ come funzione ricorsiva.

minimizzazione

Sia $g : \mathbb{N}^{k+1} \to \mathbb{N}_\perp$ Turing computabile, e vogliamo implementare $f : \mathbb{N}^k \to \mathbb{N}_\perp$, definita via minimizzazione a partire da $g$:

$$f(y_1, \dots, y_k) := \mu_x g(x, y_1, \dots, y_k)$$

Partiamo da $1 \blacktriangleright 0y_1 \dots y_k$, dove usiamo lo 0 come base del contatore che cis serve. Dimostriamo che, partendo dallo stato $2 \blacktriangleright ty_1 \dots y_k$, con $t \in \mathbb{N}$, o si ottiene l'output

voluto, oppure si ritorna nello stesso stato ma con $t+1$. Copiamo in fondo i valori $t, y_1, \ldots, y_k$ e calcoliamo $g(t, y_1, \ldots, y_k)$:

$$2 \to \texttt{push}_{k+1,1} \to \texttt{push}_{k+2,2} \to \ldots \to \texttt{push}_{2k+1,k+1} \to \texttt{exec}_{2k+2,k+1} g \to 3$$

con $3 \blacktriangleright t y_1 \ldots y_k g(t, y_1, \ldots, y_k)$, ora controlliamo se $g(t, y_1, \ldots, y_k) = 0$:

$$3 \to \texttt{if}\, x_{k+2} = 0 \to \begin{cases} 4 & \text{se } g(t, y_1, \ldots, y_k) = 0 \\ 5 & \text{se } g(t, y_1, \ldots, y_k) \neq 0 \end{cases}$$

da 4 è sufficiente cancellare i simboli iniziali e andare a 0 per concludere. Nel ramo 5, invece, cancelliamo $g(t, y_1, \ldots, y_k)$ e torniamo a 2:

$$\begin{aligned} 5 \to \texttt{rem}_{k+2,k+2} \to \texttt{push}_{k+1,1} \to \texttt{exec}_{k+2,1} + 1 \to \texttt{push}_{k+2,2} \to \\ \to \texttt{push}_{k+3,3} \to \ldots \to \texttt{push}_{2k+1,k+1} \to \texttt{rem}_{2k+2,1} \to \texttt{rem}_{2k+1,1} \to \\ \to \ldots \to \texttt{rem}_{k+2,1} \to 2 \end{aligned}$$

Quindi la macchina termina esattamente quando trova il minimo $a$ tale che $g(a, y_1, \ldots, y_k) = 0$. Se non esiste $i$ tale che la computazione di $g(i, y_1, \ldots, y_k)$ termina, oppure se $g(i, y_1, \ldots, y_k) = \perp$ per qualche $i$ prima di trovare un valore zero, allora la macchina non raggiunge mai lo stato 0, e questo è coerente con la definizione di quando si ha $f(y_1, \ldots, y_k) = \perp$ per minimizzazione.

□

Rimane ora da costruire le macchine elementari, e le macchine che implementano le funzioni $s$, $0_k$, $\pi_{k,i}$ e $\dot{-}1$.

**Osservazione 5.39** (TM che implementano $\pi_{k,i}$ e $0_k$) — Osservare che $\pi_{k,i}$ è implementabile con una sequenza di `rem` che cancella tutti i numeri tranne l'$i$-esimo, ricordando che lavoriamo in notazione unaria si lascia solo il blocco di uni relativo all'$i$-esimo numero; $0_k$ è implementabile semplicemente prendendo la rappresentazione in notazione unaria dei $k$ numeri, e cancellando tutto fino a lasciare solo $\blacktriangleright 00\ldots$, che rappresenta correttamente lo 0 in notazione unaria.

**Esercizio 5.40.** Costruire macchine di Turing che implementino le funzioni $s$ e $\dot{-}1$.

*Soluzione.* Per implementare $s$, partendo da $1 \blacktriangleright 1\ldots1$, con $x$ simboli 1, basta aggiungere un altro simbolo 1 alla fine della stringa:

$$1 \to \texttt{push}_{x,1} \to 0$$

Per implementare $\dot{-}1$, partendo da $1 \blacktriangleright x$, con $x$ simboli 1, in primis controllo se $x = 0$, e poi decido se restituire 0 o fare la sottrazione:

$$1 \to \texttt{if}\, x = 0 \to \begin{cases} 2 & \text{se } x = 0 \\ 3 & \text{se } x \neq 0 \end{cases}$$

con $2 \blacktriangleright 0$ e $3 \blacktriangleright x$, ora riscrivo $x$ in notazione unaria per comodità, $3 \blacktriangleright 1\ldots1$, con $x$ simboli 1, e poi cancello un simbolo 1:

$$3 \to \texttt{rem}_{x,1} \to 0$$

In ambo i casi le nuove macchine saranno ottenute unendo gli stati - in maniera disgiunta - e unendo le funzioni di transizione in modo che lo stato di stop di quella applicata prima si comporti come se fosse lo stato iniziale di quella applicata dopo. □

**Esercizio 5.41.** Costruire macchine di Turing che implementino le funzioni `push`, `rem`, `exec` e `if` .

*Soluzione.* Per tutte le macchine elementari usiamo $\Sigma = \{0, 1, \blacktriangleright\}$ e $Q \supseteq \{0, 1\}$, ma che dipende da quale macchina stiamo costruendo.

$\boxed{\texttt{if}\, x_i = 0}$

Vogliamo costruire una macchina che vada da $1 \blacktriangleright x_1 \dots x_n$ a $2 \blacktriangleright x_1 \dots x_n$ se $x_i = 0$, e a $3 \blacktriangleright x_1 \dots x_n$ se $x_i \neq 0$. Quindi la tabella della funzione di transizione sarà composta da $4 + i$ righe che mi servono per arrivare a $x_i$ e poi decidere se andare in 2 o 3, da cui $Q = \{0, 1, 2, 3, \alpha_1, \dots, \alpha_i\}$ e:

| | $\blacktriangleright$ | 0 | 1 |
|---|---|---|---|
| 0 | $\blacktriangleright\ 0 \downarrow$ | $0\ 0 \leftarrow$ | $1\ 0 \leftarrow$ |
| 1 | $\blacktriangleright\ \alpha_1 \rightarrow$ | $0\ \alpha_1 \rightarrow$ | $1\ \alpha_1 \rightarrow$ |
| $\alpha_j\ (1 \leq j \leq i-1)$ | $\blacktriangleright\ \alpha_j \rightarrow$ | $0\ \alpha_{j+1} \rightarrow$ | $1\ \alpha_j \rightarrow$ |
| $\alpha_i$ | $\blacktriangleright\ \alpha_i \rightarrow$ | $0\ 2 \rightarrow$ | $1\ 3 \rightarrow$ |
| 2 | $\blacktriangleright\ 2 \downarrow$ | $0\ 2 \leftarrow$ | $1\ 2 \leftarrow$ |
| 3 | $\blacktriangleright\ 3 \downarrow$ | $0\ 3 \leftarrow$ | $1\ 3 \leftarrow$ |

avendo definito $\Sigma, Q$ e $\delta$, abbiamo costruito la macchina che implementa $\texttt{if}\, x_i = 0$.

$\boxed{\texttt{rem}_{n,i}}$ e $\boxed{\texttt{push}_{n,i}}$

Non sono semplicissime da implementare usando solo $\Sigma = \{0, 1, \blacktriangleright\}$, e per far funzionare tutto il resto ci conviene restare solo con questi simboli, pertanto non le riporto, in ogni caso $\texttt{push}_{n,i}$ è più semplice da implementare, mentre $\texttt{rem}_{n,i}$ è più complicata.

$\boxed{\texttt{exec}_{n,k} f}$

Vogliamo costruire una TM che parta da $1 \blacktriangleright x_1 \dots x_n$ e arrivi alla configurazione $0 \blacktriangleright x_1 \dots x_{n-k} f(x_{n-k+1}, \dots, x_n)$. Trattiamo qui il caso $1 \leq k < n$ (se $k = n$, si applica direttamente la macchina $F$ all'intero input). Sia $F$ la macchina di Turing che implementa $f$. La configurazione iniziale corrisponde sul nastro a:

$$\blacktriangleright \underbrace{11\dots11}_{x_1} 0 \underbrace{11\dots11}_{x_2} 0 \dots 0 \underbrace{11\dots11}_{x_{n-k}} 0 \underbrace{11\dots11}_{x_{n-k+1}} 0 \dots 0 \underbrace{11\dots11}_{x_n} 00 \dots$$

in primis vogliamo che la macchina si sposti verso destra fino a raggiungere $x_{n-k+1}$; arrivati allo 0 prima di $x_{n-k+1}$, ci mettiamo $\blacktriangleright$, a questo punto si usa che la TM che implementa $f$ non va mai a sinistra di $\blacktriangleright$. Fatto questo abbiamo $f(x_{n-k+1}, \dots, x_n)$, per cui bisogna solo tornare indietro, la tabella della funzione di transizione sarà:

| | $\blacktriangleright$ | 0 | 1 |
|---|---|---|---|
| 0 | $\blacktriangleright\ 0 \downarrow$ | $0\ 0 \leftarrow$ | $1\ 0 \leftarrow$ |
| 1 | $\blacktriangleright\ \alpha_1 \rightarrow$ | $0\ \alpha_1 \rightarrow$ | $1\ \alpha_1 \rightarrow$ |
| $\alpha_j\ (1 \leq j \leq n-k-1)$ | $\blacktriangleright\ \alpha_j \rightarrow$ | $0\ \alpha_{j+1} \rightarrow$ | $1\ \alpha_j \rightarrow$ |
| $\alpha_{n-k}$ | $\blacktriangleright\ \alpha_{n-k} \rightarrow$ | $\blacktriangleright\ 1_F \downarrow$ | $1\ \alpha_{n-k} \rightarrow$ |
| tabella di $F \setminus \{0_F\}$ | | | |
| $0_F$ | $0\ 2 \leftarrow$ | $0\ 2 \leftarrow$ | $1\ 2 \leftarrow$ |
| 2 | $\blacktriangleright\ 0 \downarrow$ | $0\ 2 \leftarrow$ | $1\ 2 \leftarrow$ |

□

Occupiamoci ora di dimostrare che se $f$ è Turing-computabile, allora $f$ è ricorsiva, vedremo una cosa più forte, ovvero che esiste una funzione ricorsiva che simula ogni funzione Turing-computabile.

**Osservazione 5.42** — Ricordando le definizioni di `car` e `cdr`, come rispettivamente inversa sulla prima e sulla seconda componente di `cons`, essendo $\texttt{cons} : \mathbb{N}^2 \hookrightarrow \mathbb{N} \setminus \{0\}$, possiamo estendere le definizioni di `car` e `cdr` in 0, ponendo $\texttt{car}(0) = \texttt{cdr}(0) = 0$.

**Definizione 5.43** (Codifica di una lista)**.** Definiamo $\langle a_1, \dots, a_n \rangle$, una **codifica di una lista** di numeri naturali, come il numero:

$$\texttt{cons}(a_1, \texttt{cons}(a_2, \dots \texttt{cons}(a_n, 0) \dots))$$

ovvero codifichiamo una sequenza di numeri in un numero utilizzando la funzione `cons` iterata. Alternativamente, se ne può dare una definizione ricorsiva:

- $\langle\rangle := 0$,
- $\langle a_1, \dots, a_n \rangle := \texttt{cons}(a_1, \langle a_2, \dots, a_n \rangle)$.

- osservare che questa è compatibile con la prima definizione, in quanto $\langle a \rangle = \texttt{cons}(a, \langle\rangle) = \texttt{cons}(a, 0)$.

**Osservazione 5.44** (Iniettività della codifica di una lista) — Tale codifica è iniettiva, e si verifica per induzione e usando che `cons` è iniettiva.

**Esempio 5.45**

La lista $\langle 7, 15, 0, 9 \rangle$ viene codificata come $\texttt{cons}(7, \texttt{cons}(15, \texttt{cons}(0, \texttt{cons}(9, 0))))$.

$\langle 7, 15, 0, 9 \rangle =$

$= \texttt{cons}(7, \texttt{cons}(15, \texttt{cons}(0, \texttt{cons}(9, 0))))$

$= (7, \cdot)\ (15, \cdot)\ (0, \cdot)\ (9, 0)$

**Definizione 5.46** (Funzioni sulle liste)**.** Definiamo le seguenti funzioni PR sulle liste:

- $\texttt{nth}(\langle a_0, \dots, a_n \rangle, k) := a_k$ se $k \leq n$, e 0 altrimenti,
- $\texttt{length}(\langle a_1, \dots, a_n \rangle) := n$,
- $\texttt{cat}(\langle a_1, \dots, a_m \rangle, \langle b_1, \dots, b_n \rangle) := \langle a_1, \dots, a_m, b_1, \dots, b_n \rangle$,
- $\texttt{head}(\langle a_1, \dots, a_n \rangle, k) := \langle a_1, \dots, a_k \rangle$ se $k \leq n$, e $\langle a_1, \dots, a_n \rangle$ altrimenti,

**Esercizio 5.47** (Le funzioni sulle liste sono PR)**.** Verificare che le funzioni `nth`, `length`, `cat` e `head` sono PR.

*Soluzione.* Vediamo ora solo il caso di `nth`, `length` e `cat`. Per `nth` l'idea è che $\texttt{nth}(\cdot, k) = \texttt{car} \circ \texttt{cdr}^k$, tuttavia, non essendo $k$ fissato a priori, non è immediato che la composizione sia PR, per tale ragione introduciamo una funzione ausiliaria $g(l, k)$ che coincide con $\texttt{cdr}^k$, ed è PR in quanto definita per ricorsione primitiva via funzioni PR:

$$g(l, k) := \begin{cases} g(l, 0) := l \\ g(l, k+1) := \texttt{cdr}(g(l, k)) \end{cases}$$

ora $g(l,k)$ coincide con $\texttt{cdr}^k$, infatti il passo ricorsivo fatto $k$ volte corrisponde a $\texttt{cdr}^k$; inoltre $\texttt{nth}(l,k) = \texttt{car}(g(l,k))$, infatti, se $l = \langle a_0, \dots, a_n \rangle$, allora $g(l,k) = \langle a_k, \dots, a_n \rangle$, e prendendo `car` si ottiene $a_k$ se $k \leq n$, mentre $g(l,k) = 0$ se $k > n$, in quanto arriviamo con `cdr` a 0 e $\texttt{car}(0) = 0$. Pertanto $\texttt{nth}(l,k) = \texttt{car}(g(l,k)) = \texttt{car} \circ \texttt{cdr}^k(l)$.
Per `length` basterà osservare che $\texttt{length}(l) = \#\{i < l | \texttt{cdr}^i(l) \neq 0\}$, infatti $\{\texttt{cdr}^i(l) \neq 0\}$ è un predicato PR - il fatto che $\texttt{cdr}^i$ sia PR deriva dalla stessa definizione ricorsiva che se ne può dare come fatto sopra, e $l \mapsto \#\{i < l | \texttt{cdr}^i(l) \neq 0\}$ è una funzione che conta gli elementi di un predicato PR fino a $l$, che è quindi PR per un esempio precedente.
Per `cat` è sufficiente definire una funzione PR ausiliaria:

$$f(a,b,k) := \begin{cases} f(a,b,0) := b \\ f(a,b,k+1) := \texttt{cons}(\texttt{nth}(a,k), f(a,b,k)) \end{cases}$$

dove $\texttt{nth}(a,k)$ è l'elemento $a_k$ della lista $a$, e $f(a,b,k)$ è la concatenazione di $b$ con gli ultimi $k$ elementi di $a$; da questa si ha che $\texttt{cat}(a,b) = f(a,b,\texttt{length}(b))$. □

Per dimostrare che anche `head` è PR ci serve un lemma.

**Lemma 5.48** (Ricorsione sul (de)corso dei valori)
Sia $f : \mathbb{N}^{n+1} \to \mathbb{N}$ una funzione e $g : \mathbb{N}^{n+1} \to \mathbb{N}$ PR, tali che:

$$f(x_1, \dots, x_n, y) = g(x_1, \dots, x_n, \langle f(\mathbf{x}, y-1), f(\mathbf{x}, y-2), \dots, f(\mathbf{x}, 0) \rangle)$$

ovvero $f$ è definita in modo ricorsivo a partire da $g$, dove il parametro su cui ricorriamo è $y$, e la dipendenza è dalla lista dei valori assunta da $f$ per tutti i valori più piccoli di $y$; in questo caso si ha che $f$ è PR.

*Dimostrazione.* Sia $h(\mathbf{x}, y) := \langle f(\mathbf{x}, y-1), \dots, f(\mathbf{x}, 0) \rangle$, se riusciamo a dimostrare che $h$ è PR, segue immediatamente che $f$ è PR per composizione.
Per verificare che $h$ è PR, ne diamo una definizione per ricorsione primitiva:

$$h(\mathbf{x}, y) := \begin{cases} h(\mathbf{x}, 0) = \langle \rangle = 0 \\ h(\mathbf{x}, y+1) = \texttt{cons}(g(\mathbf{x}, h(\mathbf{x}, y)), h(\mathbf{x}, y)) \end{cases}$$

dove abbiamo usato l'ipotesi per cui $f(\mathbf{x}, y) = g(\mathbf{x}, h(\mathbf{x}, y))$. □

**Esempio 5.49** (Successione di Fibonacci)
Dal lemma precedente è facile dimostrare che la successione di Fibonacci `fib` è PR, infatti:

$$\texttt{fib}(x) = g(\langle \texttt{fib}(x-1), \dots, \texttt{fib}(0) \rangle)$$

e possiamo definire $g$ per casi, sui predicati PR $\{\texttt{length}(l) = 0\}$, $\{\texttt{length}(l) = 1\}$ e $\{\texttt{length}(l) > 1\}$, come segue:

$$g(l) := \begin{cases} 0 & \text{se } \texttt{length}(l) = 0 \\ 1 & \text{se } \texttt{length}(l) = 1 \\ \texttt{car}(l) + \texttt{car}(\texttt{cdr}(l)) & \text{altrimenti} \end{cases}$$

A questo punto possiamo anche verificare che `head` è PR.

*Soluzione.* Per `head` potremmo dare la seguente definizione ricorsiva:

$$\texttt{head}(l,k) := \begin{cases} \texttt{head}(l,0) := 0 \\ \texttt{head}(l,k+1) := \texttt{cons}(\texttt{car}(l), \texttt{head}(\texttt{cdr}(l),k)) \end{cases}$$

il problema è che nella ricorsione passo a $\texttt{cdr}(l)$, anziché ad $l$, per prendere la lista restante dopo aver preso il primo elemento, per cui non siamo in una ricorsione primitiva diretta su $k$ e quindi questo approccio non funziona.
Usiamo allora il lemma di ricorsione sul decorso dei valori e definiamo:

$$\texttt{head}(l,y) = g\left(l, \langle \texttt{head}(l,y-1), \dots, \texttt{head}(l,0)\rangle\right)$$

dove $g$ è PR e vale

$$g(l,s) := \begin{cases} 0 & \text{se } \texttt{length}(s) = 0, \\ \texttt{car}(s) & \text{se } \texttt{length}(s) > \texttt{length}(l), \\ \texttt{cat}(\texttt{car}(s), \texttt{cons}(\texttt{nth}(l, \texttt{length}(s)-1), 0)) & \text{altrimenti} \end{cases}$$

infatti, se $s = \langle \texttt{head}(l,y-1), \dots, \texttt{head}(l,0)\rangle$, allora $\texttt{car}(s) = \texttt{head}(l,y-1)$. Quando $\texttt{length}(s) = 0$ siamo nel caso base $y = 0$, e quindi il valore corretto è 0. Quando $\texttt{length}(s) > \texttt{length}(l)$ abbiamo già superato la lunghezza di $l$, perciò il prefisso richiesto non cresce più e resta uguale all'ultimo valore calcolato, cioè $\texttt{car}(s)$. Nel caso rimanente, aggiungiamo a $\texttt{head}(l,y-1) = \texttt{car}(s)$ l'elemento successivo di $l$, cioè $\texttt{nth}(l, \texttt{length}(s)-1)$, e otteniamo esattamente $\texttt{head}(l,y)$. Quindi siamo esattamente nel caso del lemma, e `head` è PR. □

**Teorema 5.50** (Funzione ricorsiva universale)
Esiste una funzione ricorsiva parziale $u : \mathbb{N}^2 \to \mathbb{N}_\perp$ tale che per ogni $f : \mathbb{N}^k \to \mathbb{N}_\perp$ Turing-computabile, esiste un numero $\ulcorner f \urcorner \in \mathbb{N}$, detto **codice** di $f$, tale che:

$$\forall x_1, \dots, x_k \in \mathbb{N}\ f(x_1, \dots, x_k) = u(\ulcorner f \urcorner, \langle x_1, \dots, x_k\rangle)$$

ovvero esiste una funzione universale $u$ che simula ogni funzione Turing-computabile, e le basta il codice della funzione e la codifica degli input.

**Corollario 5.51** (Turing-computabilità $\Rightarrow$ ricorsività)
Se $f : \mathbb{N}^k \to \mathbb{N}_\perp$ è Turing-computabile, allora $f$ è ricorsiva.

*Dimostrazione.* La dimostrazione segue dal teorema osservando che:

$$f(\cdot) = u(\ulcorner f \urcorner, \cdot)$$

$u$ è ricorsiva dal teorema ed è composta con una costante. □

**Corollario 5.52** (Esistenza di una funzione ricorsiva universale per ogni arità)
$\forall n \in \mathbb{N}\ \exists u_n : \mathbb{N}_\perp^{n+1} \to \mathbb{N}_\perp$ ricorsiva, tale che, data $f : \mathbb{N}_\perp^n \to \mathbb{N}_\perp$ ricorsiva, esiste un codice $\ulcorner f \urcorner$ tale che:

$$\forall x_1, \dots, x_n \in \mathbb{N}_\perp\ f(x_1, \dots, x_n) = u_n(\ulcorner f \urcorner, x_1, \dots, x_n)$$

*Dimostrazione.* Dal teorema precedente, $f$ è ricorsiva, quindi Turing-computabile, per cui esiste $\ulcorner f \urcorner$ tale che $f(\cdot) = u(\ulcorner f \urcorner, \cdot)$, a questo punto è sufficiente definire $u_n$ come:

$$u_n(\ulcorner f \urcorner, x_1, \dots, x_n) := u(\ulcorner f \urcorner, \langle x_1, \dots, x_n \rangle)$$

essendo la codifica una composizione di $n$ (con $n$ fisso) funzioni `cons`, che è PR, $u_n$ è ricorsiva per composizione di funzioni ricorsive. □

**Osservazione 5.53** (Tesi di Church) — "La classe delle funzioni ricorsive coincide con la classe delle funzioni effettivamente calcolabili". Tale tesi non è un'affermazione matematica, ma è l'idea che esprime la filosofia dietro alle varie nozioni di calcolabilità che abbiamo dato - e anche non dato, e.g. il $\lambda$-calcolo, che è quella di descrivere tutte le funzioni che possono essere calcolate effettivamente nella realtà. La tesi di Church può essere espressa in maniera più forte o debole in base a quanto si richieda buona la corrispondenza tra computabilità pratica e modello teorico, noi ci siamo limitati al risultato semantico - ovvero a quale sia il valore effettivo, ma si può dare anche una corrispondenza tenendo conto della complessità, ed a seconda di quanto uno richiede che la corrispondenza sia precisa viene una tesi più forte o debole.

Passiamo alla dimostrazione del teorema.

*Dimostrazione.* Sia $f : \mathbb{N}^k \to \mathbb{N}_\perp$ Turing-computabile, e sia $M$ una macchina di Turing che implementa $f$. Per definizione sappiamo che $\{0, 1, \blacktriangleright\} \subseteq \Sigma$ e $\{0, 1\} \subseteq Q$, inoltre, essendo entrambi insiemi finiti, possiamo assumere che $\Sigma, Q \subseteq \mathbb{N}$ - assumiamo che $\blacktriangleright$ corrisponda a 2, ricordiamo infine che, per le nostre convenzioni, rappresentiamo $\{\leftarrow, \downarrow, \rightarrow\}$ come $\{0, 1, 2\}$.
Ora codifichiamo con un numero naturale una configurazione e la funzione $\delta$ di transizione di $M$. Per la codifica di una configurazione usiamo semplicemente una lista:

$$\mathbb{N} \ni \text{configurazione} := \langle \text{stato, posizione, nastro} \rangle$$

dove stato e posizione sono i numeri naturali stessi, mentre la funzione nastro, che va da $\mathbb{N}$ a $\Sigma$, è codificata come una lista di numeri naturali, ovvero la configurazione $a_0 a_1 \dots a_n 00 \dots$ corrisponde alla lista $\langle a_0, a_1, \dots, a_n \rangle$.

**Osservazione 5.54** (Non iniettività della codifica del nastro) — Si osserva che la codifica della funzione nastro introdotta non è iniettiva, infatti $\langle a_0, a_1, \dots, a_n \rangle$ codifica la stessa configurazione di $\langle a_0, a_1, \dots, a_n, 0 \rangle$ e così via, tuttavia questo non è un problema, in quanto non si perde alcuna informazione rispetto a quale sia la configurazione considerata.

Per codificare la funzione di transizione $\delta$, associamo a tale funzione il numero $\ulcorner \delta \urcorner$ tale che:

$$\texttt{nth}(\ulcorner \delta \urcorner, \langle \text{stato}, \text{simbolo} \rangle) = \langle \text{stato'}, \text{simbolo'}, \text{direzione} \rangle$$

ovvero se penso $\ulcorner \delta \urcorner$ non come un numero, ma come la funzione da $\mathbb{N}^2$ a $\mathbb{N}^3$ dico che in ogni casella del suo dominio c'è la tripla immagina, questo però lo vedo in termini puramente di codifiche. Il fatto che $\ulcorner \delta \urcorner$ esista è garantito dal fatto che la funzione, insiemisticamente, è contenuta in un sottoinsieme finito di $\mathbb{N}^5$ e quindi posso letteralmente considerare $m := \max\{\langle \text{stato}, \text{simbolo} \rangle\}$, chiamare $a_i := \langle \text{stato}', \text{simbolo}', \text{direzione} \rangle$ con $i = \langle \text{stato}, \text{simbolo} \rangle$, questo se $\delta$ manda effettivamente la configurazione corrispondente all'indice in quella corrispondente alla tripla, altrimenti $a_i := 0$, infine porre $\ulcorner \delta \urcorner := \langle a_0, \dots, a_m \rangle$, quindi, semplicemente, listo gli output di $\delta$ indicizzati dagli input.

**Osservazione 5.55** (Non iniettività della codifica di $\delta$) — Per la stessa ragione di prima, la codifica $\ulcorner\delta\urcorner$ non è iniettiva.

A questo punto abbiamo bisogno di una funzione che simuli uno step di $M$ a partire dalla codifica di una configurazione.

**Lemma 5.56** (Funzione PR che simula uno step della TM)
Esiste una funzione PR $\texttt{step} : \mathbb{N}^2 \to \mathbb{N}$, tale che mappa ogni codifica di configurazione $c$, vale:

$$c \overset{M}{\to} \texttt{step}(\ulcorner\delta\urcorner, c)$$

ovvero `step` è una funzione che, data la codifica di una configurazione e quella della funzione di transizione, restituisce la codifica della configurazione che si ottiene applicando uno step della macchina di Turing.

*Dimostrazione.* Con la codifica data all'inizio abbiamo che:

- stato $= \texttt{nth}(c, 0)$,
- posizione $= \texttt{nth}(c, 1)$,
- simbolo $= \texttt{nth}(c, 2)$

ora il nastro è a sua volta la codifica di una lista:

$$\text{nastro} = \langle \underbrace{a_1, \ldots, a_{p-1}}_{=:\alpha}, \underbrace{a_p}_{=:\sigma}, \underbrace{a_{p+1}, \ldots}_{=:\beta} \rangle$$

dove:

- $\alpha := \texttt{head}(\text{nastro}, \text{posizione} \dot{-} 1) = \langle a_1, \ldots, a_{p-1} \rangle = \texttt{head}(\texttt{nth}(c, 2), \texttt{nth}(c, 1) \dot{-} 1)$,
- $\sigma := \texttt{nth}(\text{nastro}, \text{posizione}) = a_p = \texttt{nth}(\texttt{nth}(c, 2), \texttt{nth}(c, 1))$,
- $\beta := \texttt{cdr}^{\text{posizione}}(\text{nastro}) = \langle a_{p+1}, \ldots \rangle = \texttt{cdr}^{\texttt{nth}(c,1)}(\texttt{nth}(c, 2))$.

Quello che vogliamo fare è calcolare $\delta(\text{stato}, \text{simbolo})$, ovvero $\texttt{nth}(\ulcorner\delta\urcorner, \langle \text{stato}, \text{simbolo} \rangle) = \texttt{nth}(\ulcorner\delta\urcorner, \langle \texttt{nth}(c, 0), \sigma \rangle)$, che dà la tripla codificata da $\langle \text{stato}', \text{simbolo}', \text{direzione} \rangle :=$`update`. Da questo posso scrivere la configurazione di arrivo, infatti il nuovo nastro sarà:

$$\text{nastro}' := \texttt{cat}(\alpha, \texttt{cons}(\texttt{nth}(\texttt{update,1}), \beta))$$

dove $\texttt{nth}(\texttt{update,1})$ è simbolo$'$; il nuovo stato sarà $\texttt{nth}(\texttt{update,0})$ e la nuova posizione sarà $(\text{posizione} + \texttt{nth}(\texttt{update,2})) \dot{-} 1$. Segue quindi che la nuova configurazione sarà codificata da:

$$\langle \texttt{nth}(\texttt{update,0}), (\texttt{nth}(c, 1) + \texttt{nth}(\texttt{update,2})) \dot{-} 1, \texttt{cat}(\alpha, \texttt{cons}(\texttt{nth}(\texttt{update,1}), \beta)) \rangle$$

e chiamiamo questo numero $\texttt{step}(\ulcorner\delta\urcorner, c)$, che è PR per composizione di funzioni PR. $\square$

Da questa vogliamo scrivere la funzione che, in un certo numero di step, a partire da un certo input e dal codice di $f$, restituisce l'output.
Iniziamo definendo una codifica dell'input:

$$\texttt{codifica_input}(\langle x_1, \ldots, x_n \rangle) := \langle 1, 0, \texttt{cons}(\blacktriangleright, \texttt{nastro}(\langle x_1, \ldots, x_k \rangle)) \rangle$$

dove:

$$\texttt{nastro}(\langle x_1, \dots, x_k \rangle) := \langle \texttt{unario}(x_1), \texttt{cons}(0, \texttt{nastro}(\langle x_2, \dots, x_k \rangle)) \rangle$$

e $\texttt{unario}(x) := \langle 1, \dots, 1 \rangle$ è la codifica unaria di $x$, tutte queste cose sono PR per composizione di funzioni PR, va verificato.
Ora, la funzione che itera uno step $(n, c, \ulcorner \delta \urcorner) \mapsto \texttt{step}^n(\ulcorner \delta \urcorner, c)$ è PR, la verifica deve usare il lemma di ricorsione sul decorso dei valori. A questo punto usiamo l'operatore di minimizzazione per procurarci l'$n$ in cui la macchina termina, consideriamo:

$$\texttt{n}(c, \ulcorner \delta \urcorner) := \mu_n(\texttt{nth}(\texttt{step}^n(\ulcorner \delta \urcorner, c), 0) = 0)$$

ovvero $\texttt{n}$ rappresenta il numero di passi in cui la TM termina a partire da input codificato da $c$, tale funzione non è PR in quanto, per un certo input, la macchina potrebbe non terminare, per cui $\texttt{n}$ è ricorsiva parziale. E siamo alla fine perché ora per ottenere l'output basta prendere il nastro della configurazione finale, ovvero, indicando con $\langle \mathbf{x} \rangle = \langle x_1, \dots, x_k \rangle$:

$$\texttt{nth}(\texttt{step}^{\texttt{n}(\texttt{codifica_input}(\langle \mathbf{x} \rangle), \ulcorner \delta \urcorner)}(\ulcorner \delta \urcorner, \texttt{codifica_input}(\langle \mathbf{x} \rangle))), 2)$$

a questo punto prendo il nastro della configurazione finale e ci applico una funzione che conta gli 1, che darà proprio l'output di $f$, posso definire:

$$\texttt{conta_uni}(l) := \begin{cases} 0 & \text{se } \texttt{car}(l) = 0 \\ 1 + \texttt{conta_uni}(\texttt{cdr}(l)) & \text{altrimenti} \end{cases}$$

e quindi $u$ si definisce componendo $\texttt{conta_uni}$ con la funzione scritta sopra, tale composizione, eccetto per la $\texttt{n}$, che fa uso di minimizzazione, è PR, per cui la composizione è ricorsiva parziale. □

**Corollario 5.57** (Forma normale per le funzioni ricorsive)
$\forall k \in \mathbb{N}$ esiste $T_k : \mathbb{N}^{k+2} \to \mathbb{N}$ predicato PR, e una funzione $U : \mathbb{N} \to \mathbb{N}$ PR, tali che $\forall f : \mathbb{N}_\perp^k \to \mathbb{N}_\perp$ ricorsiva (parziale) vale:

$$\forall x_1, \dots, x_k \; f(x_1, \dots, x_k) \neq \perp \iff \exists n \; T_k(\ulcorner f \urcorner, x_1, \dots, x_k, n) = 0$$

e, quando questo avviene:

$$f(x_1, \dots, x_k) = U\big(\mu_n(T_k(\ulcorner f \urcorner, x_1, \dots, x_k, n) = 0)\big)$$

ovvero se $f$ termina su un certo input, allora l'output è dato da $U$ valutata in $\mu_n(T_k(\ulcorner f \urcorner, x_1, \dots, x_k, n) = 0)$.

*Dimostrazione.* Mettiamoci nello stesso setting della dimostrazione del teorema precedente. Definiamo la configurazione raggiunta dopo $t$ passi a partire da un certo input e dal codice di $f$ come:

$$c_k(\ulcorner f \urcorner, x_1, \dots, x_k, t) := \texttt{step}^t\big(\ulcorner f \urcorner, \texttt{codifica_input}(\langle x_1, \dots, x_k \rangle)\big)$$

che è PR, essendo composizione di PR. Codifichiamo una coppia step-configurazione $(t, c)$ come $n = \langle t, c \rangle$, e definiamo esplicitamente:

$$T_k(\ulcorner f \urcorner, x_1, \dots, x_k, n) := \begin{cases} 0 & \text{se } \begin{cases} \texttt{nth}(n, 1) = c_k(\ulcorner f \urcorner, x_1, \dots, x_k, \texttt{nth}(n, 0)) \\ \texttt{nth}(\texttt{nth}(n, 1), 0) = 0 \end{cases} \\ 1 & \text{altrimenti.} \end{cases}$$

ovvero se la configurazione raggiunta dopo $t$ passi, a partire dalla configurazione iniziale è proprio $c$, $c = c_k(\ulcorner f \urcorner, x_1, \ldots, x_k, t)$, e se lo stato di tale configurazione è 0, i.e. la macchina si ferma, allora $T_k$ restituisce 0, altrimenti restituisce 1. Abbiamo dato la definizione per casi con le codifiche in modo da avere una definizione PR di $T_k$. In questo modo:

$\boxed{\Longrightarrow}$ se $f(x_1, \ldots, x_k) \neq \perp$, esiste un tempo di arresto $t$ e la relativa configurazione $c = c_k(\ulcorner f \urcorner, x_1, \ldots, x_k, t)$ con stato 0, e quindi scegliendo proprio $n = \langle t, c \rangle$ si ha che $T_k(\ulcorner f \urcorner, x_1, \ldots, x_k, n) = 0$;

$\boxed{\Longleftarrow}$ se esiste $n$ con $T_k(\ulcorner f \urcorner, x_1, \ldots, x_k, n) = 0$, allora per la definizione per casi, data sopra, $n$ codifica una configurazione di arresto raggiunta in un certo numero finito di passi - per la precisione $\texttt{nth}(n, 0)$, quindi la computazione termina e $f(x_1, \ldots, x_k) \neq \perp$.

Mettiamoci nel caso in cui $f(x_1, \ldots, x_k) \neq \perp$, allora esiste un tempo di arresto minimo:

$$n_0 := \mu_n(T_k(\ulcorner f \urcorner, x_1, \ldots, x_k, n))$$

per cui il nastro della configurazione raggiunta dopo $n_0$ passi è data da $\texttt{nth}(\texttt{nth}(n_0, 1), 2)$, quindi non bisogna far altro che convertire il nastro di tale configurazione in output, e questo è esattamente quello che fa $U$:

$$U(n) := \texttt{conta_uni}(\texttt{nth}(\texttt{nth}(n, 1), 2))$$

ed $U$ è PR per composizione di funzioni PR. □

Per cui, d'ora in avanti, con forma normale per le funzioni ricorsive, ci riferiremo al test $T_k$ che verifica se la funzione è PR, ed alla $U$ che ne estrae il testimone.

## §5.6 Teoremi di punto fisso e $s$-$m$-$n$

**Teorema 5.58** ($s - m - n$)
Esiste, per ogni $m, n \in \mathbb{N}$, una funzione ricorsiva totale (in realtà PR):

$$s_n^m : \mathbb{N}^{m+1} \to \mathbb{N}$$

tale che per ogni $\alpha, x_1, \ldots, x_m, y_1, \ldots, y_n \in \mathbb{N}$ vale:

$$u_{n+m}(\alpha, \langle x_1, \ldots, x_m, y_1, \ldots, y_n \rangle) = u_n(s_n^m(\alpha, x_1, \ldots, x_m), \langle y_1, \ldots, y_n \rangle)$$

**Esempio 5.59** (Somma PR)
Sia $f : \mathbb{N}^2 \to \mathbb{N} : (x, y) \mapsto x + y$, allora esiste $s_1^1 : \mathbb{N}^2 \to \mathbb{N}$ tale che:

$$u_2(\ulcorner f \urcorner, x, y) = u_1(s_1^1(\ulcorner f \urcorner, x), y)$$

ovvero $s_1^1$ è una funzione che, dato il codice di una funzione calcolabile $f$ di due variabili, e un input $x$, restituisce il codice di una funzione che simula $f$ con il primo input fissato a $x$, e quindi che prende un solo input $y$ e restituisce $f(x, y)$. Ad esempio per $x = 7$, ho $\ulcorner g \urcorner := s_1^1(\ulcorner f \urcorner, 7)$, e $g(y) = f(7, y) = 7 + y$, e vale:

$$g(y) = u_1(\ulcorner g \urcorner, y) = u_1(s_1^1(\ulcorner f \urcorner, 7), y) = u_2(\ulcorner f \urcorner, 7, y) = f(7, y) = 7 + y$$

**Teorema 5.60** ((Secondo) teorema di punto fisso)
Sia $h : \mathbb{N} \to \mathbb{N}$ una funzione ricorsiva totale, e sia $n \in \mathbb{N}$, allora esiste $\alpha$ tale che:

$$\forall x_1, \dots, x_n \; u_n(\alpha, x_1, \dots, x_n) = u_n(h(\alpha), x_1, \dots, x_n)$$

**Esempio 5.61** (Successione di Fibonacci PR - v.2)
Consideriamo la classica funzione di Fibonacci $f : \mathbb{N} \to \mathbb{N}$ definita da:

$$f(n) := \begin{cases} 0 & \text{se } n = 0 \\ 1 & \text{se } n = 1 \\ f(n \dot{-} 1) + f(n \dot{-} 2) & \text{altrimenti} \end{cases}$$

che sappiamo essere già PR per il lemma sulla ricorsione sul decorso dei valori, ma vediamo come dimostrarlo usando il secondo teorema di punto fisso. Definiamo la funzione:

$$g(\alpha, n) := \begin{cases} 0 & \text{se } n = 0 \\ 1 & \text{se } n = 1 \\ u_1(\alpha, n \dot{-} 1) + u_1(\alpha, n \dot{-} 2) & \text{altrimenti} \end{cases}$$

che questa sia una funzione ricorsiva (non necessariamente totale) è ovvio - è definita per casi su un predicato PR, a questo punto definisco:

$$h(\alpha) := s_1^1(\ulcorner g \urcorner, \alpha)$$

cioè la funzione tale che $h(\alpha) = g(\alpha, \cdot)$. Sia $\widetilde{\alpha}$ un punto fisso di $h$, per cui $u_1(\widetilde{\alpha}, x) = u_1(h(\widetilde{\alpha}), x)$, e dico che $f(x) := u_1(\widetilde{\alpha}, x)$, per cui la successione di Fibonacci coincide con una funzione ricorsiva. Vediamo che $f$ coincide con la funzione che ha codice $\widetilde{\alpha}$:

$$\begin{aligned} u_1(\widetilde{\alpha}, x) &= u_1(h(\widetilde{\alpha}), x) && \text{(punto fisso)} \\ &= u_1(s_1^1(\ulcorner g \urcorner, \widetilde{\alpha}), x) && \text{(definizione di } h\text{)} \\ &= u_2(\ulcorner g \urcorner, \widetilde{\alpha}, x) && \text{(definizione di } s_1^1\text{)} \\ &= g(\widetilde{\alpha}, x) \quad \forall x \in \mathbb{N} && \text{(definizione di funzione universale)} \end{aligned}$$

e, per la definizione di $g$, si ottiene che $u_1(\widetilde{\alpha}, x)$ soddisfa esattamente la definizione ricorsiva di $f(x)$. Se dimostro nella metateoria che c'è una sola funzione che soddisfa la definizione ricorsiva di $f$ - ed in ZFC è il teorema di ricorsione numerabile, allora $f$ coincide con $u_1(\widetilde{\alpha}, x)$, e quindi $f$ è ricorsiva.[a]

[a] Morale: la nozione di funzione ricorsiva che aveva Gödel è quella giusta.

Dimostriamo il teorema $s - m - n$[47].

*Dimostrazione.* Iniziamo con l'osservare che basta fare il caso $m = 1$, infatti se riesco a dimostrare che esiste $s_n^1$, allora posso iterare $m$ volte $s_n^1$ per ottenere $s_n^m$ nel modo seguente:

$$s_n^m(\alpha, x_1, \dots, x_m) := s_n^1\big(s_{n+1}^1(\dots s_{n+m-1}^1(\alpha, x_1) \dots, x_{m-1}), x_m\big)$$

[47] Si tratterà di una sorta di half-evaluation per gli informatici.

e RHS messo in $u_n$ dà:

$$\begin{aligned}
u_n(s_n^m(\alpha, x_1, \dots, x_m), \mathbf{y}) &= u_n\big(s_n^1(s_{n+1}^1(\dots s_{n+m-1}^1(\alpha, x_1) \dots, x_{m-1}), x_m), \mathbf{y}\big) \\
&= u_{n+1}\big(s_{n+1}^1(\dots s_{n+m-1}^1(\alpha, x_1) \dots, x_{m-1}), x_m, \mathbf{y}\big) \\
&= u_{n+2}\big(s_{n+2}^1(\dots s_{n+m-1}^1(\alpha, x_1) \dots, x_{m-2}), x_{m-1}, x_m, \mathbf{y}\big) \\
&= \dots \\
&= u_{n+m-1}(s_{n+m-1}^1(\alpha, x_1), x_2, \dots, x_m, \mathbf{y}) \\
&= u_{n+m}(\alpha, x_1, x_2, \dots, x_m, \mathbf{y})
\end{aligned}$$

quindi vogliamo solo dimostrare che:

$$\forall \alpha, x_1 \in \mathbb{N}\ \forall \mathbf{y} \in \mathbb{N}^n\ u_{n+1}(\alpha, x_1, \mathbf{y}) = u_n(s_n^1(\alpha, x_1), \mathbf{y})$$

Fisso quindi $\alpha$, $x_1$ e $\mathbf{y}$, e considero la TM che implementa la funzione codificata da $\alpha$, l'idea è quella di costruire una macchina che riceve in input $1 \blacktriangleright y_1 \dots y_n$ e restituisce $1_\alpha \blacktriangleright x_1 y_1 \dots y_n$, ovvero una macchina che aggiunge $x_1$ davanti all'input, e poi simula la macchina che implementa la funzione codificata da $\alpha$. Unendo tale macchina e quella che implementa la funzione codificata da $\alpha$ ottengo la macchina che implementa la funzione codificata da $s_n^1(\alpha, x_1)$, e quindi ho questa funzione ricorsiva.
Consideriamo $1 \blacktriangleright y_1 \dots y_n$, WLOG possiamo supporre di avere già uno 0 in fondo - altrimenti si fa `push` di uno dei valori, e si applica la macchina che implementa la funzione costante 0, quindi $1 \blacktriangleright y_1 \dots y_n 0$, a questo punto trasformo lo 0 alla fine in $x_1$:

$$1 \to \underbrace{\texttt{exec}_{n+1,n+1} s \to \texttt{exec}_{n+1,n+1} s \to \dots \to \texttt{exec}_{n+1,n+1} s}_{x_1 \text{ volte}} \to 2$$

con $2 \blacktriangleright y_1 \dots y_n x_1$, tutto ciò che rimane da fare è riscrivere l'input nell'ordine corretto e cancellare quello che rimane all'inizio:

$$2 \to \texttt{push}_{n+1,1} \to \texttt{push}_{n+2,2} \to \dots \to \texttt{push}_{2n,n} \to 3$$

con $3 \blacktriangleright y_1 \dots y_n x_1 y_1 \dots y_n$, e infine:

$$3 \to \texttt{rem}_{2n+1,1} \to \texttt{rem}_{2n,1} \to \dots \to \texttt{rem}_{n+2,1} \to 4$$

con $4 \blacktriangleright x_1 y_1 \dots y_n$, e da qui si manda in $0 \blacktriangleright x_1 y_1 \dots y_n$ (che poi verrà attaccato allo stato iniziale $1_\alpha$ come al solito).
Osservare infine che la macchina costruita termina sempre, perché è composta da un numero finito di step e nessuna delle macchine che la compongono è non-terminante, per di più abbiamo usato solo macchine elementari che corrispondono a funzioni PR, per cui $s_n^1$ è PR, e per composizione anche $s_n^m$ è PR. □

**Esercizio 5.62** (Effetto collaterale del teorema $s - m - n$)**.** Dimostrare che esiste una funzione $\texttt{k} : \mathbb{N} \to \mathbb{N}$, ricorsiva totale, per cui:

$$\forall n \in \mathbb{N}\ n = u_0(\texttt{k}(n))$$

ovvero c'è una funzione ricorsiva totale che, dato un numero $n$, restituisce un codice di una funzione costante sempre $n$.

In pratica esiste una funzione che, valutata in un numero dà proprio il codice della funzione costante di quel numero.

*Soluzione.* Applichiamo il teorema $s-m-n$ alla funzione $\mathrm{id}_{\mathbb{N}} : \mathbb{N} \to \mathbb{N}$, che è PR, per cui usa $m = 1$ (l'argomento della funzione che simulo) e $n = 0$ (input aggiuntivi), ottengo $s_0^1 : \mathbb{N}^2 \to \mathbb{N}$ tale che:

$$u_0(s_0^1(\ulcorner \mathrm{id}_{\mathbb{N}} \urcorner, n)) = u_1(\ulcorner \mathrm{id}_{\mathbb{N}} \urcorner, n) = \mathrm{id}_{\mathbb{N}}(n) = n$$

per cui basta definire:

$$\mathtt{k}(n) := s_0^1(\ulcorner \mathrm{id}_{\mathbb{N}} \urcorner, n)$$

che è ricorsiva totale, perché $s_0^1$ è PR. □

**Esercizio 5.63** (Generalizzazione dell'esercizio precedente)**.** Data $f : \mathbb{N}_\perp \to \mathbb{N}_\perp$ ricorsiva parziale, dimostrare che esiste $\mathtt{k}f : \mathbb{N} \to \mathbb{N}$ ricorsiva totale tale che:

$$\forall n \; f(n) \in \mathbb{N} \implies f(n) = u_0(\mathtt{k}f(n))$$

In questo caso vogliamo fare la stessa cosa di prima, ma per una funzione non costante e soprattutto non necessariamente totale.

*Soluzione.* Esattamente come per l'esercizio precedente, applichiamo il teorema $s-m-n$ alla funzione $f : \mathbb{N}_\perp \to \mathbb{N}_\perp$, con $m = 1$ e $n = 0$, ottenendo $s_0^1 : \mathbb{N}^2 \to \mathbb{N}$ tale che:

$$u_0(s_0^1(\ulcorner f \urcorner, n)) = u_1(\ulcorner f \urcorner, n) = f(n)$$

essendo $s_0^1$ PR, è sufficiente definire:

$$\mathtt{k}f(n) := s_0^1(\ulcorner f \urcorner, n)$$

che è ricorsiva totale. □

**Osservazione 5.64** (Eager evaluation vs lazy evaluation) — Nell'esercizio precedente, se avessimo provato ad usare come soluzione $\mathtt{k}f(n) := \mathtt{k}(f(n))$, avremmo ottenuto qualcosa che funziona solo nel caso di funzioni ricorsive totali, perché se $f(n) = \perp$, allora $\mathtt{k}(f(n))$ non è definita - visto che $s_0^1$ è PR, e quindi non è ricorsiva totale.
Abbiamo già accennato come il teorema $s-m-n$ sia quello che gli informatici chiamano half-evaluation, perché dato il codice di una funzione e un input, restituisce il codice di una funzione che simula la prima con l'input fissato, e quindi è come se avessimo fatto metà della valutazione.
In questo contesto, l'esempio precedente ci permette di osservare la differenza tra due diversi tipi di valutazione, la valutazione eager (usata da linguaggi come C, Java, Python), in cui gli argomenti di una funzione vengono valutati prima di essere passati alla funzione, quindi nel caso di $\mathtt{k}(f(n))$ si valuta prima $f(n)$, e se $f(n) = \perp$ allora si ottiene un errore di runtime; e la valutazione lazy (usata da linguaggi come Haskell), in cui gli argomenti di una funzione vengono valutati solo quando sono effettivamente usati, quindi nel caso di $\mathtt{k}f(n) = s_0^1(\ulcorner f \urcorner, n)$, si ottiene un codice di una funzione che simula $f$ con input $n$, e se $f(n) = \perp$ allora si ottiene un errore di runtime solo quando si prova ad eseguire la funzione codificata da $\mathtt{k}f(n)$, e non prima.

Vediamo infine la dimostrazione del (secondo) teorema di punto fisso.

*Dimostrazione.* Fissiamo $h : \mathbb{N} \to \mathbb{N}$ ricorsiva totale e $n \in \mathbb{N}$, e cerchiamo $\alpha$ tale che $u_n(\alpha, \mathbf{y}) = u_n(h(\alpha), \mathbf{y})$ per ogni $\mathbf{y} \in \mathbb{N}^n$.
Euristicamente, se potessimo applicare qualsiasi cosa a qualsiasi cosa - come si fa nel

$\lambda$-calcolo, definiamo $F(x) := h(x(x))$, e poi consideriamo $F(F) = h(F(F))$, e quindi $F(F)$ è un punto fisso di $h$. Questo non possiamo farlo nel nostro contesto, tuttavia, a meno di passare alle codifiche di funzioni, possiamo fare qualcosa di simile.
Vediamo come la prima cosa possibile, per imitare l'euristica, non funziona, infatti se provassimo a usare $u_1(x, x)$ per $x(x)$ avremmo:

$$F(x) := h(u_1(x, x))$$

da cui $F(\ulcorner F \urcorner) = h(u_1(\ulcorner F \urcorner, \ulcorner F \urcorner))$, cioè $h(F(\ulcorner F \urcorner))$, che è un punto fisso. Il problema sta nel fatto che $F(\ulcorner F \urcorner)$ potrebbe fare $\perp$, che è contro la tesi che il punto fisso stia in $\mathbb{N}$.
Definiamo allora:

$$g(x, \mathbf{y}) := u_n(h(u_1(x, x)), \mathbf{y})$$

e definiamo $F(x) := s_n^1(\ulcorner g \urcorner, x)$, per cui $F$ è una funzione ricorsiva totale, in tal modo, sfruttando l'half-evaluation, abbiamo ottenuta la stessa cosa di prima, perché $s_1^1$ si comporta come se avessimo fatto $g(x, \cdot)$, ma mantenendo la totalità e quindi usando $x = \ulcorner F \urcorner$ otteniamo:

$$\begin{aligned} u_n(F(\ulcorner F \urcorner), \mathbf{y}) &= u_n(s_n^1(\ulcorner g \urcorner, \ulcorner F \urcorner), \mathbf{y}) \\ &= u_{n+1}(\ulcorner g \urcorner, \ulcorner F \urcorner, \mathbf{y}) \\ &= g(\ulcorner F \urcorner, \mathbf{y}) \\ &= u_n(h(u_1(\ulcorner F \urcorner, \ulcorner F \urcorner)), \mathbf{y}) \\ &= u_n(h(F(\ulcorner F \urcorner)), \mathbf{y}) \end{aligned}$$

e quindi $\alpha := F(\ulcorner F \urcorner)$ è un punto fisso di $h$. $\square$

# §6 Gerarchie aritmetiche e decidibilità

## §6.1 Insiemi decidibili, semidecidibili e ricorsivamente enumerabili

**Definizione 6.1** (Insiemi decidibili, semidecidibili e ricorsivamente enumerabili)**.** Sia $R \subseteq \mathbb{N}^k$, allora $R$ è:

- **semidecidibile** se esiste una funzione $f : \mathbb{N}^k \to \mathbb{N}_\perp$ ricorsiva parziale tale che $R = \{\mathbf{x} \in \mathbb{N}^k | f(\mathbf{x}) = 0\}$, i.e. se $R$ è il luogo di zero di una funzione ricorsiva parziale,
- **decidibile** se esiste $f$ ricorsiva totale $f : \mathbb{N}^k \to \mathbb{N}$ tale che $R = \{\mathbf{x} \in \mathbb{N}^k | f(\mathbf{x}) = 0\}$, i.e. se $R$ è il luogo di zero di una funzione ricorsiva totale,
- **ricorsivamente enumerabile** se esiste $f : \mathbb{N} \to \mathbb{N}^k$ ricorsiva totale tale che $R = \{f(n) | n \in \mathbb{N}\}$, i.e. se $R$ è l'immagine di una funzione ricorsiva totale.

**Proposizione 6.2** (Equivalenza delle nozioni di decidibilità)
Sia $R \subseteq \mathbb{N}^k$, allora le seguenti sono equivalenti:

1. $R$ è semidecidibile,
2. $R$ è dominio di una funzione ricorsiva parziale, $R = \{\mathbf{x} \in \mathbb{N}^k | \exists y\ f(\mathbf{x}) = y \neq \perp\}$,
3. $R$ è ricorsivamente enumerabile.

Morale: essere dominio, luogo di zero o immagine totale di una funzione ricorsiva sono tutte la stessa cosa.

*Dimostrazione.* Vediamo le implicazioni in ordine circolare.

$\boxed{1. \Rightarrow 2.}$ se $R = \{\mathbf{x} \in \mathbb{N}^k | f(\mathbf{x}) = 0\}$, $R$ non è già il dominio di $f$, perché $f$ potrebbe assumere altri valori oltre a 0, ma poco manca, infatti basta definire $g : \mathbb{N}^k \to \mathbb{N}_\perp$ come:

$$g(\mathbf{y}) := \begin{cases} 0 & \text{se } \mathbf{y} = \mathbf{0} \\ \perp & \text{altrimenti} \end{cases}$$

che è ricorsiva parziale - ed in pratica la composizione manda tutti i valori di $f$ che non siano 0 a $\perp$, per cui si ha $R = \{\mathbf{x} \in \mathbb{N}^k | g \circ f(\mathbf{x}) \neq \perp\}$, infatti $g \circ f(\mathbf{x}) \neq \perp$ se e solo se $f(\mathbf{x}) = 0$.

$\boxed{2. \Rightarrow 3.}$ Sia $R = \{\mathbf{x} \in \mathbb{N}^k | f(\mathbf{x}) \neq \perp\}$, per qualche funzione ricorsiva parziale $f : \mathbb{N}^k \to \mathbb{N}_\perp$, per il teorema della forma normale esistono $T_k$ predicato PR e $U$ funzione PR tali che:

$$f(\mathbf{x}) \neq \perp \iff \exists m\ T_k(\ulcorner f \urcorner, \mathbf{x}, m) = 0$$

e, in tal caso, $f(\mathbf{x}) = U(\mu_m(T_k(\ulcorner f \urcorner, \mathbf{x}, m))))$, non possiamo usare direttamente $U$ in quanto dipende da $\mathbf{x}$, tuttavia possiamo definire una funzione che per ogni $n \in \mathbb{N}$, che corrisponderà alla codifica della lista $\langle \mathbf{x}, m \rangle$, restituisce $\mathbf{x}$ se la minimizzazione del predicato risulta ben definita, altrimenti restituisce un elemento qualsiasi $\mathbf{c} \in R$:

$$g(n) := \begin{cases} \mathtt{car}(n) & \text{se } T_k(\ulcorner f \urcorner, \mathtt{nth}(n, 0), \mathtt{nth}(n, 1), \ldots, \mathtt{nth}(n, k)) = 0 \\ \mathbf{c} & \text{altrimenti} \end{cases}$$

che è ricorsiva totale, perché in particolare è PR, e vale che $R = \operatorname{Im} g$ in quanto $\mathbf{x} \in R$ se e solo se $\exists m\ T_k(\ulcorner f \urcorner, \mathbf{x}, m) = 0$, e in tal caso $g(\langle \mathbf{x}, m \rangle) = \mathbf{x}$.

$\boxed{3. \Rightarrow 1.}$ Sia $R = \{f(n) | n \in \mathbb{N}\}$, per qualche funzione ricorsiva totale $f : \mathbb{N} \to \mathbb{N}^k$, allora basta definire $g : \mathbb{N}^k \to \mathbb{N}_\perp$ come:

$$g(\mathbf{y}, n) := \begin{cases} 0 & \text{se } f(n) = \mathbf{y} \\ 1 & \text{altrimenti} \end{cases}$$

definiamo infine:

$$h(\mathbf{y}) := \begin{cases} 0 & \text{se } \exists n \; g(\mathbf{y}, n) = 0 \\ \perp & \text{altrimenti} \end{cases}$$

che è ricorsiva parziale, e vale che $h(\mathbf{x}) = 0$ se e solo se $\mu_n g(\mathbf{x}, n)$ è ben definita, cioè se e solo se $\exists n \; g(\mathbf{x}, n) = 0$, che equivale a $\exists n \; f(n) = \mathbf{x}$.

□

**Proposizione 6.3** (Caratterizzazione della decidibilità in termini di semidecidibilità)
$R$ è decidibile se e solo se $R$ e $R^{\complement}$ sono semidecidibili.

**Osservazione 6.4** (Il complementare di un insieme decidibile è decidibile) — Se $R$ è decidibile, allora $R^{\complement}$ è decidibile, infatti se $f : \mathbb{N}^k \to \mathbb{N}$ è ricorsiva totale tale che $R = \{f(\mathbf{x}) = 0\}$, allora $R^{\complement} = \{1 \dot{-} f(\mathbf{x}) = 0\}$, e quindi è decidibile, perché la funzione ottenuta dalla ricorsione è ancora ricorsiva totale.

*Dimostrazione.* È ovvio che se $R$ è decidibile, allora $R$ e $R^{\complement}$ sono semidecidibili. Per il viceversa, se $R$ e $R^{\complement}$ sono semidecidibili, per la proposizione precedente sono enumerabili, quindi esistono $f, g : \mathbb{N} \to \mathbb{N}^k$ ricorsive totali tali che:

$$R = \{f(n) | n \in \mathbb{N}\} \quad \text{e} \quad R^{\complement} = \{g(n) | n \in \mathbb{N}\}$$

a questo punto posso definire $h : \mathbb{N} \to \mathbb{N}^k$ come:

$$h(n) := \begin{cases} f(n \,\texttt{div}\, 2) & \text{se } n \bmod 2 = 0 \\ g(n \,\texttt{div}\, 2) & \text{se } n \bmod 2 = 1 \end{cases}$$

in pratica $h$ alterna le due enumerazioni, calcolate ciascuna su tutti i valori. La funzione ricorsiva totale che decide $R$ è quindi:

$$\mu_n(\chi_=(h(n), \mathbf{x})) \bmod 2$$

dove $\chi_=$ è il predicato PR che definisce l'uguaglianza, e quindi la funzione definita restituisce il minimo $n$, modulo 2, tale che $h(n) = \mathbf{x}$. Se tale funzione fa 0 allora $\mathbf{x} = f(n \,\texttt{div}\, 2)$ per qualche $n$, e quindi $\mathbf{x} \in R$; viceversa se $\mathbf{x} \in R$, allora $\mathbf{x} = f(n)$ per qualche $n$, da cui $h(2n) = f(n) = \mathbf{x}$, per cui la minimizzazione è ben definita e restituisce un valore pari, che dà 0 modulo 2. □

Vediamo un insieme non decidibile.

**Proposizione 6.5** (Halting problem)
Sia $H_1 := \{(\alpha, x) \in \mathbb{N}^2 | u_1(\alpha, x) \neq \perp\}$, ovvero l'insieme di codice-input per cui la macchina universale in dimensione 1 si ferma. Vale che $H_1$ non è decidibile.

**Osservazione 6.6** (Definizione per casi di funzioni ricorsive) — Osserviamo che se $R$ è decidibile, ovvero $R$ è il luogo di zero di una funzione ricorsiva totale, allora:

$$f(\mathbf{x}) := \begin{cases} g(\mathbf{x}) & \text{se } \mathbf{x} \in R \\ h(\mathbf{x}) & \text{altrimenti} \end{cases}$$

con $g$ e $h$ funzioni ricorsive parziali, è ricorsiva parziale in quanto $f(\mathbf{x}) = g(\mathbf{x})(1 - R(\mathbf{x})) + h(\mathbf{x})(1 \dot{-} (1 \dot{-} R(\mathbf{x})))$, dove con $R(\mathbf{x})$ indichiamo la funzione che restituisce 0 se $\mathbf{x} \in R$ e un numero naturale $\geq 1$ altrimenti, che è ricorsiva totale, e quindi $f$ è ricorsiva parziale. Se $g, h$ sono ricorsive totali, allora $f$ è ricorsiva totale.
Osserviamo infine che se $R$ è semidecidibile allora lo stesso discorso non funzione, perché se $R(\mathbf{x}) = \perp$ otteniamo solo che $f(\mathbf{x})$ è uguale ad un'espressione aritmetica indefinita, perché c'è un $\perp$ in mezzo, e quindi $f$ non fa $\perp$ perché è mal definita.

*Dimostrazione.* Se per assurdo $H_1$ sia decidibile, allora la funzione definita per casi da:

$$f(\alpha) := \begin{cases} u_1(\alpha, \alpha) + 1 & \text{se } (\alpha, \alpha) \in H_1 \\ 0 & \text{altrimenti} \end{cases}$$

è ricorsiva totale, infatti nel caso $(\alpha, \alpha) \in H_1$ sappiamo a priori che $u_1(\alpha, \alpha) \neq \perp$.
Applichiamo ora $f$ al suo stesso codice, si danno due casi:

- se $(\ulcorner f \urcorner, \ulcorner f \urcorner) \in H_1$, allora $f(\ulcorner f \urcorner) = u_1(\ulcorner f \urcorner, \ulcorner f \urcorner) + 1$, ma per la definizione di $u_1$ si ha che $u_1(\ulcorner f \urcorner, \ulcorner f \urcorner) = f(\ulcorner f \urcorner)$, da si otterrebbe che $f(\ulcorner f \urcorner) = f(\ulcorner f \urcorner) + 1$ $\lightning$;
- se $(\ulcorner f \urcorner, \ulcorner f \urcorner) \notin H_1$, questo equivale a $u_1(\ulcorner f \urcorner, \ulcorner f \urcorner) = \perp$, ovvero $f(\ulcorner f \urcorner) = \perp$, che è assurdo visto che $f$ è ricorsiva totale - oppure ancora più nello specifico perché dovrebbe venire proprio 0 per la sua definizione.

□

**Esercizio 6.7** (Halting problem generalizzato)**.** Dimostrare che $H_n$ non è decidibile.

*Soluzione.* Supponiamo per assurdo che $H_n := \{(\alpha, \mathbf{x}) \in \mathbb{N}^{n+1} | u_n(\alpha, \mathbf{x}) \neq \perp\}$ sia decidibile, allora si procede allo stesso identico modo, definendo:

$$f(\alpha, \ldots, \alpha) := \begin{cases} u_n(\alpha, \alpha, \ldots, \alpha) + 1 & \text{se } (\alpha, \alpha, \ldots, \alpha) \in H_n \\ 0 & \text{altrimenti} \end{cases}$$

che è ricorsiva totale, e poi si applica $f$ al suo stesso codice, ottenendo una contraddizione esattamente come prima:

- se $(\ulcorner f \urcorner, \ulcorner f \urcorner, \ldots, \ulcorner f \urcorner) \in H_n$, allora $f(\ulcorner f \urcorner, \ldots, \ulcorner f \urcorner) = u_n(\ulcorner f \urcorner, \ldots, \ulcorner f \urcorner) + 1$, ma, per definizione, si ha $u_n(\ulcorner f \urcorner, \ldots, \ulcorner f \urcorner) = f(\ulcorner f \urcorner, \ldots, \ulcorner f \urcorner)$, da cui $f(\ulcorner f \urcorner, \ldots, \ulcorner f \urcorner) = f(\ulcorner f \urcorner, \ldots, \ulcorner f \urcorner) + 1$ $\lightning$;
- se $(\ulcorner f \urcorner, \ulcorner f \urcorner, \ldots, \ulcorner f \urcorner) \notin H_n$, questo equivale a $u_n(\ulcorner f \urcorner, \ldots, \ulcorner f \urcorner) = \perp$, ovvero $f(\ulcorner f \urcorner, \ldots, \ulcorner f \urcorner) = \perp$, che è assurdo visto che $f$ è ricorsiva totale, o in particolare dovrebbe essere 0 per definizione.

□

**Esercizio 6.8** (Semidecidibilità dell'halting problem)**.** Dimostrare che $H_n$ è in particolare semidecidibile, per cui è un esempio di insieme semidecidibile non decidibile.

*Soluzione.* Per dimostrare che $H_n$ è semidecidibile dimostriamo che è il dominio di una funzione ricorsiva parziale, ed è infatti banalmente il dominio della funzione ricorsiva $u_n : \mathbb{N}^{n+1} \to \mathbb{N}_\perp$. □

**Esercizio 6.9** (Insiemi non semidecidibili)**.** Dimostrare che l'insieme:

$$TOT := \{\alpha | n \mapsto u_1(\alpha, n) \text{ è totale}\}$$

ovvero l'insieme dei codici dei programmi che terminano su ogni input, non è semidecidibile e che anche $TOT^\complement$ non è semidecidibile.[48]

**Osservazione 6.10** (Algebra degli insiemi decidibili) — Osserviamo che gli insiemi decidibili formano un'algebra, infatti: sono chiusi per unione, è sufficiente prendere il prodotto delle due funzioni che decidono i due insiemi; sono chiusi per complementare come già visto in precedenza, inoltre la funzione $0_k$ è PR, e quindi $\mathbb{N}^k$ è decidibile.
Osserviamo invece che gli insiemi semidecidibili non formano un'algebra, infatti se la chiusura per unione ed il fatto che $\mathbb{N}^k$ è semidecidibile funzionano come sopra, il passaggio al complementare non funziona in generale, infatti ad esempio $H_1$ è semidecidibile, ma $H_1^\complement$ non lo è, perché se lo fosse, allora $H_1$ sarebbe decidibile $\lightning$.
Tuttavia, nonostante gli insiemi semidecidibili non siano chiusi per complementare è comunque vero che sono chiusi per intersezione, infatti se $R = \{\mathbf{x} \in \mathbb{N}^k | f(\mathbf{x}) = 0\}$ e $S = \{\mathbf{x} \in \mathbb{N}^k | g(\mathbf{x}) = 0\}$, con $f, g : \mathbb{N}^k \to \mathbb{N}_\perp$ ricorsive parziali, allora $R \cap S = \{\mathbf{x} \in \mathbb{N}^k | h(\mathbf{x}) = 0\}$, con $h : \mathbb{N}^k \to \mathbb{N}_\perp$ definita come:

$$h(\mathbf{x}) := \begin{cases} 0 & \text{se } f(\mathbf{x}) = 0 \text{ e } g(\mathbf{x}) = 0 \\ \perp & \text{altrimenti} \end{cases}$$

che è ricorsiva parziale.

*Soluzione.* Se riuscissimo a trovare una funzione ricorsiva totale $f : \mathbb{N}^2 \to \mathbb{N}$ tale che per ogni $(\alpha, x) \in \mathbb{N}^2$ si abbia che $(\alpha, x) \in \mathbb{N}^2 \setminus H_1$ se e solo se $f(\alpha, x) \in TOT$, per cui si avrebbe $H_1^\complement = f^{-1}[TOT]$, allora $TOT$ non è semidecidibile, perché se lo fosse, allora anche $f^{-1}[TOT]$ sarebbe semidecidibile - infatti $TOT$ è semidecidibile se e solo se è dominio di $\varphi$ ricorsiva parziale, e quindi $f^{-1}[TOT]$ è dominio di $\varphi \circ f$, che è ricorsiva parziale, e quindi semidecidibile. Ora definiamo $f : \mathbb{N}^2 \to \mathbb{N}$ come:

$$f(\alpha, x) := \ulcorner g_{\alpha,x} \urcorner, \qquad g_{\alpha,x}(n) := \begin{cases} 0 & \text{se } \forall m \leq n \; T_1(\alpha, x, m) \neq 0 \\ \perp & \text{altrimenti} \end{cases}$$

che è ricorsiva totale, e vediamo che soddisfa la condizione richiesta. Se $(\alpha, x) \in H_1^\complement$, allora $u_1(\alpha, x) = \perp$, per cui $T_1(\alpha, x, m) \neq 0$ per ogni $m \in \mathbb{N}$, perché non esiste minimo, per cui $g_{\alpha,x} \equiv 0$, e quindi il suo codice sta in $TOT$.
Viceversa, se $(\alpha, x) \in f^{-1}[TOT]$, allora $g_{\alpha,x}$ è totale, per cui per ogni $m \in \mathbb{N}$ si ha $T_1(\alpha, x, m) \neq 0$, quindi la minimizzazione non è ben definita, quindi $u_1(\alpha, x) = \perp$, e quindi $(\alpha, x) \in H_1^\complement$. □

**Esercizio 6.11** (Separazione di insiemi semidecidibili)**.** Esistono $A, B$ semidecidibili, con $A \cap B = \varnothing$, tali che non esiste $C$ decidibile con $A \subseteq C$ e $B \subseteq C^\complement$. Ovvero non esistono due insiemi semidecidibili disgiunti che possono essere separati da un insieme decidibile.

*Soluzione.* TODO □

[48] **Hint**: Esibisco una $f$ ricorsiva totale tale che, $(\alpha, x) \in \mathbb{N}^2 \setminus H_1 \iff f(\alpha, x) \in TOT$.

## §6.2 Insiemi aritmetici

**Definizione 6.12** (Insiemi aritmetici)**.** Diciamo che $R \subseteq \mathbb{N}^k$ è **aritmetico** se è definibile in $(\mathbb{N}; +, \cdot, 0, s)$, ovvero:

$$R = \{(x_1, \ldots, x_k) \in \mathbb{N}^k : \mathbb{N} \models \varphi(x_1, \ldots, x_k)\}$$

i.e. aritmetico $= L_{\text{arit}}$-definibile.

**Definizione 6.13** (Classi di complessità aritmetica)**.** Diciamo che una formula aritmetica $\varphi$ è **di classe** (o ha **complessità aritmetica**):

- $\Delta_0^0 = \Sigma_0^0 = \Pi_0^0$ se si ottiene dagli $L_{\text{arit}}$-termini per chiusura sotto: $\neg$, $\wedge$, $\vee$, $\forall x \leq y$, $\exists x \leq y$, dove le ultime due sono abbreviazioni per le formule $\forall x\,(x \leq y \to \ldots)$ e $\exists x\,(x \leq y \wedge \ldots)$, con $x < y \overset{\text{def}}{=} \exists t\,(x + t = y)$, cioè possiamo applicare solo **quantificatori limitati** - i.e. quantificatori che quantificano su un intervallo finito, pertanto non possono essere usati per esprimere proprietà di insiemi infiniti;
- $\Sigma_{n+1}^0$ se $\varphi = \exists x_1 \ldots \exists x_k \psi$ con $\psi$ di classe $\Pi_n^0$;
- $\Pi_{n+1}^0$ se $\varphi = \forall x_1 \ldots \forall x_k \psi$ con $\psi$ di classe $\Sigma_n^0$.

Ovvero il pedice conta il numero di alternanze di blocchi di quantificatori dello stesso tipo, la lettera greca indica se il blocco di quantificatori più esterno è esistenziale o universale.

$$\begin{tikzcd}
 & \Delta_0^0 = \Sigma_0^0 = \Pi_0^0 \arrow[dl, dash] \arrow[dr, dash] & \\
\Sigma_1^0 = \exists\Pi_0^0 \arrow[d, dash] \arrow[drr, dash] & & \Pi_1^0 = \forall\Sigma_0^0 \arrow[d, dash] \arrow[dll, dash] \\
\Sigma_2^0 = \exists\Pi_1^0 \arrow[d, dash] & & \Pi_2^0 = \forall\Sigma_1^0 \arrow[d, dash] \\
\vdots & & \vdots
\end{tikzcd}$$

**Esempio 6.14** ($p$ è primo)

La formula $\mathsf{prime}(p) := \forall n \leq p\ (n = 1 \vee n = p \vee \neg\exists m \leq p\ m \cdot n = p)$, che esprime che $p$ è un numero primo, è di classe $\Delta_0^0$, perché è una formula ottenuta solo con quantificatori limitati, e quindi è senza quantificatori veri.

**Definizione 6.15** (Classe di un sottoinsieme di $\mathbb{N}^k$)**.** $R \subseteq \mathbb{N}^k$ è di classe $\Box$ se esiste una $L_{\text{arit}}$-formula $\varphi$ di classe $\Box$ tale che:

$$R = \{(x_1, \ldots, x_k) \in \mathbb{N}^k : \mathbb{N} \models \varphi(x_1, \ldots, x_k)\}$$

ovvero se è definibile da una formula di classe $\Box$.

**Esempio 6.16** (Classe di un predicato polinomiale)
Sia $p(x, y)$ polinomio fissato. Il predicato binario

$$R = \{(x, y) \in \mathbb{N}^2 : p(x, y) = 0\}$$

è di classe $\Delta_0^0$, perché la formula $p(x, y) = 0$ è senza quantificatori. Invece il predicato unario

$$S = \{x \in \mathbb{N} : \exists y\, p(x, y) = 0\}$$

è di classe $\Sigma_1^0$.

**Osservazione 6.17** — Ogni insieme aritmetico appartiene alla gerarchia aritmetica, è sufficiente portare la $L_{\text{arit}}$-formula che definisce l'insieme in **forma normale premessa**, ovvero in una forma in cui tutti i quantificatori esterni sono dello stesso tipo.

**Definizione 6.18** (Classe $\Delta_n^0$)**.** $R \subseteq \mathbb{N}^k$ è di classe $\Delta_n^0$ se $R$ è sia un insieme di classe $\Sigma_n^0$ che di classe $\Pi_n^0$, i.e. è definibile sia da una formula di classe $\Sigma_n^0$ che da una formula di classe $\Pi_n^0$.

**Osservazione 6.19** (Chiusura delle classi aritmetiche) — Valgono le seguenti.

- $\Sigma_n^0$ è chiusa sotto: $\wedge$, $\vee$, $\exists$, $\forall y \leq x$,
- $\Pi_n^0$ è chiusa sotto: $\wedge$, $\vee$, $\forall$, $\exists y \leq x$,
- $X \subseteq \mathbb{N}^k$ è di classe $\Sigma_n^0$ se e solo se $X^{\complement}$ è di classe $\Pi_n^0$.

*Dimostrazione.* La terza è immediata dalla definizione del complementare. Inoltre, poiché la negazione scambia $\Sigma_n^0$ e $\Pi_n^0$, la prima affermazione è duale della seconda: basta dimostrare la prima. Siano dunque $\varphi, \psi$ formule di classe $\Sigma_n^0$.

$\boxed{\wedge, \vee}$ Portando entrambe in forma premessa, posso accodare i blocchi esistenziali esterni e lasciare dentro una congiunzione/disgiunzione di formule di classe $\Pi_{n-1}^0$:

$$\begin{aligned}\varphi &= \exists x_1 \dots \exists x_k\, \psi_1 * \exists y_1 \dots \exists y_m \psi_2 \\ &= \exists x_1 \dots \exists x_k \exists y_1 \dots \exists y_m\, (\psi_1 * \psi_2)\end{aligned}$$

con $* \in \{\wedge, \vee\}$, e $\psi_1 * \psi_2$ è di classe $\Pi_{n-1}^0$ per ipotesi induttiva, quindi $\varphi$ è di classe $\Sigma_n^0$. Notare infine che abbiamo supposto WLOG che $y_i \notin \text{vl}(\psi_1)$ e $x_i \notin \text{vl}(\psi_2)$, e possiamo farlo perché le variabili sono legate, e.g. le $y_i$ sono legate nel secondo blocco, ma possono essere libere nel primo, nel cui caso le rinominiamo con $x_j$ per qualche $j$ che non compare già, e viceversa; tale procedimento non cambi nulla in quanto $\exists z\, \psi \leftrightarrow \exists z'\, \psi[z'/z]$ se $z'$ è una variabile che non compare in $\psi$, e ugualmente per i quantificatori universali.

$\boxed{\exists}$ È banale.

$\boxed{\forall y \leq x}$ Scriviamo $\varphi(x, y)$ in forma premessa $\exists z_1 \dots \exists z_r\, \theta(x, y, \mathbf{z})$ con $\theta \in \Pi_{n-1}^0$. Allora:

$$\forall y \leq x\, \varphi(x, y) \leftrightarrow \exists b\, \forall y \leq x\, \exists z_1 \leq b \dots \exists z_r \leq b\, \theta(x, y, \mathbf{z})$$

e l'equivalenza è facile dimostrare, ad esempio semanticamente, in particolare per $\rightarrow$ c'è bisogno che gli $z_i$ siano in numero finito.

□

Vediamo che la chiusura delle $L_{\text{arit}}$-formule corrisponde alla chiusura per delle operazioni di insiemi aritmetici.

**Osservazione 6.20** (Corrispettivo insiemistico della chiusura delle classi aritmetiche) — Se $R, S \subseteq \mathbb{N}^k$ sono di classe $\Sigma_n^0$, allora:

- $R \cap S$ è di classe $\Sigma_n^0$,
- $R \cup S$ è di classe $\Sigma_n^0$,
- se $R \subseteq \mathbb{N}^2$ si ha che $\{x \in \mathbb{N} : \exists y \leq x \ (x, y) \in R\}$ e $\{x \in \mathbb{N} : \forall y \leq x \ (x, y) \in R\}$ sono di classe $\Sigma_n^0$.

In tutti i casi la dimostrazione segue semplicemente dal fatto che lo sono le $L_{\text{arit}}$-formule che definiscono $R$ e $S$, e che le classi aritmetiche sono chiuse per le operazioni corrispondenti. Analogamente nel caso di $\Pi_n^0$.

Quindi il corollario seguente è una proprietà di chiusura per operazioni su sottoinsiemi $\Delta_n^0$ di $\mathbb{N}^k$, non avendo il corrispettivo di $L_{\text{arit}}$-formule in questo caso.

**Corollario 6.21** (Chiusura di $\Delta_n^0$)
$\Delta_n^0$ è stabile per: $\wedge$, $\vee$, $\neg$, $\forall y \leq x$, $\exists y \leq x$.

*Dimostrazione.* Non c'è bisogno di fare verifiche esplicite, visto che abbiamo già l'osservazione di chiusura sia per $\Sigma_n^0$ che per $\Pi_n^0$, quindi ad esempio se $R, S \in \Delta_n^0$, allora $R \cap S$ è sia di classe $\Sigma_n^0$ che di classe $\Pi_n^0$, e quindi è di classe $\Delta_n^0$, e così via per le altre operazioni. □

**Teorema 6.22** (Semidecidibilità in termini della gerarchia aritmetica)
$S \subseteq \mathbb{N}^k$ è semidecidibile se e solo se $S$ è di classe $\Sigma_1^0$.

**Corollario 6.23** (Decidibilità in termini della gerarchia aritmetica)
$S \subseteq \mathbb{N}^k$ è decidibile se e solo se $S$ è di classe $\Delta_1^0$.

Vediamo subito la dimostrazione del corollario.

*Dimostrazione.* Sappiamo che $S \subseteq \mathbb{N}^k$ è decidibile se e solo se $S$ e $S^{\complement}$ sono semidecidibili, che equivale per il teorema a dire che $S$ è di classe $\Sigma_1^0$ e $S^{\complement}$ è di classe $\Sigma_1^0$. Ora $S^{\complement}$ è di classe $\Sigma_1^0$ se e solo se $S$ è di classe $\Pi_1^0$, per cui $S$ è sia di classe $\Sigma_1^0$ che di classe $\Pi_1^0$, e quindi è di classe $\Delta_1^0$, e viceversa.
Abbiamo usato che se un insieme è di classe $\Sigma_n^0$ allora il suo complementare è di classe $\Pi_n^0$, ciò deriva dal fatto che la negazione inverte i quantificatori, e dalla definizione di classe di un sottoinsieme di $\mathbb{N}^k$. □

Il nostro obiettivo è ora dimostrare il teorema ed ottenere la seguente catena di inclusioni:

$$\Delta_0^0 \subseteq \{\text{predicati PR}\} \subseteq \{\text{decidibili}\} \subseteq \{\text{semidecidibili}\} = \Sigma_1^0$$

Si osserva che mettendo un blocco di $\exists$ davanti ad una $L_{\text{arit}}$-formula che definisce uno degli insiemi qualsiasi appartenenti ad un elemento della catena si ottiene un insieme di classe $\Sigma_1^0$, e quindi gli insiemi di classe $\Sigma_1^0$ possono essere pensati come "blocco di esistenziali" seguito da una formula aritmetica qualsiasi in uno degli insiemi della catena. Alla fine del corso vedremo anche che $\{\text{insiemi polinomiali}\} \subsetneq \Delta_0^0$, in quanto un insieme polinomiale è definito da $\{p(x_1, \dots, x_k) = 0\}$, che è una formula senza quantificatori, e quindi di classe $\Delta_0^0$, ma soprattutto se mettessimo un blocco di $\exists$ davanti ad una formula polinomiale otterremmo i cosiddetti **insiemi diofantei**, ovvero della forma:

$$\{(x_1, \dots, x_k) \in \mathbb{N}^k : \exists y_1 \dots \exists y_m \; p(x_1, \dots, x_k, y_1, \dots, y_m) = 0\}$$

cioè l'insieme dei parametri per cui esistono dei valori che soddisfano un'equazione polinomiale, e questi insiemi sono di classe $\Sigma_1^0$. In particolare sempre per la catena sopra abbiamo scoperto allora che ogni insieme semidecidibile è diofanteo, per cui esiste una corrispondenza 1:1 tra insiemi di parametri che risolvono un'equazione diofantea ed insiemi di parametri per cui esiste un programma che può terminare o meno. Quello che scopriremo è che appunto esisterà una diofantea, il cui insieme di parametri che la risolvono corrispondente all'halting set, e quindi non necessariamente decidibile, ovvero non esisterà un algoritmo che, dati i coefficienti di un polinomio, riesca a decidere se esiste una soluzione o meno e questo ci permetterà di rispondere - negativamente - al decimo problema di Hilbert.

Introduciamo ora i lemmi necessari per dimostrare il teorema.

**Lemma 6.24** (Corrispondenza tra funzioni ricorsive parziali e formule di classe $\Sigma_1^0$)
Sia $f : \mathbb{N}^k \to \mathbb{N}_\perp$ ricorsiva parziale, allora esiste $\varphi(x_1, \dots, x_k, y)$ di classe $\Sigma_1^0$ tale che:

$$f(x_1, \dots, x_k) = y \iff \mathbb{N} \models \varphi(x_1, \dots, x_k, y)^a$$

[a] Si intende sempre con la convenzione che le variabili tra parantesi indicano "per ogni valutazione $v$ tale che la valutazione di $x_i$ corrisponda al valore di $x_i$ della formula a sinistra".

Da questo lemma l'implicazione $\implies$ del teorema è immediata come segue.

*Dimostrazione.* Se $S$ è semidecidibile, allora $S = \{\mathbf{x} \in \mathbb{N}^k : f(\mathbf{x}) = 0\}$ per qualche $f : \mathbb{N}^k \to \mathbb{N}_\perp$ ricorsiva parziale, per il lemma esiste $\varphi$ $L_{\text{arit}}$-formula di classe $\Sigma_1^0$ tale che $f(\mathbf{x}) = 0$ se e solo se $\mathbb{N} \models \varphi(\mathbf{x}, 0)$, quindi $S = \{\mathbf{x} \in \mathbb{N}^k : \mathbb{N} \models \varphi(\mathbf{x}, 0)\}$, per cui $S$ è di classe $\Sigma_1^0$. $\square$

Dimostriamo ora il lemma.

*Dimostrazione.* Induzione sulla costruzione di $f$ come funzione ricorsiva.

casi base — Se $f$ è $\mathbf{0}_k$, allora $f(\mathbf{x}) = y$ se e solo se $y = 0$, quindi è sufficiente prendere $\varphi(\mathbf{x}, y) := (y = 0)$; se $f$ è $s$, allora $f(x) = y$ se e solo se $y = s(x)$, quindi è sufficiente prendere $\varphi(x, y) := (y = s(x))$; se $f$ è $\pi_{k,i}$, allora $f(\mathbf{x}) = y$ se e solo se $y = x_i$, quindi è sufficiente prendere $\varphi(\mathbf{x}, y) := (y = x_i)$.

composizione — Se $h = f \circ (g_1, \dots, g_k)$, con $f : \mathbb{N}^k \to \mathbb{N}_\perp$ e $g_i : \mathbb{N}^h \to \mathbb{N}_\perp$ ricorsive parziali, per ipotesi induttiva esistono formule $\varphi_f(\mathbf{z}, y)$ e $\varphi_{g_i}(\mathbf{x}, z_i)$ di classe $\Sigma_1^0$ che rappresentano i rispettivi grafi. Allora

$$h(\mathbf{x}) = y \iff \exists z_1 \dots \exists z_k \Big( \bigwedge_{i=1}^{k} \varphi_{g_i}(\mathbf{x}, z_i) \wedge \varphi_f(z_1, \dots, z_k, y) \Big),$$

dove quantifico solo sugli $z_i$ perché sono variabili ausiliarie che testimoniano il fatto che $g_i(\mathbf{x}) = z_i$ per ogni $i$, mentre $x_1, \dots, x_h$ e $y$ sono variabili libere da ambo i lati dell'equivalenza. Osserviamo infine che avendo usato solo $\wedge$ ed un blocco di $\exists$ all'inizio la formula è ancora di classe $\Sigma_1^0$.

minimizzazione

Sia $f : \mathbb{N}^k \to \mathbb{N}_\perp$ definita per minimizzazione da $g : \mathbb{N}^{k+1} \to \mathbb{N}_\perp$ come $f(\mathbf{x}) := \mu_n g(\mathbf{x}, n)$, con $g$ ricorsiva parziale. Per ipotesi induttiva esiste $\varphi_g(\mathbf{x}, n, z)$ di classe $\Sigma_1^0$ che rappresenta il grafico di $g$, da questa, usando la definizione di minimizzazione abbiamo:

$$f(\mathbf{x}) = y \iff \varphi_g(\mathbf{x}, y, 0) \wedge \forall i < y \, \exists v \, (\varphi_g(\mathbf{x}, i, v) \wedge v \neq 0),$$

e questa formula è ancora di classe $\Sigma_1^0$ perché abbiamo aggiunto solo un $\wedge$, una quantificazione limitata $\forall i < y$, ed un esistenziale, che serve per coprire il caso in cui $g(\mathbf{x}, i)$ non è definita, e quindi non è zero - richiedere solo $\neg\varphi_g(\mathbf{x}, i, 0)$ non coprirebbe questo caso.

ricorsione primitiva

Sia $f : \mathbb{N}^{k+1} \to \mathbb{N}_\perp$ definita per ricorsione da:

$$f(\mathbf{x}, 0) = g(\mathbf{x}), \qquad f(\mathbf{x}, y+1) = g(\mathbf{x}, y, f(\mathbf{x}, y)),$$

con $h, g$ ricorsive parziali, per ipotesi induttiva esistono $\varphi_g(\mathbf{x}, y)$ e $\varphi_h(\mathbf{a}, b, c, t)$ di classe $\Sigma_1^0$, che corrispondono ai grafici di $g$ e $h$. Quello che vorremmo fare è scrivere la formula:

$$f(\mathbf{x}, y) = z \iff \exists z_0, \dots z_y \, (\varphi_g(\mathbf{x}, 0, z_0)) \wedge (\forall i < y \, \varphi_h(\mathbf{x}, i, z_i)) \wedge (z_y = z)$$

ma questa formula è sbagliata perché non posso quantificare su un numero variabile di variabili $z_i$, essendo $y$ una variabile libera.
Abbiamo quindi necessità di una formula che, fissata a priori una quantità finita di parametri fissata a priori, sia in grado di rappresentare tutte le sequenze finite di lunghezza al più $y$, e tale formula la vogliamo di complessità aritmetica controllabile. Per tale ragione introduciamo la **codifica** $\beta$ di Gödel delle sequenze finite, che ci permetterà di rimpiazzare $\exists z_0 \dots \exists z_y$ con $\exists a, b$ due parametri e $z_k$ con $\beta(a, b, k)$, e quindi da sopra otteniamo la formula desiderata di classe $\Sigma_1^0$.

**Lemma 6.25** (Codifica $\beta$ di Gödel delle sequenze finite)
Sia $\beta(a, b, k) := a \bmod (b \cdot (k+1) + 1)$. Dati $y, z_0, \dots, z_y$, esistono $a, b$ tali che:

$$\forall k \leq y \; \beta(a, b, k) = z_k$$

ovvero la $\beta$ codifica la sequenza $z_0, \dots, z_y$ usando i parametri $a, b$.

*Dimostrazione.* Dati $y, z_0, \dots, z_y$ cerchiamo $a, b$ tali che $\beta(a, b, k) = z_k$ per ogni $k \leq y$, ovvero cerchiamo $a, b$ tali che il seguente sistema di $y+1$ equazioni abbia soluzione:

$$\begin{cases} a \bmod (b+1) &= z_0 \\ a \bmod (2b+1) &= z_1 \\ & \vdots \\ a \bmod (b(y+1)+1) &= z_y \end{cases}$$

che può essere riscritto come sistema di congruenze:

$$\begin{cases} a & \equiv & z_0 & \mod (b+1) \\ a & \equiv & z_1 & \mod (2b+1) \\ & \vdots & & \\ a & \equiv & z_y & \mod (b(y+1)+1) \end{cases}$$

se scegliamo $b$ in modo che $\gcd(b(i+1)+1, b(j+1)+1) = 1$ per ogni $i \neq j$, vale il teorema cinese del resto esiste $a$ che soddisfa tutte le congruenze, tuttavia non vogliamo solo che $a$ sia congruente a $z_k$ modulo $b(k+1)+1$, ma vogliamo proprio che $a \bmod (b(k+1)+1) = z_k$, ovvero che $z_k < b(k+1)+1$ per ogni $k \leq y$, quindi la scelta di $b$ deve essere tale che $b(k+1)+1 > z_k$, quindi ad esempio è sufficiente scegliere $b > \max\{z_0, \dots, z_y\}$.
Ora una scelta possibile è $b = (y+1)! \cdot (\max\{z_0, \dots, z_y\} + 1)$, che è ovviamente tale che $b > \max\{z_0, \dots, z_y\}$, e che soddisfa la condizione di coprimalità in quanto, supponendo WLOG $i < j$:

$$\begin{aligned} \gcd(b(i+1)+1, b(j+1)+1) &= \gcd(b(i+1)+1, b(j-i)) \\ &= \gcd(b(i+1)+1, b) \gcd(b(i+1)+1, j-i) \\ &= \gcd(b(i+1)+1, j-i) = 1 \end{aligned}$$

dove l'ultima uguaglianza deriva dal fatto che $i, j \leq y \implies j - i < y \implies j - i \mid b$, e quindi $b(i+1)+1 = k(j-i)(i+1)+1$ per qualche $k \in \mathbb{N}$, da cui la coprimalità. □

**Osservazione 6.26** (Complessità di $\beta(a,b,k)$) — La relazione $\beta(a,b,k) = z$ è equivalente a:

$$z < b(k+1)+1 \wedge \exists t(z + t(b(k+1) = a))$$

che è di classe $\Sigma_1^0$, ma in realtà $t$ è limitato da $a$, quindi potremmo rimpiazzare $\exists t$ con $\exists t \leq a$, e quindi la relazione è addirittura di classe $\Delta_0^0$.

Quindi possiamo tornare alla formula che rappresenta $f(\mathbf{x}, y) = z$ e riscriverla come:

$$(\beta(a,b,0) = g(\mathbf{x})) \wedge (\forall i < y\, \beta(a,b,i+1) = h(\mathbf{x}, i, \beta(a,b,i))) \wedge (\beta(a,b,y) = z)$$

tale formula è di classe $\Sigma_1^0$, ma va scritta per esteso per verificarlo davvero, anche perché altrimenti sembra che stiamo usando $\beta$ come simbolo di funzione, cosa che non è. È facile riscrivere la prima e la terza congiunzione, quella in mezzo:

$$\beta(a,b,i+1) = h(\mathbf{x}, i, \beta(a,b,i))$$

è equivalente a:

$$\exists m \exists n\, (\beta(a,b,i+1) = n) \wedge \varphi_h(\mathbf{x}, i, m) \wedge (\beta(\mathbf{x}, i, m) = n) \wedge (\beta(a,b,i) = m)$$

che è di classe $\Sigma_1^0$.

□

Abbiamo dimostrato che semidecidibile implica $\Sigma_1^0$, vediamo ora l'implicazione inversa. Ci serve un lemma che corrisponde ad uno dei contenimenti indicati all'inizio.

**Lemma 6.27** ($\Delta_0^0 \subseteq \{\text{predicati PR}\}$)
Vale che:
$$\Delta_0^0 \subseteq \{\text{predicati PR}\}$$

*Dimostrazione.* Procediamo per induzione strutturale sulla costruzione di una formula di classe $\Delta_0^0$.

formule atomiche — Se $\varphi$ è una $L_{\text{arit}}$-formula atomica, allora o è $\top, \bot$, nel cui caso gli insiemi definiti sono $\mathbb{N}^k$ e $\varnothing$, che sono entrambi predicati PR, oppure è della forma $t_1 = t_2$ con $t_1, t_2$ termini, che nel nostro caso possono essere solo polinomi, per cui formule del tipo:
$$p(x_1, \dots, x_k) = q(x_1, \dots, x_k)$$
e l'insieme definito è quindi $\{p(x_1, \dots, x_k) = q(x_1, \dots, x_k)\}$, che è un predicato PR per quanto visto.

$\wedge, \vee$ — Se $\varphi = \psi_1 * \psi_2$ con $\{\mathbb{N} \models \psi_1\}$ e $\{\mathbb{N} \models \psi_2\}$, predicati PR, allora $\{\mathbb{N} \models \varphi\} = \{\mathbb{N} \models \psi_1\} *' \{\mathbb{N} \models \psi_2\}$, con $*' \in \{\cap, \cup\}$, è un predicato PR poiché abbiamo visto che i predicati PR formano un'algebra.

$\neg$ — Se $\varphi = \neg\psi$ con $\{\mathbb{N} \models \psi\}$ un predicato PR, allora $\{\mathbb{N} \models \varphi\} = \{\mathbb{N} \models \psi\}^{\complement}$ è un predicato PR, poiché i predicati PR sono chiusi per complementare, essendo un'algebra.

$\forall y \leq x$ — Se $\varphi = \forall y \leq x\ \psi(y, \mathbf{z})$, con $\{\mathbb{N} \models \psi(y, \mathbf{z})\} =: R$ un predicato PR, allora esiste $f : \mathbb{N}^{k+1} \to \mathbb{N}$ PR tale che $R = \{f(\mathbf{z}, y) = 0\}$, e quindi $\{\mathbb{N} \models \varphi\} = \{\mathbf{z} \in \mathbb{N}^k : \forall y \leq x\ f(\mathbf{z}, y) = 0\}$, si può ottenere come $\{\#\{y \leq x : f(\mathbf{z}, y) \neq 0\} = 0\}$ - cioè quando tutti gli elementi sotto $x$ effettivamente azzerano la funzione $f$, dove la funzione $\#$ che conta il numero di $y \leq x$ tali che $f(\mathbf{z}, y) \neq 0$, ovvero è la funzione che conta il numero di elementi che appartengono ad un altro predicato sotto un certo valore, che abbiamo visto essere PR.

$\exists y \leq x$ — Se $\varphi = \exists y \leq x\ \psi(y, \mathbf{z})$, con $\{\mathbb{N} \models \psi(y, \mathbf{z})\} =: R$ un predicato PR, allora, possiamo usare direttamente il caso precedente e scrivere:
$$\begin{aligned}\{\mathbb{N} \models \varphi\} &= \{\mathbf{z} \in \mathbb{N}^k : \neg(\forall y \leq x\ \neg R(\mathbf{z}, y))\} \\ &= \{\mathbf{z} \in \mathbb{N}^k : \forall y \leq x\ \neg R(\mathbf{z}, y)\}^{\complement}\end{aligned}$$
e quindi è un predicato PR, essendo i predicati PR chiusi per complementare, e quindi anche $\neg R$ è un predicato PR, da questo uso il caso precedente e poi di nuovo il complementare.

□

E quindi possiamo dimostrare la freccia $\Longleftarrow$ del teorema.

*Dimostrazione.* Sia $S \in \Sigma_1^0$, allora $S = \{\mathbf{x} \in \mathbb{N}^k : \mathbb{N} \models \exists y_1 \dots \exists x_h \varphi(\mathbf{x}, y_1, \dots, y_h)\}$, con $\varphi$ di classe $\Delta_0^0$. Per il lemma precedente allora esiste $T$ predicato PR tale che $\varphi(\mathbf{x}, y_1, \dots, y_h) \iff (\mathbf{x}, y_1, \dots, y_h) \in T$, e quindi:
$$S = \{\mathbf{x} \in \mathbb{N}^k : \exists y_1 \dots \exists y_h\ (\mathbf{x}, y_1, \dots, y_h) \in T\}$$

A questo punto $\mathbf{x} \in S$ se e solo se:

$$0_1 \circ \mu_{\mathbf{y}}((\mathbf{x}, y_1, \ldots, y_h) \in T) = 0$$

cioè se il minimo $\mathbf{y}$ tale per cui sto nell'insieme converge, quindi compongo con 0, e se la minimizzazione converge non viene $\perp$, per cui esce 0. In pratica ho definito $S$ come il luogo di zero di quest'ultima funzione (dove l'appartenenza a $T$ è in realtà l'azzeramento della funzione PR che definisce $T$), che è ricorsiva parziale, e la condizione è che effettivamente esista $\mathbf{y}$ tale che $(\mathbf{x}, \mathbf{y}) \in T$. Istanziare che ci sono degli $\mathbf{y}$ che funzionano equivale a dire che la ricerca del minimo che funziona termina. □

Vediamo infine un interessante corollario sulla non semidecidibilità dell'insieme delle verità aritmetiche. Supponiamo che l'insieme delle $L_{\text{arit}}$-formule chiuse sia enumerabile - vedremo più avanti una numerazione di Gödel che ci permetterà di fare ciò, $\varphi \mapsto \ulcorner\varphi\urcorner$, per ora supponiamo che tale enumerazione esista - e che sia compatibile con l'operazione di sostituire un numero al posto di una variabile -, quindi l'insieme $\{\ulcorner\varphi\urcorner : \mathbb{N} \models \varphi\}$ non è semidecidibile.

**Corollario 6.28** (Teorema di indecidibilità di Church)
L'insieme $\{\ulcorner\varphi\urcorner : \mathbb{N} \models \varphi, \varphi \text{ chiusa}\}$ non è semidecidibile.

*Dimostrazione.* Poniamo $V := \{\ulcorner\varphi\urcorner : \mathbb{N} \models \varphi, \varphi \text{ chiusa}\}$ e vediamo che non è decidibile. Poiché $H_1$ è semidecidibile, per il teorema precedente esiste una formula $\psi(x, y)$ di classe $\Sigma_1^0$ tale che:

$$(x, y) \in H_1 \iff \mathbb{N} \models \psi(x, y)$$

definiamo la funzione che a $n, m$ associa il codice della formula $\psi$ con $n$ e $m$ al posto di $x$ e $y$ (per ipotesi sulla nostra numerazione questo darà il codice di una formula chiusa dove la sostituzione è stata effettuata):

$$f(n, m) := \ulcorner\psi(n, m)\urcorner$$

a questo punto $(n, m) \in H_1$ se e solo se $\mathbb{N} \models \psi(n, m)$, se e solo se $f(n, m) \in V$, ovvero se il codice di $\psi(n, m)$ è una formula vera in $\mathbb{N}$, e quindi:

$$H_1 = f^{-1}[V]$$

Il fatto di aver assunto che la codifica sia compatibile con la sostituzione ci dirà poi che $f$ è ricorsiva totale, e quindi $H_1$ sarebbe decidibile, in quanto la preimmagine di un insieme decidibile tramite una funzione ricorsiva totale è decidibile[49], assurdo.
Mostriamo ora che $V$ non è semidecidibile. Dalla semidecidibilità di $H_1$ e dalla non decidibilità di $H_1$ sappiamo che $H_1^{\complement}$ non è semidecidibile, consideriamo la funzione:

$$g(n, m) := \ulcorner\neg\psi(n, m)\urcorner$$

Allora, analogamente a prima:

$$(n, m) \in H_1^{\complement} \iff \mathbb{N} \models \neg\psi(n, m)$$

da cui $H_1^{\complement} = g^{-1}[V]$, e quindi $H_1^{\complement}$ sarebbe semidecidibile, assurdo. □

Morale l'insieme delle verità aritmetiche non è semidecidibile, e quindi non esiste un'assiomatizzazione ricorsiva dell'aritmetica.

[49] Abbiamo già visto ciò nel caso semidecidibile, per dimostrare questo lemmetto nel caso decidibile è sufficiente partire da quello ed usare la caratterizzazione degli insiemi decidibili.

## §6.3 Teorema di Rosser

Riepiloghiamo gli assiomi della teoria Q di Robinson nel linguaggio $L_{\text{arit}} = \{0, +, \cdot, s\}$.

Q1 $\forall x\ \neg s(x) = 0$ (0 non successore);

Q2 $\forall x\ \forall y\ s(x) = s(y) \to x = y$ (iniettività del successore);

Q3 $\forall x\ x + 0 = x$ (def. ricorsiva somma);

Q4 $\forall x\ \forall y\ x + s(y) = s(x + y)$ (def. ricorsiva somma);

Q5 $\forall x\ x \cdot 0 = 0$ (def. ricorsiva prodotto);

Q6 $\forall x\ \forall y\ x \cdot s(y) = (x \cdot y) + x$ (def. ricorsiva prodotto).

Q7 $\forall x\ x = 0 \vee \exists y\ s(y) = x$ (ogni numero eccetto 0 è successore).

Il nostro obiettivo è dimostrare il teorema di Rosser, che sarà una versione più forte del teorema di Gödel, e soprattutto è un risultato che estende il teorema di incompletezza di Church che abbiamo già visto. Diamo per buona l'idea intuitiva di numerazione di Gödel come una codifica di formule, dimostrazioni, eccetera in numeri naturali, e che sia compatibile con le operazioni semantiche e diamo la seguente definizione.

**Definizione 6.29** (Teoria ricorsivamente assiomatizzabile)**.** Sia $T$ una $L$-teoria, diciamo che $T$ è **ricorsivamente assiomatizzabile** se esiste una numerazione di Gödel $\varphi \mapsto \ulcorner\varphi\urcorner$ tale che l'insieme $\{\ulcorner\varphi\urcorner : \varphi \in T\}$ sia ricorsivamente enumerabile.

**Teorema 6.30** (Rosser)

Sia $T$ una $L$-teoria tale che:

(i) $T \vdash \mathsf{Q}$, ovvero $T$ estende Q,

(ii) $T$ è ricorsivamente assiomatizzabile,

(iii) $T$ è coerente,

allora $T$ è incompleta.[a]

[a] Si può rimpiazzare l'ipotesi di $T \vdash \mathsf{Q}$ con $T$ **interpreta** Q, ovvero, informalmente, $T$ è in grado di definire alcuni oggetti, che corrisponderanno ai numeri naturali, e simboli di funzione che corrisponderanno a $0, +, \cdot, s$, e di dimostrare le stesse identità aritmetiche che Q dimostra su questi oggetti, pur non avendo lo stesso linguaggio; un esempio può essere ZFC che interpreta Q, in quanto è in grado di definire $\omega$ e i simboli di funzione necessari.

L'idea del perché questa sia la formulazione giusta, con $Q$ e non PA, è che Q ha un numero finito di assiomi, al contrario di PA.
Per arrivare alla dimostrazione del teorema di Rosser, vediamo prima come Q si comporta rispetto alle funzioni calcolabili, in particolare vogliamo dimostrare la seguente proposizione.

**Proposizione 6.31** (Equivalenza tra validità in $\mathbb{N}$ e dimostrabilità in $\mathsf{Q}$)
Sia $\varphi$ una formula aritmetica $\boxed{\text{chiusa}}$, valgono i seguenti fatti.

- Se $\varphi$ è di classe $\Delta_0^0$, allora:
  1. $\mathbb{N} \models \varphi \iff \mathsf{Q} \vdash \varphi$,
  2. $\mathbb{N} \models \neg\varphi \iff \mathsf{Q} \vdash \neg\varphi$.
- Se $\varphi$ è di classe $\Sigma_1^0$, allora:
  (3) $\mathbb{N} \models \varphi \iff \mathsf{Q} \vdash \varphi$.

**Osservazione 6.32** — Vale ovviamente che (3) $\implies$ (1), e (1) $\iff$ (2), passando semplicemente alla negazione $\neg\varphi \in \Delta_0^0$.
Inoltre (1), (2) implicano che le formule chiuse in $\Delta_0^0$ sono decidibili in $\mathsf{Q}$, in quanto a sinistra la decidibilità è garantita dal modello $\mathbb{N}$ e questa cosa viene tradotta dalla proposizione in una decidibilità sintattica per $\mathsf{Q}$.

Operativamente, nonostante (3) $\implies$ (2) $\iff$ (1), la dimostrazione procede nell'ordine opposto, quindi dimostriamo prima (1) e (2), per poi dimostrare (3). Iniziamo con lo studiare l'aritmetica di $\mathsf{Q}$.

**Definizione 6.33** (Numerali)**.** Sia $n \in \mathbb{N}$, chiamo $\overline{n} \overset{\text{def}}{=} s(s(\ldots s(0)\ldots))$, con $n$ successori, **numerale** di $n$, formalmente questi sono termini di $L_{\text{arit}}$ e:

$$\overline{n} := \begin{cases} \overline{0} = 0 & \text{se } n = 0 \\ \overline{n+1} = s(\overline{n}) & \text{altrimenti} \end{cases}$$

Alternativamente un numerale è una mappa che associa ogni naturale ad un $L_{\text{arit}}$-termine.

**Lemma 6.34** (Q sa fare i conti)
Sia $t$ un termine di $L_{\text{arit}}$ chiuso, detto $n = \{\ldots\}_{\mathbb{N}} t$, allora vale che:

$$Q \vdash t = \overline{n}$$

ovvero l'interpretazione del numerale coincide con quella del termine $t$ che dà lo stesso numero.

*Dimostrazione.* Induzione strutturale sugli $L_{\text{arit}}$-termini chiusi. Un $L_{\text{arit}}$-termine chiuso è o un successore di qualche altro termine, o il simbolo di costante 0, o una somma o un prodotto di termini chiusi - non possono esserci variabili, essendo libere.

- $t = 0$ In questo caso $n = 0$, indipendentemente dalla valutazione essendo chiuso, e $\overline{0} = 0$ per definizione, quindi $Q \vdash t = \overline{n}$, per Wk + $\text{In}_=$.
- $t = s(t')$ Per ipotesi induttiva, detto $m := \{\ldots\}_{\mathbb{N}} t'$, vale che $Q \vdash t' = \overline{m}$, e quindi $Q \vdash s(t') = s(\overline{m})$, per $\text{El}_=$, inoltre per definizione di numerale $s(\overline{m}) = \overline{m+1}$, e per definizione di interpretazione $\{\ldots\}_{\mathbb{N}} s(t') = m + 1$.
- $t = t_1 + t_2$ Detti $a := \{\ldots\}_{\mathbb{N}} t_1$, $b := \{\ldots\}_{\mathbb{N}} t_2$ e $c := \{\ldots\}_{\mathbb{N}} t$, vale per ipotesi induttiva che $Q \vdash t_1 = \overline{a}$ e $Q \vdash t_2 = \overline{b}$, inoltre, poiché $\mathbb{N} \models (t = t_1 + t_2)$ vale

$c = a + b$. Vogliamo dimostrare che $Q \vdash t = \overline{c}$, ovvero, per $\text{El}_=$, che $Q \vdash t_1 + t_2 = \overline{c}$, che si riduce, sempre per $\text{El}_=$, a dimostrare che $Q \vdash \overline{a} + \overline{b} = \overline{c}$. Ci siamo quindi ridotti a voler verificare:

$$a + b = c \implies Q \vdash \overline{a} + \overline{b} = \overline{c}$$

e lo dimostriamo usando l'induzione metateorica su $b$. Se $b = 0$, allora $a + 0 = a$, e quindi $c = a$, per Q3 vale che $\mathsf{Q} \vdash \overline{a} + \overline{0} = \overline{a}$, e per definizione di numerale $\overline{0} = 0$, quindi vale la tesi. Assumiamo che $a + b = c \implies Q \vdash \overline{a} + \overline{b} = \overline{c}$, e vediamo che $a + s(b) = c$, allora $\mathsf{Q} \vdash \overline{a} + \overline{b+1} = \overline{c}$, ora per definizione di numerale $\overline{b+1} = \overline{b} + \overline{1}$, e per Q4 vale che $\mathsf{Q} \vdash \overline{a} + \overline{b+1} = s(\overline{a} + \overline{b})$, che, per $\text{El}_=$, è equivalente a $\mathsf{Q} \vdash \overline{a} + \overline{b+1} = s(\overline{c})$, e per definizione di numerale $s(\overline{c}) = \overline{c+1}$, che corrisponde alla tesi perché semanticamente $a + s(b) = s(a + b) = s(c)$.

- $t = t_1 \cdot t_2$ Analogo al caso precedente.

□

**Lemma 6.35** (Caso base del teorema di equivalenza)
Sia $\varphi$ una formula atomica chiusa, allora:

$$\mathbb{N} \models \varphi \iff \mathsf{Q} \vdash \varphi \quad \text{e} \quad \mathbb{N} \models \neg\varphi \iff \mathsf{Q} \vdash \neg\varphi$$

*Dimostrazione.* L'unica $L_{\text{arit}}$-formule atomica chiusa è del tipo $t_1 = t_2$, con $t_1, t_2$ termini chiusi. Se $\mathbb{N} \models t_1 = t_2$, allora $\{\ldots\}_{\mathbb{N}} t_1 = \{\ldots\}_{\mathbb{N}} t_2$, per il lemma precedente valgono $\mathsf{Q} \vdash t_1 = \overline{\{\ldots\}_{\mathbb{N}} t_1}$ e $\mathsf{Q} \vdash t_2 = \overline{\{\ldots\}_{\mathbb{N}} t_2}$, e quindi usando due volte $\text{El}_=$ otteniamo questa implicazione, $\mathsf{Q} \vdash t_1 = t_2$.
Per il viceversa dimostriamo la contronominale, i.e. $\mathbb{N} \models \neg(t_1 = t_2)$, allora $\mathsf{Q} \vdash \neg(t_1 = t_2)$. WLOG assumiamo che $n_1 := \{\ldots\}_{\mathbb{N}} t_1 < \{\ldots\}_{\mathbb{N}} t_2 =: n_2$, dal lemma sappiamo che $\mathsf{Q} \vdash t_1 = \overline{n_1}$ e $\mathsf{Q} \vdash t_2 = \overline{n_2}$, per cui è sufficiente dimostrare che $\mathsf{Q} \vdash \neg(\overline{n_1} = \overline{n_2})$. Per induzione metateoretica su $n_1$ (precisamente vogliamo dimostrare $P(n) : \forall m(n < m \implies \mathsf{Q} \vdash \neg(\overline{n} = \overline{m}))$, se $n_1 = 0$, allora $n_2 > 0$ e vogliamo dimostrare che $\mathsf{Q} \vdash \neg(\overline{0} = \overline{n_2})$, con $\overline{0} = 0$ per definizione, da $0 < n_2$ segue che $\overline{n_2} = s(\overline{x})$ per qualche $x$, e quindi:

$$Q \vdash \neg(\overline{0} = s(\overline{x}))$$

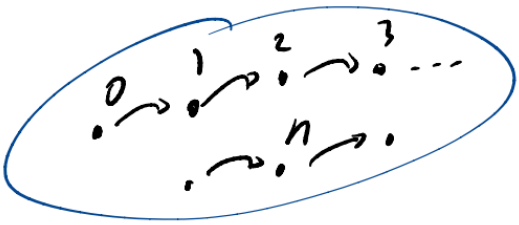
che è proprio Q1. Ora per ipotesi induttiva $\forall m(n < m \implies \mathsf{Q} \vdash \neg(\overline{n} = \overline{m}))$, e vogliamo dimostrare che $\forall q\ n + 1 < q$ allora $\mathsf{Q} \vdash \neg(\overline{n+1} = \overline{q})$. Fisso $q$ tale che $n + 1 < q$, allora $q = m + 1$ per qualche $m$, da questo segue che $n < m$, e quindi $\mathsf{Q} \vdash \neg(\overline{n} = \overline{m})$, infine per Q2 vale che $\mathsf{Q} \vdash \neg(\overline{n+1} = \overline{m+1})$, che è proprio $\mathsf{Q} \vdash \neg(\overline{n+1} = \overline{q})$. □

**Definizione 6.36** ("Ordinamento" sui termini aritmetici)**.** Siano $t_1, t_2$ $L_{\text{arit}}$-termini, dico che $t_1 \leq t_2$ se $\exists x\ x + t_1 = t_2$.[50]

**Definizione 6.37** (Elementi standard e non standard)**.** Sia $M \models \mathsf{Q}$, sia $n \in M$, dico che $n$ è **standard** o **non standard** se $\exists x\ M \models n = \overline{x}$ o $\forall x \in M\ M \models n \neq \overline{x}$. Cioè dico che un elemento è standard se è uguale ad un numerale, e non standard se è diverso da ogni numerale.

[50] Questa è una relazione di ordinamento, ma Q non è in grado di dimostrare che è un ordinamento, inoltre si poteva anche mettere $+x$ a posto di $x+$, visto che Q non è in grado di dimostrare che $+$ è commutativo, quindi in principio sono due cose diverse.

Gli elementi non standard si possono pensare come elementi fuori dalla catena dei successori, e andranno pensati come più grandi di tutti i numerali.




Indaghiamo ora come funziona questa buffa nozione di $\leq$ per $\mathsf{Q}$, e vediamo che è proprio questa nozione di ordinamento che ci permette di distinguere tra elementi standard e non standard.

**Lemma 6.38** (Caratterizzazione di $a \leq \overline{n}$)
$\forall n \in \mathbb{N}$ vale che:

$$\mathsf{Q} \vdash \forall a(a \leq \overline{n} \leftrightarrow (a = \overline{0} \vee a = \overline{1} \vee \ldots \vee a = \overline{n}))$$

cioè ogni elemento del modello minore o uguale ad un numerale è uguale ad un numerale.

**Lemma 6.39** (Ogni elemento non standard è maggiore di ogni elemento standard)
Sia $M \models \mathsf{Q}$, sia $\alpha \in M$ non standard, allora:

$$\forall n \in \mathbb{N}\ M \models \alpha \leq \overline{n}$$

Dimostriamo in primis il secondo lemma, che è più semplice.

*Dimostrazione.* Procediamo per induzione, metateorica, su $n$.

$\boxed{n = 0}$ Vogliamo dimostrare che $M \models \overline{0} \leq \alpha$, ovvero che $M \models \exists x\ x + \overline{0} = \alpha$, e questo, per la semantica di Tarski equivale a:

$$\exists x \in M\ x + 0 = \alpha$$

prendendo $x = \alpha$, e poiché vale $\mathsf{Q}4$, otteniamo la tesi.

$\boxed{n \implies n+1}$ Vogliamo dimostrare che $M \models \alpha \leq \overline{n+1}$, come sopra, questo equivale a $M \models \exists x\ x + \overline{n+1} = \alpha$, e usando la semantica di Tarski diventa:

$$\exists x \in M\ x + \{\ldots\}_M \overline{n+1} = \alpha$$

dove, essendo per definizione $\overline{n+1} = s(\overline{n})$, vale che $\{\ldots\}_M \overline{n+1} = s(\{\ldots\}_M \overline{n})$, e quindi, per $\mathsf{Q}4$, $x + \{\ldots\}_M \overline{n+1} = s(x + \{\ldots\}_M \overline{n})$. Per $\mathsf{Q}7$ vale che $\alpha = s(\beta)$ - con $\beta$ non standard altrimenti si ha un assurdo, e banalmente, per $\mathsf{Q}2$, da $\overline{n+1} \leq \alpha$ segue che $\overline{n} \leq \beta$; usano infine l'ipotesi induttiva troviamo $y \in M$ tale che $y + \{\ldots\}_M \overline{n} = \beta$, e quindi $s(y + \{\ldots\}_M \overline{n}) = s(\beta)$, pertanto è sufficiente prendere $x = y$ per ottenere la tesi.

$\square$

Vediamo infine la dimostrazione del primo lemma.

*Dimostrazione.* Per induzione metateorica su $n$.

$\boxed{n = 0}$ Vogliamo dimostrare che $\mathsf{Q} \vdash \forall a(a \leq \overline{0} \leftrightarrow a = \overline{0})$, per definizione $a \leq 0$ equivale a $\exists x\ x + a = 0$, da cui segue subito $\leftarrow$, infatti in questo caso cerchiamo un $x$ tale che $x + 0 = 0$ e prendendo $x = 0$ otteniamo la tesi per Q3.
Per $\rightarrow$, se $a \leq 0$, allora $\exists x\ x + a = 0$ e vogliamo dimostrare che $a = 0$. Per assurdo, se $a \neq 0$, allora per Q7 vale che $a = s(y)$ per qualche $y$, e quindi, per Q4 vale che $x + s(y) = s(x + y) = 0$ che contraddice Q1, assurdo.

$\boxed{n \implies n + 1}$ Ci serve un lemma preliminare.

**Lemma 6.40** (Il successore non fa salti)
Nello stesso setting di prima vale:

$$\mathsf{Q} \vdash a \leq \overline{n+1} \leftrightarrow (a = 0) \vee \exists b(a = s(b) \wedge b \leq \overline{n})$$

*Dimostrazione.* In primis la tesi si può riscrivere come:

$$\mathsf{Q} \vdash \exists x(x + a = s(\overline{n})) \leftrightarrow (a = 0) \vee \exists b(a = s(b) \wedge \exists y(y + b = \overline{n}))$$

Per $\leftarrow$, se $a = 0$, allora si prende come $x$ $\{\ldots\}_M s(\overline{n})$ e si conclude per Q3; se $a = s(b)$ e $y + b = \overline{n}$, allora per Q4 vale che $y + s(b) = s(y + b) = s(\overline{n})$, e quindi si prende $x = y$ per concludere.
Viceversa, per $\rightarrow$, se $a \leq \overline{n+1}$, allora, se $a = 0$ abbiamo finito, altrimenti per Q7 vale che $a = s(b)$ e dall'ipotesi $\exists x\ x + s(b) = s(\overline{n})$, per Q4 e Q2 vale che $x + b = \overline{n}$, e quindi $b \leq \overline{n}$. □

Ora vogliamo dimostrare che:

$$\mathsf{Q} \vdash a \leq \overline{n+1} \leftrightarrow (a = \overline{0}) \vee (a = \overline{1}) \vee \ldots \vee (a = \overline{n+1})$$

Per $\leftarrow$, se $a = \overline{n+1}$ allora è banale che $a \leq \overline{n+1}$, se $a = \overline{m}$ con $m \leq n$, allora per ipotesi induttiva $\mathsf{Q} \vdash a \leq \overline{n}$, e quindi banalmente $a \leq \overline{n+1}$.
Viceversa, per $\rightarrow$, se $a \leq \overline{n+1}$, allora per il lemma precedente vale che $a = 0$ o $a = s(b)$ con $b \leq \overline{n}$, se $a = 0$ allora abbiamo finito; altrimenti, se $a = s(b)$ con $b \leq \overline{n}$, allora per ipotesi induttiva $b = \overline{m}$ per qualche $m \leq n$, e quindi $a = s(\overline{m}) = \overline{m+1}$, con $m + 1 \leq n + 1$, e quindi abbiamo finito.

□

Finalmente possiamo dimostrare la prima parte della proposizione sull'equivalenza tra validità in $\mathbb{N}$ e dimostrabilità in $\mathsf{Q}$ per le formule di classe $\Delta_0^0$ e $\Sigma_1^0$. In particolare ora vediamo la dimostrazione di (1) e (2).

*Dimostrazione.* Dimostriamo contemporaneamente (1) e (2). L'implicazione $\Longleftarrow$ è banale poiché $\mathbb{N}$ è un modello di $\mathsf{Q}$, per cui se $\mathsf{Q} \vdash \varphi$, allora $\mathbb{N} \models \varphi$.
Per dimostrare $\Longrightarrow$, procediamo per induzione strutturale, contemporaneamente sia su (1) che su (2),sulla $L_{\text{arit}}$-formula $\varphi$ chiusa di classe $\Delta_0^0$, possiamo limitarci ai soli connettivi logici $\neg$ e $\wedge$, perché $\vee$ si ottiene da loro, e au quantificatori $\forall x \leq t$ e $\exists x \leq t$.

- $\varphi$ è atomica È il lemma già visto.

- $\varphi = \neg\psi$ Abbiamo due casi o $\mathbb{N} \models \neg\psi$ o $\mathbb{N} \models \neg(\neg\psi)$, nel primo caso basta l'ipotesi induttiva (2): $\mathbb{N} \models \neg\psi \implies \mathsf{Q} \vdash \neg\psi$. Il caso in cui invece $\mathbb{N} \models \neg(\neg\psi)$ è banale, infatti sappiamo già che $\models \neg(\neg\psi) \iff \psi$ e che $\vdash \neg(\neg\psi) \iff \psi$, per cui si riduce all'ipotesi induttiva (1), cioè $\mathbb{N} \models \psi \implies \mathsf{Q} \vdash \psi$.

- $\varphi = \psi_1 \wedge \psi_2$ Come prima si danno due casi, quello in cui $\mathbb{N} \models \psi_1 \wedge \psi_2$, che segue dalla semantica di Tarski:

$$\begin{aligned} \mathbb{N} \models \psi_1 \wedge \psi_2 &\iff \mathbb{N} \models \psi_1 \ \wedge\ \mathbb{N} \models \psi_2 && \text{(Tarski)} \\ &\iff \mathsf{Q} \vdash \psi_1 \ \wedge\ \mathsf{Q} \vdash \psi_2 && \text{(hp induttiva (1))} \\ &\iff \mathsf{Q} \vdash \psi_1 \wedge \psi_2 && (\mathrm{In}_\wedge) \end{aligned}$$

  Se invece $\mathbb{N} \models \neg(\psi_1 \wedge \psi_2)$, allora (De Morgan) vale $\mathbb{N} \models \neg\psi_1 \vee \neg\psi_2$; dunque si danno necessariamente due sottocasi.

  - Se $\mathbb{N} \models \neg\psi_1$, per ipotesi induttiva (2) abbiamo $\mathsf{Q} \vdash \neg\psi_1$, e quindi, usando $\mathrm{In}_\vee$. e De Morgan, $\mathsf{Q} \vdash \neg(\psi_1 \wedge \psi_2)$.
  - Se $\mathbb{N} \models \neg\psi_2$, analogamente otteniamo $\mathsf{Q} \vdash \neg\psi_2$, e quindi $\mathsf{Q} \vdash \neg(\psi_1 \wedge \psi_2)$.

- $\varphi = \forall x \leq t\ \psi$ Assumiamo che $\mathbb{N} \models \forall x \leq t\ \psi$, con $t$ termine chiuso perché lo è $\varphi$, e vogliamo dimostrare che $\mathsf{Q} \vdash \forall x \leq t\ \psi$. Sia $\{t\}_{\mathbb{N}} = n$, per un lemma sappiamo che $\mathsf{Q} \vdash t = \overline{n}$, inoltre, per un altro lemma, $\mathsf{Q} \vdash \forall x \leq t\ \psi \leftrightarrow \forall x(x = \overline{0} \vee x = \overline{1} \vee \ldots \vee x = \overline{n}) \to \psi$, e questa formula equivale a:

$$(\forall x\ x = \overline{0} \to \psi) \wedge (\forall x\ x = \overline{1} \to \psi) \wedge \ldots \wedge (\forall x\ x = \overline{n} \to \psi)$$

  dove ciascuna delle sottoformule $\forall x\ x = \overline{m} \to \psi$ è proprio $\psi[m/x]$ - e questo si verifica tramite semantica di Tarski. Per cui la tesi si riduce a dimostrare che:

$$\mathsf{Q} \vdash \psi[0/x] \wedge \psi[1/x] \wedge \ldots \wedge \psi[n/x]$$

  ed a questo punto la tesi segue dal caso dell'$\wedge$.

- $\varphi = \exists x \leq t\ \psi$ Esattamente come nel caso di sopra si arriva a voler dimostrare:

$$\mathsf{Q} \vdash \psi[0/x] \vee \psi[1/x] \vee \ldots \vee \psi[n/x]$$

  e anche in questo caso la tesi segue dal caso dell'$\vee$.

□

Infine vediamo anche la dimostrazione di (3).

*Dimostrazione.* Vogliamo dimostrare che se $\varphi$ è chiusa di classe $\Sigma^0_1$, allora $\mathbb{N} \models \varphi \iff \mathsf{Q} \vdash \varphi$. Come per il caso $\Delta^0_0$, l'implicazione $\Longleftarrow$ è banale in quanto $\mathbb{N}$ è un modello di $\mathsf{Q}$. Sia ora $\varphi = \exists x_1 \ldots \exists x_k\ \psi$, con $\psi$ di classe $\Delta^0_0$ e supponiamo che $\mathbb{N} \models \varphi$, allora, per la semantica di Tarski + il lemma sulla sostituzione, esistono $n_1, \ldots, n_k \in \mathbb{N}$ tali che:

$$\mathbb{N} \models \psi[n_1/x_1, \ldots, n_k/x_k]$$

ora, la formula ottenuta sostituendo i numerali $\overline{n_1}, \ldots, \overline{n_k}$ a $x_1, \ldots, x_k$ è di classe $\Delta^0_0$, per cui vale l'ipotesi induttiva e si ha $\mathsf{Q} \vdash \psi[\overline{n_1}/x_1, \ldots, \overline{n_k}/x_k]$. Per concludere è sufficiente usare $\mathrm{In}_\exists$ per $k$ volte e si ottiene $\mathsf{Q} \vdash \varphi$. □

A questo punto, dalle cose viste, sappiamo che il luogo di zero di una funzione ricorsiva è un insieme semidecidibile, e quindi è rappresentabile da una formula aritmetica di classe $\Sigma^0_1$, e che per i parametri per cui la formula è vera, $\mathsf{Q}$ dimostra proprio la formula con i numerali al posto dei parametri, ovvero enunciato precisamente.

**Corollario 6.41** (Rappresentazione (del grafico) di funzioni ricorsive in Q)
Se $f : \mathbb{N}^k \to \mathbb{N}_\perp$ è ricorsiva, allora c'è $\varphi(x_1, \dots, x_k, y)$ di classe $\Sigma_1^0$ tale che:

$$\forall x_1, \dots, x_k \in \mathbb{N}\ \forall y \in \mathbb{N}\ (\mathsf{Q} \vdash \varphi(\overline{x_1}, \dots, \overline{x_k}, \overline{y}) \iff f(x_1, \dots, x_k) = y)$$

*Dimostrazione.* La dimostrazione si ottiene considerando $\{f(x_1, \dots, x_k) - y = 0\}$, che è semidecidibile, e per la caratterizzazione degli insiemi semidecidibili come gli insiemi di classe $\Sigma_1^0$, esiste una formula di classe $\Sigma_1^0$ tale $(x_1, \dots, x_k, y) \in \{f(x_1, \dots, x_k) - y = 0\}$ se e solo se $\mathbb{N} \models \varphi(x_1, \dots, x_k, y)$. Infine per la proposizione appena dimostrata vale che $\mathbb{N} \models \varphi(x_1, \dots, x_k, y)$ se e solo se $\mathsf{Q} \vdash \varphi(\overline{x_1}, \dots, \overline{x_k}, \overline{y})$, e questa è proprio l'equivalenza della tesi. □

Vediamo un risultato più forte, visto che il precedente è una condizione puntuale, nel senso che fissati a priori i parametri si ha l'equivalenza, possiamo migliorare l'equivalenza richiedendo che la funzione ricorsiva sia totale, e ottenendo di conseguenza che la Q capisca che $f$ è una funzione, ovvero che per ogni $x_1, \dots, x_k$ esiste un unico $y$ tale che $\varphi(x_1, \dots, x_k, y)$ è vera, e che questo $y$ è proprio $f(x_1, \dots, x_k)$.

**Lemma 6.42** (Rappresentazione di funzioni ricorsive totali in Q)
Sia $f : \mathbb{N}^k \to \mathbb{N}$ ricorsiva totale, allora c'è $\varphi(x_1, \dots, x_k, y)$ di classe $\Sigma_1^0$ tale che:

$$\forall x_1, \dots, x_k \in \mathbb{N}\ \mathsf{Q} \vdash \forall y\ (\varphi(\overline{x_1}, \dots, \overline{x_k}, y) \leftrightarrow y = \overline{f(x_1, \dots, x_k)})$$

*Dimostrazione.* Sia $\psi$ una formula di classe $\Delta_0^0$ tale che per ogni $x_1, \dots, x_k, y \in \mathbb{N}$:

$$y = f(x_1, \dots, x_k) \iff \mathsf{Q} \vdash \exists z_1 \dots \exists z_l\, \psi(\overline{x_1}, \dots, \overline{x_k}, \overline{y}, z_1, \dots, z_l)$$

Consideriamo la formula $\varphi(x_1, \dots, x_k, y)$ definita come:

$$\begin{aligned}&\exists N(\exists z_1 \le N \dots \exists z_l \le N\, (y \le N \wedge \psi(x_1, \dots, x_k, y, z_1, \dots, z_l))\\ &\quad \wedge \forall t \le N\, \forall w_1 \le N \dots \forall w_l \le N\, (\psi(x_1, \dots, x_k, t, w_1, \dots, w_l) \to t = y))\end{aligned}$$

Chiamiamo $\theta_1$ la prima riga tra parentesi e $\theta_2$ la seconda. Dimostriamo separatamente le due implicazioni:

$\boxed{\Longrightarrow}$ Vogliamo mostrare che $\mathsf{Q} \vdash \forall y\, (\varphi(\overline{x_1}, \dots, \overline{x_k}, y) \to y = \overline{f(x_1, \dots, x_k)})$. Supponiamo, per assurdo, che non sia così e fissiamo $x_1, \dots, x_k$ e un modello $M \models \mathsf{Q}$ in cui esiste $y \neq f(x_1, \dots, x_k)$ tale che $M \models \varphi(\overline{x_1}, \dots, \overline{x_k}, y)$. Siccome $M \models \varphi$, possiamo fissare $N \in M$ tale che $N$ e $y$ soddisfino, in $M$, le formule $\theta_1$ e $\theta_2$. Si presentano due casi:

- *Caso 1: $N$ è standard.* Per $\theta_1$, anche $y$ deve essere standard, ed esistono $z_1, \dots, z_l \in M$ standard tali che $M \models \psi(\overline{x_1}, \dots, \overline{x_k}, \overline{y}, \overline{z_1}, \dots, \overline{z_l})$. Siccome $\psi$ è $\Delta_0^0$, essa è decidibile, quindi $\mathsf{Q} \vdash \psi(\overline{x_1}, \dots, \overline{x_k}, \overline{y}, \overline{z_1}, \dots, \overline{z_l})$, il che implica $y = f(x_1, \dots, x_k)$, contro l'ipotesi.
- *Caso 2: $N$ non è standard.* La formula $\theta_2$ asserisce che ogni testimone di $\psi$ minore di $N$ deve corrispondere a $y$. Per costruzione, esistono $z_1, \dots, z_l$ standard tali che $\mathsf{Q} \vdash \psi(\overline{x_1}, \dots, \overline{x_k}, \overline{f(x_1, \dots, x_k)}, \overline{z_1}, \dots, \overline{z_l})$. Poiché ogni numerale standard è minore di un elemento non standard $N$, ponendo $t = \overline{f(x_1, \dots, x_k)}$, i testimoni standard soddisfano la premessa di $\theta_2$, da cui $M \models \overline{f(x_1, \dots, x_k)} = y$, assurdo.

$\boxed{\Longleftarrow}$ Vogliamo mostrare che $\mathsf{Q} \vdash \forall y\,(y = \overline{f(x_1, \dots, x_k)} \to \varphi(\overline{x_1}, \dots, \overline{x_k}, y))$. È sufficiente provare che $\mathsf{Q} \vdash \varphi(\overline{x_1}, \dots, \overline{x_k}, \overline{f(x_1, \dots, x_k)})$.
Siano $z_1, \dots, z_l$ tali che $\mathsf{Q} \vdash \psi(\overline{x_1}, \dots, \overline{x_k}, \overline{f(x_1, \dots, x_k)}, \overline{z_1}, \dots, \overline{z_l})$. Scegliamo $N := \max\{f(x_1, \dots, x_k), z_1, \dots, z_l\}$, in ogni modello $M \models \mathsf{Q}$, il numerale $\overline{N}$ funge da limitazione corretta:

1. $\theta_1$ è soddisfatta in $M$ dai testimoni $\overline{z_1}, \dots, \overline{z_l}$ e $\overline{f(x_1, \dots, x_k)}$, che sono tutti minori o uguali a $\overline{N}$.
2. Per $\theta_2$, se $M \models t \leq \overline{N} \wedge \psi(\dots, t, \dots)$, allora $t$ deve essere standard (perché minore di un numerale standard). Per la costruzione di $\psi$, l'unico valore standard che soddisfa la relazione è proprio $f(x_1, \dots, x_k)$, quindi $M \models t = \overline{f(x_1, \dots, x_k)}$.

$\square$

# §7 Teoremi di incompletezza

## §7.1 Numerazione di Gödel

Una numerazione di Gödel è in generale un modo di associare ad ogni formula di un linguaggio una numero naturale, il modo specifico di farlo non è importante, l'importante è che sia una codifica efficace, ovvero che sia possibile risalire alla formula a partire dal numero e viceversa, e che sia possibile decidere se un numero è la codifica di una formula o meno.

Fissiamo una codifica effettiva dei simboli di $L_{\text{arit}}$ mediante numeri naturali distinti e usiamo la codifica di liste $\langle a_1, \dots, a_n \rangle$ introdotta prima per concatenare i codici. Usiamo i tag per numerare i simboli, usiamo il sistema ridotto, quindi i simboli numerati sono:

| $\bot$ | $\to$ | $\forall$ | $=$ | $0$ | $s$ | $+$ | $\cdot$ | $x_i$ |
|---|---|---|---|---|---|---|---|---|
| $0$ | $1$ | $2$ | $3$ | $4$ | $5$ | $6$ | $7$ | $8+i$ |
| $\#\bot$ | $\#\to$ | $\#\forall$ | $\#=$ | $\#0$ | $\#s$ | $\#+$ | $\#\cdot$ | $\#x_i$ |

quindi d'ora in poi $\#\cdot$ è il numero che abbiamo scelto per codificare il simbolo di moltiplicazione, $\#\forall$ è il numero che abbiamo scelto per codificare il quantificatore universale, e così via. La scelta concreta dei numeri è irrilevante: ciò che conta è che le operazioni sintattiche fondamentali siano funzioni ricorsive.

**Definizione 7.1** (Codifica di termini e formule)**.** Fissata una numerazione dei simboli di $L_{\text{arit}}$, possiamo definire ricorsivamente la **codifica di Gödel** (o **numerazione di Gödel**) $\ulcorner \cdot \urcorner$ come segue.

- $L_{\text{arit}}$-termini:

$$\ulcorner 0 \urcorner := \langle \#0 \rangle \qquad \ulcorner x_i \urcorner := \langle \#x_i \rangle$$
$$\ulcorner s(t) \urcorner := \langle \#s, \ulcorner t \urcorner \rangle \qquad \ulcorner t_1 + t_2 \urcorner := \langle \#+, \ulcorner t_1 \urcorner, \ulcorner t_2 \urcorner \rangle$$
$$\ulcorner t_1 \cdot t_2 \urcorner := \langle \#\cdot, \ulcorner t_1 \urcorner, \ulcorner t_2 \urcorner \rangle$$

- $L_{\text{arit}}$-formule:

$$\ulcorner t_1 = t_2 \urcorner := \langle \#=, \ulcorner t_1 \urcorner, \ulcorner t_2 \urcorner \rangle \qquad \ulcorner \neg\varphi \urcorner := \langle \#\neg, \ulcorner \varphi \urcorner \rangle$$
$$\ulcorner \varphi \to \psi \urcorner := \langle \#\to, \ulcorner \varphi \urcorner, \ulcorner \psi \urcorner \rangle \qquad \ulcorner \forall x_i\, \varphi \urcorner := \langle \#\forall, \#x_i, \ulcorner \varphi \urcorner \rangle$$

Abbiamo così ottenuto una numerazione iniettiva di ogni $L_{\text{arit}}$-termine e di ogni $L_{\text{arit}}$-formula. Il nostro obiettivo sarà dimostrare il seguente lemma.

**Lemma 7.2** (Lemma di diagonalizzazione di Gödel)
Sia $\varphi$ una formula aritmetica, esiste una formula aritmetica $\psi$ tale che:

$$\mathsf{Q} \vdash \psi \leftrightarrow \varphi[\overline{\ulcorner \psi \urcorner}/x_1]$$

cioè per ogni formula con una variabile libere né esiste un'altra che è vera se e solo se è vera "$\varphi(\psi)$".

Di fatto il lemma ci dà una specie di teorema di punto fisso per le formule, cioè per ogni formula ne esiste un'altra la cui validità è equivalente alla validità della proprietà che la prima formula esprime valida per la seconda. In altre parole, c'è sempre una formula che dice di sé stessa che è vera se e solo se è vera una certa proprietà per lei. La proprietà che ci interesserà sarà "io sono dimostrabile".
Osserviamo che non tutti i numeri naturali sono codici di formule, per la nostra codifica, da cui la necessità di poter decidere se un numero è codice di una formula o meno.

**Osservazione 7.3** ($n$ è il codice di una formula) — Il predicato $\mathsf{For}(n) :=$ "$n$ è codice di una formula" è PR.

*Dimostrazione.* Per decidere se $n$ è codice di una formula, basta verificare che il primo simbolo della lista sia un connettivo o che la formula sia atomica, per cui il predicato $\mathsf{For}(n)$ si può definire per casi come segue:

$$\begin{aligned}
\mathsf{For}(n) = {} & (\mathtt{nth}(n,0) = \#\bot \wedge \mathtt{length}(n) = 1) \\
& \wedge (\mathtt{nth}(n,0) = \# = \wedge \mathtt{length}(n) = 3 \wedge \mathsf{Term}(\mathtt{nth}(n,1)) \wedge \mathsf{Term}(\mathtt{nth}(n,2))) \\
& \wedge (\mathtt{nth}(n,0) = \#\neg \wedge \mathtt{length}(n) = 2 \wedge \mathsf{For}(\mathtt{nth}(n,1))) \\
& \wedge (\mathtt{nth}(n,0) = \# \rightarrow \wedge \mathtt{length}(n) = 3 \wedge \mathsf{For}(\mathtt{nth}(n,1)) \wedge \mathsf{For}(\mathtt{nth}(n,2))) \\
& \wedge (\mathtt{nth}(n,0) = \#\forall \wedge \mathtt{length}(n) = 3 \wedge \mathtt{nth}(n,1) \geq 8 \wedge \mathsf{For}(\mathtt{nth}(n,2)))
\end{aligned}$$

dove aggiungiamo gli altri $\mathsf{For}$ ricorsivamente, per la precisione per ricorsione sul corso dei valori, perché la formula codificata da $n$ è una formula se e solo se le sottoformule codificate dai sotto-codici sono formule, ovviamente, essendo la definizione fatta da un'intersezione finita di predicati PR, $\mathsf{For}$ è PR. Osserviamo infine che c'è necessità anche di introdurre il predicato $\mathsf{Term}$, che dice se un numero è codice di un termine, ma anche questo è PR, e si definisce in modo seguente:

$$\begin{aligned}
\mathsf{Term}(n) = {} & (\mathtt{nth}(n,0) = \#0 \wedge \mathtt{length}(n) = 1) \\
& \wedge (\mathtt{nth}(n,0) = \#x_i \wedge \mathtt{length}(n) = 1) \\
& \wedge (\mathtt{nth}(n,0) = \#f_i \wedge \mathtt{length}(n) = 1) \\
& \wedge (\mathtt{nth}(n,0) = \#s \wedge \mathtt{length}(n) = 2 \wedge \mathsf{Term}(\mathtt{nth}(n,1))) \\
& \wedge (\mathtt{nth}(n,0) = \# + \wedge \mathtt{length}(n) = 3 \wedge \mathsf{Term}(\mathtt{nth}(n,1)) \wedge \mathsf{Term}(\mathtt{nth}(n,2))) \\
& \wedge (\mathtt{nth}(n,0) = \# \cdot \wedge \mathtt{length}(n) = 3 \wedge \mathsf{Term}(\mathtt{nth}(n,1)) \wedge \mathsf{Term}(\mathtt{nth}(n,2)))
\end{aligned}$$

anche in questo caso la definizione è fatta per ricorsione sul corso dei valori intersecando cose PR. □

**Osservazione 7.4** ($n$ è il codice di un numerale) — Definiamo la funzione $\mathsf{Num}(n)$ :

$n \mapsto \ulcorner \overline{n} \urcorner$, ovvero la funzione che associa ad ogni numero naturale il numerale corrispondente, tale funzione è PR.

*Dimostrazione.* Come prima si può definire a partire da altre funzioni PR come segue:

$$\mathsf{Num}(n) := \begin{cases} \mathsf{Num}(0) = \ulcorner 0 \urcorner = \langle \#0 \rangle \\ \mathsf{Num}(n+1) = \ulcorner s(\overline{n}) \urcorner = \langle \#s, \mathsf{Num}(n) \rangle \end{cases}$$

□

**Osservazione 7.5** (Codice della sosituzione) — Sia $\mathsf{Sub}(\ulcorner \varphi \urcorner, i, \ulcorner t \urcorner) := \ulcorner \varphi[t/x_i] \urcorner$ la funzione che associa il codice di una formula, di una variabile e di un termine al codice ottenuto facendo la sostituzione del termine al posto della variabile, tale funzione è PR.

*Dimostrazione.* Anche questa può essere definita a partire da funzioni PR:

$$\mathsf{Sub}(n, i, x) := \begin{cases} 0 & \text{se } \neg\mathsf{For}(n) \\ \langle \#\bot \rangle & \text{se } n = \langle \#\bot \rangle \\ \langle \# =, \mathsf{Sub}(a, i, x), \mathsf{Sub}(b, i, x) \rangle & \text{se } n = \langle \# =, \ulcorner a \urcorner, \ulcorner b \urcorner \rangle \\ \langle \#\neg, \mathsf{Sub}(a, i, x) \rangle & \text{se } n = \langle \#\neg, \ulcorner a \urcorner \rangle \\ \langle \# \to, \mathsf{Sub}(a, i, x), \mathsf{Sub}(b, i, x) \rangle & \text{se } n = \langle \# \to, \ulcorner a \urcorner, \ulcorner b \urcorner \rangle \\ \langle \#\forall, j, \mathsf{Sub}(a, i, x) \rangle & \text{se } n = \langle \#\forall, j, \ulcorner a \urcorner \rangle \text{ e } j \neq i \\ \langle \#\forall, j, a \rangle & \text{se } n = \langle \#\forall, j, \ulcorner a \urcorner \rangle \text{ e } j = i \end{cases}$$

□

Possiamo quindi dimostrare il lemma di diagonalizzazione di Gödel.

*Dimostrazione.* Come nel caso di teorema di punto fisso per le funzioni ricorsive, vediamo informalmente cosa vorremmo fare, e poi lo formalizziamo. Supponiamo che tutto sia applicabile a tutto allora considero la funzione $f(x) := x(x)$, ovvero la funzione che prende una formula e la applica a se stessa, il nostro scopo è trovare $\psi = \varphi(\psi)$ - dove con l'$=$ intendiamo l'equivalenza, allora se potessimo definire $f$ potremmo definire la formula $\theta := \varphi(f(x))$ e quindi applicando $\theta$ a se stessa otteniamo $\theta(\theta) = \varphi(f(\theta)) = \varphi(\theta(\theta))$ e chiamando $\theta(\theta) := \psi$ avremmo finito.
La vera funzione $f$ che vogliamo definire fa praticamente la stessa cosa, ma usa i codici invece che le formule:

$$f : \mathbb{N} \to \mathbb{N} : \ulcorner \alpha \urcorner \mapsto \ulcorner \alpha[\overline{\ulcorner \alpha \urcorner}/x_k] \urcorner \quad \text{con } x_k \in \mathrm{vl}(\alpha)$$

tale funzione è PR, perché si può definire come:

$$f(n) = \mathsf{Sub}(n, k, \mathsf{Num}(n))$$

dove la cosa sostituita è appunto il numerale corrispondente al numero di Gödel della formula. Ora vorremmo definire la nostra "$\theta := \varphi(f(x))$", tecnicamente, in primis applichiamo il lemma di rappresentazione di funzioni ricorsive in $\mathsf{Q}$ alla funzione ricorsiva totale $f$, per quel lemma esiste una formula $\pi$, per la quale possiamo assumere (WLOG a meno di rinominare) che $\mathrm{vl}(\pi) \subseteq \{x_1, \ldots, x_k\}$, con $k > 1$,e vale:

$$\forall n \in \mathbb{N}\ \mathsf{Q} \vdash \forall x_1 (\pi[\overline{n}/x_k] \leftrightarrow x_1 = \overline{f(n)})$$

dove $\pi[\overline{n}/x_k]$ è in vece della usuale $\pi(\overline{n}, x_1)$ del lemma, e scelgo proprio $x_k$ perché è quella della definizione di $f$. Ora definiamo $\theta := \forall x_1 \pi \to \varphi$ e $\psi := \theta[\overline{\ulcorner\theta\urcorner}/x_k]$ e verifichiamo la tesi. Per la costruzione di $\pi$, usando $\ulcorner\theta\urcorner \in \mathbb{N}$ abbiamo:

$$\mathsf{Q} \vdash \forall x_1(\pi[\overline{\ulcorner\theta\urcorner}/x_k] \leftrightarrow x_1 = \overline{f(\ulcorner\theta\urcorner)})$$

e per la definizione di $f(\ulcorner\theta\urcorner) = \ulcorner\theta[\overline{\ulcorner\theta\urcorner}/x_k]\urcorner$, che unita alla definizione di $\psi$ è $f(\ulcorner\theta\urcorner) = \ulcorner\psi\urcorner$, dà:

$$\mathsf{Q} \vdash \forall x_1(\pi[\overline{\ulcorner\theta\urcorner}/x_k] \leftrightarrow x_1 = \overline{\ulcorner\psi\urcorner})$$

Ora la formula $\psi = \theta[\overline{\ulcorner\theta\urcorner}/x_k] = (\forall x_1 \pi \to \varphi)[\overline{\ulcorner\theta\urcorner}/x_k]$, sfruttando il fatto che $x_k$ non compare in $\varphi$, per ipotesi la sua variabile libera è $x_1$, è equivalente a:

$$\psi \equiv \forall x_1 \pi[\ulcorner\theta\urcorner/x_k] \to \varphi$$

e quindi, usando la formula precedente, posso rimpiazzare $\pi[\ulcorner\theta\urcorner/x_k]$ con $x_1 = \overline{\ulcorner\psi\urcorner}$, ottenendo:

$$\mathsf{Q} \vdash \psi \leftrightarrow \forall x_1(x_1 = \overline{\ulcorner\psi\urcorner} \to \varphi)$$

infine, osservo che $\forall x_1(x_1 = \overline{\ulcorner\psi\urcorner} \to \varphi)$ è equivalente a $\varphi[\overline{\ulcorner\psi\urcorner}/x_1]$. Per $\to$:

$$\mathrm{El}_{\to}\dfrac{\mathrm{El}_{\forall}\dfrac{\mathrm{Ax}\dfrac{}{\forall x_1(x_1=\overline{\ulcorner\psi\urcorner}\to\varphi)\vdash\forall x_1(x_1=\overline{\ulcorner\psi\urcorner}\to\varphi)}}{\forall x_1(x_1=\overline{\ulcorner\psi\urcorner}\to\varphi)\vdash\overline{\ulcorner\psi\urcorner}=\overline{\ulcorner\psi\urcorner}\to\varphi[\overline{\ulcorner\psi\urcorner}/x_1]} \qquad \mathrm{Wk}\dfrac{\mathrm{In}_{=}\dfrac{}{\vdash\overline{\ulcorner\psi\urcorner}=\overline{\ulcorner\psi\urcorner}}}{\forall x_1(x_1=\overline{\ulcorner\psi\urcorner}\to\varphi)\vdash\overline{\ulcorner\psi\urcorner}=\overline{\ulcorner\psi\urcorner}}}{\forall x_1(x_1 = \overline{\ulcorner\psi\urcorner} \to \varphi) \vdash \varphi[\overline{\ulcorner\psi\urcorner}/x_1]}$$

Per il viceversa:

$$\mathrm{In}_{\forall}\dfrac{\mathrm{In}_{\to}\dfrac{\mathrm{El}_{=}\dfrac{\mathrm{Ax}\dfrac{}{x_1=\overline{\ulcorner\psi\urcorner}\vdash x_1=\overline{\ulcorner\psi\urcorner}} \qquad \mathrm{Wk}\dfrac{\mathrm{In}_{=}\dfrac{}{\vdash x_1=x_1}}{x_1=\overline{\ulcorner\psi\urcorner}\vdash x_1=x_1}}{x_1=\overline{\ulcorner\psi\urcorner}\vdash\overline{\ulcorner\psi\urcorner}=x_1} \qquad \mathrm{Wk}\dfrac{\mathrm{Ax}\dfrac{}{\varphi[\overline{\ulcorner\psi\urcorner}/x_1]\vdash\varphi[\overline{\ulcorner\psi\urcorner}/x_1]}}{x_1=\overline{\ulcorner\psi\urcorner},\varphi[\overline{\ulcorner\psi\urcorner}/x_1]\vdash\varphi[\overline{\ulcorner\psi\urcorner}/x_1]}}{\varphi[\overline{\ulcorner\psi\urcorner}/x_1]\vdash x_1=\overline{\ulcorner\psi\urcorner}\to\varphi}}{\varphi[\overline{\ulcorner\psi\urcorner}/x_1] \vdash \forall x_1(x_1 = \overline{\ulcorner\psi\urcorner} \to \varphi)}$$

dove nell'ultimo passaggio ho usato $\mathrm{In}_{\forall}$, e posso farlo perché $x_1 \notin \mathrm{vl}(\varphi[\overline{\ulcorner\psi\urcorner}/x_1])$. Quindi:

$$\mathsf{Q} \vdash \psi \leftrightarrow \varphi[\overline{\ulcorner\psi\urcorner}/x_1]$$

che è la tesi. □

## §7.2 Predicato di dimostrazione

Il nostro scopo in questa sezione sarà dimostrare che esiste una $L_{\text{arit}}$-formula che dice di sé stessa che è dimostrabile, per poi combinarla col lemma di diagonalizzazione di Gödel ed ottenere il primo teorema di incompletezza di Gödel. Richiamiamo una definizione già data, ma senza formalizzare bene le enumerazioni di Gödel.

**Definizione 7.6** (Teoria ricorsivamente (o semidecidibilmente) assiomatizzabile)**.** Una teoria $T$ è **ricorsivamente assiomatizzabile**, o **semidecidibilmente assiomatizzabile**, se:

$$T = \{\sigma : P(\ulcorner\sigma\urcorner) \neq \bot\}$$

con $P$ una funzione ricorsiva parziale, ovvero esiste un algoritmo che stabilisce se una formula è un assioma di $T$ o no.

In realtà, per via di un trucco, possiamo sempre supporre che l'insieme di assiomi sia addirittura decidibile.

**Esercizio 7.7** (Teorema di Craig)**.** Sia $T := \{\varphi : P(\ulcorner\varphi\urcorner) = 0\}$ ricorsivamente assiomatizzabile, allora esiste una teoria **decidibilmente assiomatizzabile**, i.e. l'insieme dei codici degli assiomi è decidibile, tale che $T$ e $T'$ sono equivalenti.

*Soluzione.* Per ipotesi, $T$ è descrivibile come insieme semidecidibile, quindi esiste una funzione ricorsiva totale $f$ tale che $T = \{\varphi : \exists n\, f(n) = \ulcorner\varphi\urcorner\}$, quindi possiamo supporre $T = \{\varphi_1, \varphi_2, \ldots\}$, WLOG $f(1) = \ulcorner\varphi_1\urcorner$, $f(2) = \ulcorner\varphi_2\urcorner$ e così via.
Ora, definiamo la nuova lista di assiomi di $T'$ come segue:

$$\begin{aligned}
\varphi'_1 &:= \varphi_1 \\
\varphi'_2 &:= \varphi_2 \wedge \varphi_2 \\
\varphi'_3 &:= \varphi_3 \wedge \varphi_3 \wedge \varphi_3 \\
&\vdots
\end{aligned}$$

si verifica banalmente, al livello sintattico, che $T$ è equivalente a $T'$. Inoltre, se poniamo $h(n) := \ulcorner\varphi'_n\urcorner$, la funzione $h : \mathbb{N} \to \mathbb{N}$ è ricorsiva totale (si ottiene da $f$ componendo con operazioni sintattiche ricorsive, cioè codifica e congiunzione iterata). Quindi posso definire il test totale ricorsivo degli assiomi di $T'$ come:

$$g(z) := \begin{cases} 0 & \text{se } \exists n \leq z\; h(n) = z \\ 1 & \text{altrimenti} \end{cases}$$

infatti $g(z) = 0$ se e solo se $z$ è codice di un assioma di $T'$ - la ricerca è limitata ai primi $z$ numeri perché altrimenti potrebbe non fermarsi e venire $\perp$[51]; inoltre $g$ è ricorsiva totale, in quanto è definita per casi su una funzione ricorsiva totale. $\square$

**Lemma 7.8** (Predicato di dimostrazione)
Sia $T$ una teoria aritmetica ricorsivamente assiomatizzabile, allora esiste un predicato PR $\mathsf{Dim}_T(x, y)$ tale che:

$$T \vdash \varphi \iff \mathbb{N} \models \exists d\, \mathsf{Dim}_T(d, \ulcorner\varphi\urcorner)$$

ovvero $T$ dimostra $\varphi$ se e solo se esiste un codice $d$ di una dimostrazione di $\varphi$ in $T$.

*Dimostrazione.* In primis diciamo come è codificata una dimostrazione, sappiamo che $T \vdash \varphi$ se esiste una sequenza finita di formule - anche se tecnicamente per noi è sempre una sequenza finita di sequenti[52] - con l'ultima che è $\varphi$ e quelle precedenti che sono o assiomi o si ottengono da quelle prima usando le regole di inferenza del sistema di deduzione naturale $\mathsf{ND}$, quindi una dimostrazione $\delta$ la si può codificare come la lista delle formule che la compongono:

$$\ulcorner\delta\urcorner := \langle \ulcorner\delta_1\urcorner, \ulcorner\delta_2\urcorner, \ldots, \ulcorner\delta_n\urcorner \rangle$$

[51] Andrebbe controllato inoltre che il predicato per la definizione per casi sia ricorsivo totale.

[52] Per essere precisi la codifica vera nel nostro sistema $\mathsf{ND}$ è la codifica di un sequente $\sigma_i = \ulcorner\varphi_1, \varphi_2, \ldots, \varphi_n \vdash \psi\urcorner$, che possiamo codificare come $\langle\langle\ulcorner\varphi_1\urcorner, \ulcorner\varphi_2\urcorner, \ldots, \ulcorner\varphi_n\urcorner\rangle, \ulcorner\psi\urcorner\rangle$, codificato un sequente in questo modo, codifichiamo la dimostrazione come lista di sequenti $\ulcorner\sigma_1, \sigma_2, \ldots, \sigma_m\urcorner$ come $\langle\ulcorner\sigma_1\urcorner, \ulcorner\sigma_2\urcorner, \ldots, \ulcorner\sigma_m\urcorner\rangle$ così abbiamo codificato esattamente la nostra definizione di dimostrazione.

dove $\delta_n$ è la formula finale, ovvero $\varphi$.
Ora possiamo definire $\mathsf{Dim}_T(d, x)$ traducendo il significato della definizione di dimostrazione in una definizione PR:

$$\begin{aligned}\mathsf{Dim}_T(d, x) := {}& \mathtt{nth}(d, \mathtt{length}(d) \dot{-} 1) = x \\ & \wedge \forall i < \mathtt{length}(d) \text{ “}\mathtt{nth}(d, i) \text{ è o un assioma di } T \text{ o segue da quelli prima...”}\end{aligned}$$

ora “$\mathtt{nth}(d, i)$ è un assioma” è esprimibile come predicato PR, perché per ipotesi $T$ è ricorsivamente assiomatizzabile, quindi esiste una funzione ricorsiva totale da mettere in quel posto; per “$\mathtt{nth}(d, i)$ segue dalle formule prima usando le regole di inferenza” basta scrivere un lungo OR che esprima tutti le possibili applicazioni delle regole di inferenza, ad esempio:

$$\mathrm{In}_{\to} \frac{T \vdash \varphi \quad T \vdash \varphi \to \psi}{T \vdash \psi}$$

equivale a:

$$\exists j < i \; \exists t < i \; \mathtt{nth}(d, t) = \mathsf{implica}(\mathtt{nth}(d, j), \mathtt{nth}(d, i))$$

dove $\mathsf{implica}$ è la funzione PR che prende i codici di due formule e restituisce il codice della formula che è l'implicazione tra le due - cioè $\langle \# \to, \mathtt{nth}(d, j), \mathtt{nth}(d, i) \rangle$ - e tutti i casi si fanno così (l'unico caso più complicato è quello di $\mathrm{In}_\forall$ dove bisogna controllare che la variabile libera sia effettivamente libera nelle premesse e quindi introdurre $\mathsf{free}(y, i) :=$ “$x_i$ è libera in $y$”). Abbiamo quindi definito $\mathsf{Dim}_T(d, x)$ come predicato PR via ricorsione e casi. □

## §7.3 Primo teorema di incompletezza

**Teorema 7.9** (Primo teorema di incompletezza di Gödel)
Sia $T$ una teoria aritmetica tale che:

(i) $T$ è ricorsivamente assiomatizzabile;

(ii) $T \vdash \mathsf{Q}$ e $\mathbb{N} \models T$;

allora $T$ è incompleta.

*Dimostrazione.* Lo scopo è costruire una formula $\gamma$ che dica “io non sono dimostrabile in $T$”, ovvero una formula che per $T$ è proprio equivalente al fatto che non esiste una dimostrazione per lei:

$$T \vdash \gamma \leftrightarrow \neg \exists d \, \mathsf{Dim}_T(d, \ulcorner \gamma \urcorner)$$

Per costruire $\gamma$ basta applicare il lemma di diagonalizzazione alla formula $\varphi(x) := \neg \exists d \, \mathsf{Dim}_T(d, x)$, e otteniamo $\gamma$, che per il lemma di diagonalizzazione soddisfa esattamente la condizione di cui sopra. Vediamo ora che $T$ non dimostra né $\gamma$ né $\neg\gamma$.

- Supponiamo $T \vdash \gamma$, allora esiste il codice di una dimostrazione di $\gamma$ in $T$, chiamiamolo $\delta$, quindi $\mathsf{Dim}_T(\ulcorner \delta \urcorner, \ulcorner \gamma \urcorner)$ è un predicato PR vero, che per il lemma di rappresentazione di funzioni ricorsive totali in $\mathsf{Q}$, significa che $\mathsf{Q} \vdash \mathsf{Dim}_T(\ulcorner \delta \urcorner, \ulcorner \gamma \urcorner)$, e quindi $T \vdash \mathsf{Dim}_T(\ulcorner \delta \urcorner, \ulcorner \gamma \urcorner)$ e per $\mathrm{In}_\exists$ si ha $T \vdash \exists d \, \mathsf{Dim}_T(d, \ulcorner \gamma \urcorner)$; ma $T$ dimostra anche $\gamma$, che per $T$ equivale a $\neg \exists d \, \mathsf{Dim}_T(d, \ulcorner \gamma \urcorner)$, quindi $T$ dimostra sia $\mathsf{Dim}_T(\ulcorner \delta \urcorner, \ulcorner \gamma \urcorner)$ che $\neg \exists d \, \mathsf{Dim}_T(d, \ulcorner \gamma \urcorner)$ $\lightning$, perché $T$ è coerente per ipotesi (ii).

- Supponiamo $T \vdash \neg\gamma$, allora, doppia negazione, $T$ dimostra $\exists d\, \mathsf{Dim}_T(d, \ulcorner\gamma\urcorner)$, ma allora per (ii) $\mathbb{N} \models \exists d\, \mathsf{Dim}_T(d, \ulcorner\gamma\urcorner)$, che per la semantica di Tarski equivale a dire che $\exists d \in \mathbb{N}\ \mathbb{N} \models \mathsf{Dim}_T(\overline{d}, \ulcorner\gamma\urcorner)$; segue che $d$ codifica una dimostrazione di $\gamma$ in $T$, per cui $T \vdash \gamma$, assurdo.

□

Ora questo teorema è un po' più forte del necessario e si possono indebolire le ipotesi per ottenere il teorema di Rosser.

**Teorema 7.10** (Teorema di Rosser)
Sia $T$ una teoria aritmetica tale che:

(i) $T$ è ricorsivamente assiomatizzabile;

(ii) $T \vdash \mathsf{Q}$;

(iii) $T$ è coerente;

allora $T$ è incompleta.

Nella dimostrazione del primo teorema di incompletezza di Gödel abbiamo fatto uso del fatto che $\mathbb{N}$ fosse modello di $T$, oltre alla coerenza di $T$, quindi quella dimostrazione non può essere riciclata.

*Dimostrazione* (IDEA). L'idea della dimostrazione, che non vediamo, è quella di usare una formula che dice "se ci fosse una mia dimostrazione, allora ci sarebbe anche una dimostrazione di una mia negazione più corta", ovvero:

$$\mathsf{Q} \vdash \varphi \leftrightarrow \forall d\, \mathsf{Dim}_T(d, \ulcorner\varphi\urcorner) \to \exists d' \leq d\, \mathsf{Dim}_T(d', \ulcorner\neg\varphi\urcorner)$$

e questo è un modo per ottenere comunque una formula che dice di sé stessa che è non dimostrabile - basta pensare alla negazione. □

**Definizione 7.11** (Teoria decidibile)**.** Una teoria $T$ è **decidibile** se l'insieme dei codici delle formule dimostrate da $T$ è decidibile, ovvero esiste una funzione ricorsiva totale $P$ tale che:

$$\{\ulcorner\sigma\urcorner : T \vdash \sigma\} = \{i \in \mathbb{N} : P(i) = 0\}$$

analogamente si può definire una teoria **semidecidibile**.

**Proposizione 7.12** (Completamento decidibile)
Sia $T$ una teoria coerente e decidibile, allora esiste un'estensione coerente $T'$ di $T$ che è completa e decidibile.

*Dimostrazione.* Per ipotesi possiamo fissare un'enumerazione delle formule chiuse $\{\varphi_i\}_{i\in\mathbb{N}}$ e definiamo per ricorsione:

$$T_0 := T$$

$$T_{i+1} := \begin{cases} T_i \cup \{\varphi_i\} & \text{se } T_i \nvdash \neg\varphi_i \\ T_i & \text{altrimenti} \end{cases}$$

cioè in ogni step se la $i$-esima teoria non dimostra la negazione dell'$i$-esima formula, allora la aggiungo, altrimenti lascio tutto com'è.
Ora, ponendo $T' := \bigcup_{i\in\mathbb{N}} T_i$, si verifica che $T'$ è un'estensione di $T$ coerente, infatti se $T'$ fosse incoerente, allora $T' \vdash \bot$, quindi per compattezza esisterebbe un sottoinsieme finito $T'' \subseteq T'$ tale che $T'' \vdash \bot$, per cui esiste un $i_0$ tale che $T'' \subseteq T_{i_0}$, ma allora $T_{i_0} \vdash \bot$, che è assurdo perché si verifica per induzione che ogni $T_i$ è coerente.
$T'$ è decidibile, presa $\{\sigma_j\}_{j\in\mathbb{N}}$ un'enumerazione di tutte le $L_{\text{arit}}$-formule, e fissato un algoritmo che decide se una formula è dimostrata da $T$ o no, basta testare, per $\sigma = \sigma_k$, per qualche $k > 0$, se $T_k \vdash \neg\sigma$ o no, e questo posso farlo perché $T_k = T \cup \{\varphi_0, \dots, \varphi_{k-1}\}$ e:

$$T_k \vdash \neg\sigma \iff T \vdash (\varphi_0 \wedge \dots \wedge \varphi_{k-1}) \to \neg\sigma$$

e questo è decidibile per ipotesi, se il risultato dell'algoritmo è falso, allora $T_{k+1} = T_k \cup \{\sigma\}$ e $T' \vdash \sigma$, altrimenti $T_{k+1} = T_k$ e $T' \vdash \neg\sigma$. Nel caso $k = 0$ la tesi è banale perché $T_0 = T$ è decidibile per ipotesi.
Infine, $T'$ è completa, fissata una enumerazione $\{\varphi_i\}_{i\in\mathbb{N}}$ di tutte le $L_{\text{arit}}$-formule, allora, per ogni $\varphi_i$, si ha che o $T_i \nvdash \neg\varphi_i$, e allora $T_{i+1} \vdash \varphi_i$, oppure $T_i \vdash \neg\varphi_i$, nel primo caso ottengo $T' \vdash \varphi_i$, nel secondo caso $T' \vdash \neg\varphi_i$, quindi $T'$ è completa. □

**Corollario 7.13** (Q e PA non sono decidibili)
Q e PA non sono decidibili.

*Dimostrazione.* Se Q fosse decidibile, allora per la proposizione precedente esisterebbe un'estensione coerente, completa e decidibile (in particolare ricorsivamente enumerabile e quindi ricorsivamente assiomatizzabile) di Q, che è impossibile per il teorema di Rosser. Idem per PA, che estende Q. □

**Corollario 7.14** (Indecidibilità della teoria vuota)
La teoria vuota nel linguaggio aritmetico non è decidibile.

*Dimostrazione.* Supponiamo per assurdo che $T = \varnothing$ sia decidibile, poiché Q è finitamente assiomatizzabile vale che:

$$\mathsf{Q} \vdash \varphi \iff T = \varnothing \vdash \mathsf{Q1} \wedge \dots \wedge \mathsf{Q7} \to \varphi$$

ovvero una formula è conseguenza di Q se e solo se l'implicazione con premessi gli assiomi di Q è conseguenza della teoria vuota; ma allora la decidibilità di $T$ implica la decidibilità di Q (e di ogni altra $L_{\text{arit}}$-teoria finitamente assiomatizzabile), assurdo per il corollario precedente. □

**Corollario 7.15** (Insieme delle verità aritmetiche)
L'insieme:

$$\{\ulcorner\varphi\urcorner : \mathbb{N} \models \varphi\}$$

non è ricorsivamente enumerabile.

*Dimostrazione.* È la teoria completa e coerente $\mathrm{Th}(\mathbb{N})$, che estende $\mathsf{Q}$, se fosse ricorsivamente enumerabile, sarebbe un controesempio al teorema di Rosser. $\square$

## §7.4 Secondo teorema di incompletezza

Il secondo teorema di incompletezza di Gödel è una conseguenza del primo, e dice che dentro la teoria $\mathsf{PA}$ non si può dimostrare la coerenza di $\mathsf{PA}$, e più in generale, se $T$ è una teoria aritmetica che estende $\mathsf{Q}$, allora $T$ non dimostra la propria coerenza. D'altronde una teoria incoerente dimostra tutto, quindi anche la propria coerenza, per cui il non dimostrare la propria coerenza è un buon segno.

**Notazione 7.16** (Formula di dimostrabilità) — Introduciamo l'abbreviazione $\square$ per indicare la formula che esprime la dimostrabilità di una proposizione nella teoria di Peano, $\mathsf{PA}$, ovvero:

$$\square\varphi := \exists d\, \mathsf{Dim}_{\mathsf{PA}}(d, \ulcorner\varphi\urcorner)$$

**Osservazione 7.17** — Con questa notazione, la formula $\gamma$ costruita nel primo teorema di incompletezza di Gödel è tale che:

$$\mathsf{PA} \vdash \gamma \leftrightarrow \neg\square\gamma$$

ovvero $\gamma$ dice di sé stessa che non è dimostrabile in $\mathsf{PA}$.

Il secondo teorema di incompletezza di Gödel si esprime nel seguente modo con questa notazione.

**Teorema 7.18** (Secondo teorema di incompletezza di Gödel)
$\mathsf{PA} \nvdash \neg\square\bot$, ovvero $\mathsf{PA}$ non dimostra la propria incoerenza.

Per dimostrare il teorema ci serve un lemma, che non dimostreremo.

**Lemma 7.19** (Proprietà della formula di dimostrabilità in PA)
Valgono le seguenti proprietà rispetto alla formula di dimostrabilità in $\mathsf{PA}$:

1. Se $\mathsf{PA} \vdash \varphi$, allora $\mathsf{PA} \vdash \square\varphi$, cioè dimostrare implica dimostrare di avere una dimostrazione;
2. $\mathsf{PA} \vdash \square\varphi \to \square\square\varphi$, cioè se dimostro di avere una dimostrazione, allora dimostro di avere una dimostrazione di avere una dimostrazione;
3. $\mathsf{PA} \vdash (\square\varphi \wedge \square(\varphi \to \psi)) \to \square\psi$, ovvero l'aritmetica di Peano capisce il modus ponens.

Usando il lemma possiamo concludere dimostrando il secondo teorema di incompletezza di Gödel.

*Dimostrazione.* Dal primo teorema di incompletezza di Gödel, esiste una formula $\gamma$ che è indecidibile in $\mathsf{PA}$, il nostro obiettivo è dimostrare che tale formula è equivalente a $\neg\square\bot$:

$$\mathsf{PA} \vdash \gamma \leftrightarrow \neg\square\bot$$

ed essendo, in particolare, che $\mathsf{PA} \nvdash \gamma$, allora $\mathsf{PA} \nvdash \neg\Box\bot$. La formula $\gamma$ è costruita in modo che $\mathsf{PA} \vdash \gamma \leftrightarrow \neg\Box\gamma$, usiamo questo fatto per dimostrare le equivalenze.

- $\mathsf{PA} \vdash \gamma \to \neg\Box\bot$, poiché $\mathsf{PA} \vdash \gamma \leftrightarrow \neg\Box\gamma$, allora è equivalente dimostrare $\mathsf{PA} \vdash \neg\Box\gamma \to \neg\Box\bot$. Procediamo dimostrando la contronominale, che è equivalente, ovvero $\mathsf{PA} \vdash \Box\bot \to \Box\gamma$, e questo segue per $\mathrm{El}_\to$:

$$\mathrm{El}_\to \frac{\mathsf{PA} \vdash \Box(\bot \to \gamma) \quad \mathsf{PA} \vdash (\Box\bot \wedge \Box(\bot \to \gamma)) \to \Box\gamma}{\mathsf{PA} \vdash \Box\bot \to \Box\gamma}$$

  dove $\mathsf{PA} \vdash \Box(\bot \to \gamma)$ perché $\mathsf{PA} \vdash \bot \to \gamma$ (è $\mathrm{El}_\bot$) unito con la (1); mentre $\mathsf{PA} \vdash (\Box\bot \wedge \Box(\bot \to \gamma)) \to \Box\gamma$ è la (2) del lemma.

- $\mathsf{PA} \vdash \neg\Box\bot \to \gamma$, come prima usiamo $\neg\Box\gamma$ al posto di $\gamma$, e dimostriamo $\mathsf{PA} \vdash \neg\Box\bot \to \neg\Box\gamma$ usando la contronominale. [COMPLETARE]

$\Box$

SISTEMARE

**Osservazione 7.20** (Osservazioni filosofiche sul secondo teorema di incompletezza) — Alla fine dei conti, la formula che esprime la coerenza di $\mathsf{PA}$, $\neg\Box\bot \equiv \exists d\, \neg\mathsf{Dim}_{\mathsf{PA}}(d, \ulcorner\bot\urcorner)$, è un numero naturale che noi scegliamo di leggere come la formula che dice "esiste una dimostrazione di $\bot$ in $\mathsf{PA}$", nulla esclude a priori che ci sia un'altro numero che corrisponde, in un'altra codifica, alla formula che dice che $\mathsf{PA}$ dimostra la propria stessa coerenza.
Non è detto, a priori, che se dovessimo scrivere una formula che dice "Peano è coerente" finiremmo sulla formula $\neg\Box\bot$.
Ad esempio, se definissimo Peano in questo modo, enumerando tutti gli assiomi e aggiungendo quelli di base ed uno alla volta quelli dello schema di induzione, fermandomi non appena arrivo al primo assioma che dà incoerenza; questa è una definizione del tutto corretta di Peano, perché sappiamo che di fatto non beccheremo mai un assioma che dà incoerenza, quindi gli assiomi ci sono tutti, e questa teoria sarà in grado di dimostrare la propria coerenza, perché è definita in modo che sia così - basterebbe usare compattezza.
Quindi il fatto che Peano non sia in grado di dimostrare la propria coerenza è un fatto che dipende da come è descritta Peano e la sua coerenza; se trovassimo un'altra descrizione di Peano e della sua coerenza magari riusciremmo a dimostrarla. Quello che alla fine abbiamo dimostrato noi è che, con le codifiche che abbiamo scelto, non riusciamo a dimostrare una certa formula, all'interno della quale leggo che Peano non può dimostrare la propria coerenza.
$\mathsf{ZF}$, che è più potente, non è in grado di dimostrare la propria coerenza, ma è in grado di dimostrare vari teoremi che caratterizzano la coerenza; ad esempio, se ci fosse un cardinale inaccessibile, allora $\mathsf{ZF}$ sarebbe coerente - e quindi $ZF$ non dimostra che c'è un cardinale inaccessibile; questa implicazione è proprio possibile grazie al fatto che ZF è abbastanza potente da poter connettere la formula che esprime la coerenza di ZF con l'esistenza di un cardinale inaccessibile; in generale non è detto che la teoria stessa sia in grado in generale di rendersi conto dei diversi modi di dire "$T$ è coerente".